

本篇笔记介绍了 OI 中基础的图论问题：最短路、最小生成树、双连通分量、欧拉回路。

定义与记号

涉及常见或可能用到的概念的定义。关于更多，见参考资料。

基本定义

- 图：一张图 G 由若干个点和连接这些点的边构成。点的集合称为 **点集** V ，边的集合称为 **边集** E ，记 $G = (V, E)$ 。
- 阶：图 G 的点数 $|V|$ 称为 **阶**，记作 $|G|$ 。
- 无向图：若 $e \in E$ 没有方向，则 G 称为 **无向图**。无向图的边记作 $e = (u, v)$ ， u, v 之间无序。
- 有向图：若 $e \in E$ 有方向，则 G 称为 **有向图**。有向图的边记作 $e = u \rightarrow v$ 或 $e = (u, v)$ ， u, v 之间有序。无向边 (u, v) 可视为两条有向边 $u \rightarrow v$ 和 $v \rightarrow u$ 。
- 重边：端点和方向（有向图）相同的边称为 **重边**。
- 自环：连接相同点的边称为 **自环**。

相邻

- 相邻：在无向图中，称 u, v **相邻** 当且仅当存在 $e = (u, v)$ 。
- 邻域：在无向图中，点 u 的 **邻域** 为所有与之相邻的点的集合，记作 $N(u)$ 。
- 邻边：在无向图中，与 u 相连的边 (u, v) 称为 u 的 **邻边**。
- 出边 / 入边：在有向图中，从 u 出发的边 $u \rightarrow v$ 称为 u 的 **出边**，到达 u 的边 $v \rightarrow u$ 称为 u 的 **入边**。
- 度数：一个点的 **度数** 为与之关联的边的数量，记作 $d(u)$ ， $d(u) = \sum_{e \in E} ([u = e_u] + [u = e_v])$ 。点的自环对其度数产生 2 的贡献。
- 出度 / 入度：在有向图中，从 u 出发的边数称为 u 的 **出度**，记作 $d^+(u)$ ；到达 u 的边数称为 u 的 **入度**，记作 $d^-(u)$ 。

路径

- 途径：连接一串相邻结点的序列称为 **途径**，用点序列 $v_{0..k}$ 和边序列 $e_{1..k}$ 描述，其中 $e_i = (v_{i-1}, v_i)$ 。常写为 $v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_k$ 。
- 迹：不经过重复边的途径称为 **迹**。
- 回路： $v_0 = v_k$ 的迹称为 **回路**。
- 路径：不经过重复点的迹称为 **路径**，也称 **简单路径**。不经过重复点比不经过重复边强，所以不经过重复点的途径也是路径。注意题目中的简单路径可能指迹。

- 环：除 $v_0 = v_k$ 外所有点互不相同的途径称为 **环**，也称 **圈** 或 **简单环**。

连通性

- 连通：对于无向图的两点 u, v ，若存在途径使得 $v_0 = u$ 且 $v_k = v$ ，则称 u, v **连通**。
- 弱连通：对于有向图的两点 u, v ，若将有向边改为无向边后 u, v 连通，则称 u, v **弱连通**。
- 连通图：任意两点连通的无向图称为 **连通图**。
- 弱连通图：任意两点弱连通的有向图称为 **弱连通图**。
- 可达：对于有向图的两点 u, v ，若存在途径使得 $v_0 = u$ 且 $v_k = v$ ，则称 u **可达** v ，记作 $u \rightsquigarrow v$ 。
- 关于点双连通 / 边双连通 / 强连通，见对应章节。

特殊图

- 简单图：不含重边和自环的图称为 **简单图**。
- 基图：将有向图的有向边替换为无向边得到的图称为该有向图的 **基图**。
- 有向无环图：不含环的有向图称为 **有向无环图**，简称 **DAG** (Directed Acyclic Graph)。
- 完全图：任意不同的两点之间恰有一条边的无向简单图称为 **完全图**。 n 阶完全图记作 K_n 。
- 树：不含环的无向连通图称为 **树**，树上度为 1 的点称为 **叶子**。树是简单图，满足 $|V| = |E| + 1$ 。若干棵（包括一棵）树组成的连通块称为 **森林**。相关知识点见“树论”。
- 稀疏图 / 稠密图： $|E|$ 远小于 $|V|^2$ 的图称为 **稀疏图**， $|E|$ 接近 $|V|^2$ 的图称为 **稠密图**。用于讨论时间复杂度为 $\mathcal{O}(|E|)$ 和 $\mathcal{O}(|V|^2)$ 的算法。

子图

- 子图：满足 $V' \subseteq V$ 且 $E' \subseteq E$ 的图 $G' = (V', E')$ 称为 $G = (V, E)$ 的 **子图**，记作 $G' \subseteq G$ 。要求 E' 所有边的两端均在 V' 中。
- 导出子图：选择若干个点以及两端都在该点集的所有边构成的子图称为该图的 **导出子图**。导出子图的形态仅由选择的点集 V' 决定，记作 $G[V']$ 。
- 生成子图： $|V'| = |V|$ 的子图称为 **生成子图**。
- 极大子图（分量）：在子图满足某性质的前提下，子图 G' 称为 **极大** 的，当且仅当不存在同样满足该性质的子图 G'' 且 $G' \subsetneq G'' \subseteq G$ 。 G' 称为满足该性质的 **分量**。例如，极大的连通的子图称为原图的连通分量，也就是我们熟知的连通块。

约定

- 记 n 表示点集大小 $|V|$ ， m 表示边集大小 $|E|$ 。

基本知识

读者需要掌握的知识：宽度优先搜索 (BFS)，深度优先搜索 (DFS)。

拓扑排序及其应用

对 DAG 进行拓扑排序，得到结点序列 $\{p_i\}$ ，满足图上每条边的起始点比终点在序列上更靠前。形式化地，设 q_i 表示结点 i 在 p 中的位置，那么对于图上每条边 $u \rightarrow v$ ，均有 $q_u < q_v$ 。

每次取入度为 0 的点 u 加入 p ，将从 u 出发的所有边删去。因为一条边被删去时，其起始点已经加入序列，而终点没有加入序列（入度大于 0），所以起始点相较终点在序列上更靠前。

拓扑排序没有结束时，必然存在入度为 0 的点。否则，从剩下的图上任意一点出发走每个点的第一条出边得到无限长的点序列。因为结点数有限，所以序列一定经过了重复的点，与原图无环矛盾。

通过 BFS 或 DFS 实现拓扑排序：

- BFS：用队列维护所有入度为 0 的点。取出队首 u ，删去从 u 出发的所有边 $u \rightarrow v$ 。如果删边导致某个点 v 从入度大于 0 变成入度等于 0，那么将 v 入队。初始将所有入度为 0 的点入队。
- DFS：对于当前点 u ，若 v 在删去 $u \rightarrow v$ 后从入度大于 0 变成入度等于 0，那么向 v DFS。初始从每个入度为 0 的点开始搜索。

时间复杂度为 $\mathcal{O}(n + m)$ 。

模板题 代码。

```
#include <bits/stdc++.h>
using namespace std;
constexpr int N = 100 + 5;
int n, deg[N];
vector<int> e[N];
int main() {
    cin >> n;
    for(int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        while(x) e[i].push_back(x), deg[x]++, cin >> x;
    }
    queue<int> q;
    for(int i = 1; i <= n; i++) if(!deg[i]) q.push(i);
    while(!q.empty()) {
        int t = q.front();
        q.pop();
        cout << t << " ";
        for(int it : e[t]) if(--deg[it]) q.push(it);
    }
    return 0;
}
```

拓扑序常用于解决建图题或图论类型的构造题。

拓扑序不唯一，例如图 $(1 \rightarrow 2, 1 \rightarrow 3)$ 有两个拓扑序 $\{1, 2, 3\}$ 和 $\{1, 3, 2\}$ 。除非有特殊性质，拓扑序计数无法做到多项式复杂度。

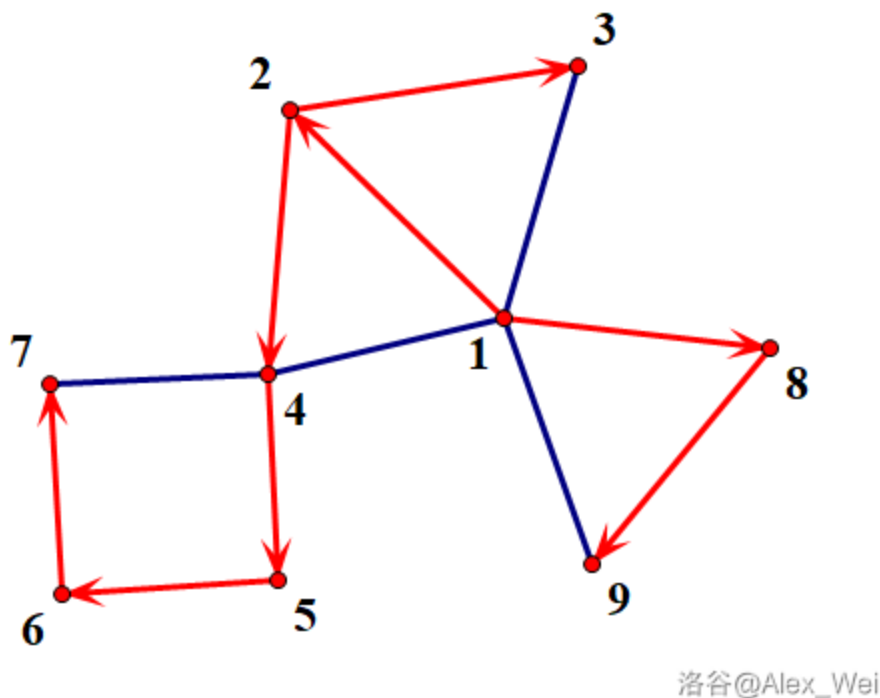
拓扑排序衍生出的各种问题：

- 拓扑序 DP，即在拓扑序上通过 DP 的方式统计答案。
 - DAG 最长路。设 f_i 表示以 i 结尾的最长路径长度，按拓扑序转移： $f_v \leftarrow \max(f_v, f_u + 1)$ 。例题：P1137，P3573（第一章例题），P3387（4.3 小节的模板题）。
 - DAG 路径计数。设 f_i 表示以 i 结尾的路径条数，按拓扑序转移： $f_v \leftarrow f_v + f_u$ 。例题：P3183，P4017。
 - 点对可达性统计。设 $f_{i,j}$ 表示 i 是否可达 j ，按拓扑序转移： $f_{v,j} \leftarrow f_{v,j} \vee f_{u,j}$ ，用 bitset 优化，时间复杂度 $\mathcal{O}(\frac{|V||E|}{w})$ 。例题：P7877（“建图相关”例题），[HDU7171](#)（[HDU 多校 2022 比赛记录](#) Round 3 1010）。
- 最小 / 最大字典序拓扑序：在 BFS 求拓扑序时，将队列换成优先队列。可归纳证明每一步都取到了字典序最小值。例题：P3243，CF1765H（[CF 合集 1751-1775](#)）。
- 树拓扑序计数，拓扑序容斥：见“计数 DP”。
- 结合有向图强连通分量缩点，缩点后拓扑排序，解决一般有向图上的问题。

无向图 DFS 树

给定无向连通图 G ，从点 r 开始 DFS，取出进入每个点 i 时对应的边 (fa_i, i) 并定向为 $fa_i \rightarrow i$ ，得到以 r 为根的叶向树。称 (fa_i, i) 为 **树边**，其它边为 **非树边**。

给每个点标号为它被访问到的次序，称为 **时间戳**，简称 dfn。按 DFS 的访问顺序得到的结点序列称为 **DFS 序**，时间戳为 i 的结点在 DFS 序上的位置为 i 。



上图是一个可能的 DFS 树以及对应的时间戳。

无向图 DFS 树的性质：

- 祖先后代性：任意非树边两端具有祖先后代关系。
- 子树独立性：结点的每个儿子的子树之间没有边（和上一条性质等价）。
- 时间戳区间性：子树时间戳为一段区间。
- 时间戳单调性：结点的时间戳小于其子树内结点的时间戳。

正确性留给读者思考。

关于有向图 DFS 树，见第四章。

1. 最短路及其应用

最短路是图论的一类经典问题。最优化 DP 在有环的图上转移就是最短路。

1.1 相关定义

在研究最短路问题（Shortest Path Problem, SPP）时，研究对象是路径而非途径，因为总可以钦定两点之间的最短路不经过重复点，否则起点和终点之间存在负环，最短路不存在。

- 带权图：每条边带有权值的图称为 **带权图（赋权图）**。所有边的权值非负的图称为 **非负权图**；所有边的权值为正的图称为 **正权图**。

- 边权：边的权值称为 **边权**，记作 w_e 或 $w_{u,v}$ 。带权边记作 (u, v, w) 。若边不带权，默认边权为 1。
- 路径长度：路径上每条边的权值之和称为 **路径长度**。
- 负环：长度为负数的环称为 **负环**。
- 最短路：在一张图上，称 s 到 t 的 **最短路** 为最短的连接 s 到 t 的路径。若不存在这样的路径（不连通或不可达），或最小值不存在（存在可经过的负环），则最短路不存在。
- 记 s 表示最短路起点， t 表示最短路终点。

1.2 单源最短路径问题

问题描述：给定 **源点** s ，求 s 到图上每个点 u 的最短路长度 D_u 。

设 dis_u 表示 s 到 u 的估计最短路长度，初始化 $dis_s = 0$ 和 $dis_u = +\infty$ ($u \neq s$)，算法结束时应有 $dis_u = D_u$ 。

接下来介绍该问题的几种常见解法。

1.2.1 Bellman-Ford

Bellman-Ford 是一种暴力求解单源最短路径的方法。

称一轮 **松弛** 表示对每条边 (u, v) ，用 $dis_u + w_{u,v}$ 更新 dis_v 。松弛 $n - 1$ 轮即可。

证明

在 $s \rightsquigarrow u$ 的最短路 $s \rightarrow p_1 \rightarrow \cdots \rightarrow u$ 中，对于每个点 p_i ， $s \rightarrow p_1 \rightarrow \cdots \rightarrow p_i$ 一定是 $s \rightsquigarrow p_i$ 的最短路。这说明一个点的最短路由另一个点的最短路扩展而来。

因为最短路至多有 $n - 1$ 条边，而第 i 轮松弛会得到边数为 i 的最短路，故只需松弛 $n - 1$ 轮。

□

该算法可以判断一张图上是否存在负环：若第 n 轮松弛时仍有结点的最短路被更新，则图上存在负环。

算法的时间复杂度为 $\mathcal{O}(nm)$ 。

1.2.2 Dijkstra

Dijkstra 算法适用于 **非负权图**。网上的一些博客认为 Dijkstra 使用了贪心思想，笔者表示怀疑。

称 **扩展** 结点 u 表示对 u 的所有邻边 (u, v) ，用 $dis_u + w_{u,v}$ 更新 dis_v 。

在 $dis_u = D_u$ 的结点中，取出 dis 最小且未扩展过的点并扩展。因为没有负权边，所以取出结点的最短路长度单调不降。

如何判断一个点的 dis_u 已经等于 D_u ? 实际上, 当前 dis 最小且未扩展过的点一定满足 $dis_u = D_u$ 。

证明

归纳假设已经扩展过的结点 $p_1, p_2, \dots, p_{k-1}$ 在扩展时取到了最短路。 p_k 为未扩展的 dis 最小的结点。

p_k 的最短路一定由 p_i ($1 \leq i < k$) 的最短路扩展而来, 不可能出现

$$dis(p_i) + w(p_i, u_1) + w(u_1, u_2) + \dots + w(u_c, p_k) < dis(p_j) + w(p_j, p_k)$$

的情况, 其中 $u_1 \sim_c$ 均未扩展。 否则由于边权非负, $dis(p_i) + w(p_i, u_1) < dis(p_j) + w(p_j, p_k)$, 即当前 $dis(u_1) < dis(p_k)$, 与 $dis(p_k)$ 的最小性矛盾。

初始令源点的 dis 为 0, 假设成立, 算法正确。 $\square$

简单地说, 一个点的最短路由长度非严格更小的最短路更新得到, dis 最小的结点一定不会被其它结点更新。

取出 dis 最小的结点可用优先队列维护。 扩展 u 时, 若 $dis_u + w_{u,v} < dis_v$, 那么将 $(v, dis_u + w_{u,v})$ 即结点编号和它更新后的 dis 作为二元组丢进优先队列。 尝试取出结点时, 以 $dis_u + w_{u,v}$ 为关键字取出最小的二元组, 扩展该二元组对应的结点。

注意, 一个结点可能有多个入边并多次进入优先队列, 但当它第一次被取出时, 对应的 dis 一定为最短路。 为此, 需判断是否有第二元 (距离) 等于第一元 (编号) 的 dis 。 若不等, 说明该点已扩展过, 直接跳过, 否则复杂度将退化为 $\mathcal{O}(m^2 \log m)$ 。

算法的时间复杂度为 $\mathcal{O}(m \log m)$ 。 当图为稠密图 (m 接近 n^2) 时, 用暴力代替优先队列, 时间复杂度 $\mathcal{O}(n^2)$ 。

扩展: 当边权只有 0 和 1 时, 使用双端队列代替优先队列: 成功扩展权值为 0 的出边时压入队首, 成功扩展权值为 1 的出边时压入队尾。 称为 **01 BFS**。

[模板题](#) 代码。

```

#include <bits/stdc++.h>
using namespace std;
#define pii pair<int, int>
const int N = 1e5 + 5;
int n, m, s, dis[N];
vector<pii> e[N];
int main() {
    cin >> n >> m >> s;
    for(int i = 1; i <= m; i++) {
        int u, v, w;
        scanf("%d%d%d", &u, &v, &w);
        e[u].push_back(make_pair(v, w));
    }
    memset(dis, 0x3f, sizeof(dis)), dis[s] = 0; // 初始化
    priority_queue<pii, vector<pii>, greater<pii>> q; // 优先取权值较小的 pair
    q.push(make_pair(0, s)); // 注意第一关键字要放到前面
    while(!q.empty()) {
        auto t = q.top();
        q.pop();
        int id = t.second; // 不要搞反了, 编号是 second
        if(t.first != dis[id]) continue; // 若权值不等于最短路, 说明已扩展, 跳过
        for(auto _ : e[id]) {
            int it = _.first, d = t.first + _.second;
            if(d < dis[it]) q.push(make_pair(dis[it] = d, it));
        }
    }
    for(int i = 1; i <= n; i++) cout << dis[i] << " ";
    return 0;
}

```

1.2.3 SPFA 与负环

关于 SPFA, ____。

SPFA (Shortest Path Faster Algorithm) 是队列优化的 Bellman-Ford。

松弛点 x 时找到接下来可能松弛的点, 即与 x 相邻且 **最短路被更新的点** 并压入队列。此外, 记录一个点是否在队列中, 若是, 则不压入, 可以显著减小常数。

时间复杂度相比 BF 并没有改进, 仍为 $\mathcal{O}(nm)$ 。在一般图上效率很高, 但可以被特殊数据卡成平方, 所以能用 Dijkstra (非负权图) 时不建议 SPFA。

注意: 若使用 SPFA 求解点对点的最短路径, 如费用流 EK, 当队头为目标结点时不能结束算法。因为一个点进入队列并不代表其 dis 已经取到了最短路长度。

SPFA 判负环: 若一个点 **进入队列** 超过 $n - 1$ 次 (不是被松弛, 因为一个点被松弛不一定进入队列), 或 **最短路边数** 大于 $n - 1$, 则整张图存在 **从源点可达** 的负环。对于后者, 记录 l_i 表示从源点到 i 的

最短路长度，松弛时令 $l_v \leftarrow l_u + 1$ 并判断是否有 $l_v < n$ 。

一般判入队次数慢于最短路长度，推荐使用后者。

模板题 代码。

```
#include <bits/stdc++.h>
using namespace std;
const int N = 2e3 + 5;
int n, m, dis[N], len[N], vis[N];
vector<pair<int, int>> e[N];
void solve() {
    cin >> n >> m;
    for(int i = 1; i <= n; i++) e[i].clear();
    for(int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        e[u].push_back(make_pair(v, w));
        if(w >= 0) e[v].push_back({u, w});
    }
    queue<int> q;
    memset(dis, 0x3f, sizeof(dis));
    memset(vis, 0, sizeof(vis));
    q.push(1), len[1] = dis[1] = 0;
    while(!q.empty()) {
        int t = q.front();
        q.pop(), vis[t] = 0;
        for(auto it : e[t]) {
            int d = dis[t] + it.second, to = it.first;
            if(d < dis[to]) {
                dis[to] = d, len[to] = len[t] + 1; // 更新时修改最短路长度
                if(len[to] == n) return puts("YES"), void();
                if(!vis[to]) vis[to] = 1, q.push(to);
            }
        }
    }
    puts("NO");
}
int main() {
    int T;
    cin >> T;
    while(T--) solve();
    return 0;
}
```

1.2.4 三角形不等式

在单源最短路径问题中，对于每条边 $(u, v) \in E$ ，有 $D_u + w_{u,v} \geq D_v$ ，称为 **三角形不等式**。

三角形不等式是每个点的单源最短路径长度满足的性质。相反，给出若干条三角形不等式的限制，求能否找到一组满足条件的解，就是经典的 **差分约束** 问题。

1.3 差分约束问题

问题描述：给定若干形如 $x_a - x_b \leq c$ 或 $x_a - x_b \geq c$ 的不等式限制，求任意一组解 $\{x_i\}$ 。

1.3.1 算法介绍

只要 x_a, x_b 在不等号同一侧时符号不同，那么所有限制均可写为 $x_i + c \geq x_j$ 。

通过单源最短路径求出的 dis 满足三角形不等式，每条边对应一个不等式。这启发我们将给出的不等式还原成边，在建出的图上跑单源最短路径。

从 $i \rightarrow j$ 连长度为 c 的边，再从超级源点 0 向每个点连长度为 0 的边防止图不连通，或初始令所有 $dis = 0$ 并将所有点入队，每个点的最短路长度就是一组解。

因为 c 可能为负值（若 c 非负，令所有 x 相等即可），所以使用 Bellman-Ford 或 SPFA 求解最短路。若出现负环则无解，这说明不等式要求一个数加上负数后不小于其本身。

时间复杂度 $\mathcal{O}(nm)$ 。

[模板题](#) 代码。

```

#include <bits/stdc++.h>
using namespace std;
const int N = 5e3 + 5;
int n, m, vis[N], dis[N], len[N];
vector<pair<int, int>> e[N];
int main() {
    cin >> n >> m;
    for(int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        e[v].push_back(make_pair(u, w));
    }
    queue<int> q;
    for(int i = 1; i <= n; i++) q.push(i), vis[i] = 1;
    while(!q.empty()) {
        int t = q.front();
        q.pop(), vis[t] = 0;
        for(auto it : e[t]) {
            int to = it.first, d = dis[t] + it.second;
            if(d < dis[to]) {
                dis[to] = d, len[to] = len[t] + 1;
                if(len[to] == n) puts("NO"), exit(0);
                if(!vis[to]) q.push(to), vis[to] = 1;
            }
        }
    }
    for(int i = 1; i <= n; i++) cout << dis[i] << " ";
    return 0;
}

```

例题：P4926，P5590，**P3530**。

1.3.2 解的字典序极值

一般而言差分约束系统的解没有“字典序极值”这个概念，因为我们只对变量之间的差值进行约束，而变量本身值随着某个变量取值的固定而固定。

字典序的极值建立于变量有界的基础上。不妨设希望求出当限制 $x_i \leq 0$ 时，整个差分约束系统的字典序的最大值。注意到 $x_i \leq 0$ 可以看成三角形不等式 $(x_0 = 0) + 0 \geq x_i$ ，所以我们建立虚点 0，将其初始 dis 赋为 0，并向其它所有变量连权值为 0 的边（等价于初始令所有点入队且 $dis = 0$ ）。

求解差分约束系统时我们使用了三角形不等式，将其转化为最短路问题。这给予它很好的性质：**通过 SPFA 求得的解恰为字典序最大解**。

对于一条 $u \rightarrow v$ 的边权为 $w(u, v)$ 的边，它的含义为限制 $x_u + w(u, v) \geq x_v$ 。

考虑 0 到 i 的最短路。因为最短路上的所有边均有 $x_u + w(u, v) = x_v$ ，所以如果将 x_i 增大 1，那么最短路至少有一条边的限制无法被满足。这说明每个变量都取到了其上界，自然满足字典序最大。

对于字典序最小解，限制 $x_i \geq 0$ 。所有结点向 0 连边 $w(i, 0) = 0$ 。字典序最小即字典序最大时的相反数。因此，考虑将所有变量取反，那么限制也要全部取反，体现为将图中所有边（包括所有新建的边 $i \rightarrow 0$ ）的 **方向**（而不是边权）取反： $-x_u + w(u, v) \geq -x_v \implies x_v + w(u, v) \geq x_u$ 。然后求字典序最大解，再取相反数即可。

1.4 全源最短路径问题

问题描述：求任意两点之间的最短路 $D_{s,t}$ 。注意有向图中 s, t 有序。

1.4.1 Johnson

前置知识：Bellman-Ford，Dijkstra。

Johnson 算法用于解决 **带有负权边** 的全源最短路径问题。

Johnson 算法的巧妙之处在于为每个点赋予 **势能** h_i 。正如物理意义上的势能，从一个点出发到达另一个点，无论走什么路径，势能的总变化量是一定的。这启发我们将边 (u, v) 的新权值 $w'_{u,v}$ 设置为 $w_{u,v} + h_u - h_v$ 。

考虑路径 $S \rightarrow p_1 \rightarrow p_2 \rightarrow \dots \rightarrow T$ ，原长度为

$$L(S \rightsquigarrow T) = w_{S,p_1} + w_{p_1,p_2} + \dots + w_{p_l,T}$$

新长度为

$$L'(S \rightsquigarrow T) = (h_S + w_{S,p_1} - h_{p_1}) + (h_{p_1} + w_{p_1,p_2} - h_{p_2}) + \dots + (h_{p_l} + w_{p_l,T} - h_T)$$

则 $L(S \rightsquigarrow T) = L'(S \rightsquigarrow T) + h_T - h_S$ 。

对于固定的 S, T ，原图的路径对应到新图上面，长度增加了 $h_S - h_T$ 。这与路径经过了哪些点无关，只与 S, T 有关。因此，**原图 $S \rightsquigarrow T$ 的最短路在新图上仍为最短路**。

由于负权，我们无法使用 Dijkstra 求解最短路。多次 Bellman-Ford 或 SPFA 的时间复杂度不优秀。尝试应用上述分析，合理地为每个点赋权值，从而消去图上的所有负权边。

为使 $w'(u, v) \geq 0$ ，只需 $w(u, v) + h_u \geq h_v$ ，求解差分约束即可：若初始图中无负环，则 h 一定存在。令所有 h 等于 0，然后使用 Bellman-Ford 进行 $n - 1$ 轮松弛。松弛在 $n - 1$ 轮后必然结束，否则原图存在负环。

操作结束后，更新每条边的权值 $w'(u, v) = w(u, v) + h_u - h_v$ 。根据一开始的分析，得到的新图与原图任意两点最短路经过的结点不变，且无负边权，可以使用 n 轮 Dijkstra 求解。

最后不要忘了将 $u \rightsquigarrow v$ 的最短路加上 $h_v - h_u$ 。

算法的时间复杂度为 $\mathcal{O}(nm \log m)$ 。

模板题 代码。

```
#include <bits/stdc++.h>
using namespace std;
#define pii pair<int, int>
const int N = 3e3 + 5;
int n, m, h[N], dis[N];
vector<pii> e[N];
int main() {
    cin >> n >> m;
    for(int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        e[u].push_back(make_pair(v, w));
    }
    for(int i = 1; i <= n; i++) {
        bool found = 0;
        for(int j = 1; j <= n; j++)
            for(auto it : e[j])
                if(h[j] + it.second < h[it.first])
                    found = 1, h[it.first] = h[j] + it.second;
        if(i == n && found) puts("-1"), exit(0);
    }
    for(int i = 1; i <= n; i++) {
        long long ans = 0;
        for(int j = 1; j <= n; j++) dis[j] = i == j ? 0 : 1e9;
        priority_queue<pii, vector<pii>, greater<pii>> q;
        q.push({0, i});
        while(!q.empty()) {
            auto t = q.top(); q.pop();
            int id = t.second;
            if(t.first != dis[id]) continue;
            for(auto _ : e[id]) {
                int it = _.first, d = t.first + h[id] - h[it] + _.second;
                if(d < dis[it]) q.push({dis[it] = d, it});
            }
        }
        for(int j = 1; j <= n; j++) ans += 1ll * j * (dis[j] + (dis[j] < 1e9 ? h[j] - h[i] : 0));
        cout << ans << endl;
    }
    return 0;
}
```

1.4.2 Floyd 与传递闭包

Floyd 也可以解决带负权边的图的全源最短路径问题。

设 $dis_{k,s,t}$ 表示 $s \rightsquigarrow t$ 只经过编号不大于 k 的点（两端除外）的最短路。从 f_k 推到 f_{k+1} 时，只需考虑往两点之间的最短路当中插入 $k+1$ ，即枚举 i, j ，用 $dis_{k,i,k+1} + dis_{k,k+1,j}$ 更新 $dis_{k+1,i,j}$ 。

省去第一维导致的重复更新不影响答案，因为最短路不经过重复点。简化后的算法描述为：初始化所有 $dis_{u,u} = 0$ ，以及 $(u \rightarrow v) \in E$ 的 $dis_{u,v}$ 为 $w_{u,v}$ ，其它为无穷大。枚举中转点 k ，起点 i ，终点 j ，用 $dis_{i,k} + dis_{k,j}$ 更新 $dis_{i,j}$ 。注意 **中转点必须在最外层枚举**。

很明显，按任意顺序枚举中转点，甚至枚举相同的中转点，只要每个点至少一次作为中转点，依然能得到正确的答案。

算法的时间复杂度为 $\mathcal{O}(n^3)$ 。不仅好写，而且在稠密图上运行效率高于 Johnson，因此 Floyd 是数据规模较小时最短路算法的最佳选择。

此外，Floyd 可以求传递闭包。**有向图 G 的传递闭包** 定义为 n 阶布尔矩阵 T ，满足 $T_{i,j} = 1$ 当且仅当 i 可达 j ，否则为 0。有边的点对之间初始化 $T_{u,v} = 1$ ，转移时内层操作为 $T_{i,j} \leftarrow T_{i,j} \vee (T_{i,k} \wedge T_{k,j})$ ，其中 $\vee$ 表示逻辑或， $\wedge$ 表示逻辑与。

- bitset 优化求传递闭包的复杂度至 $\mathcal{O}(\frac{n^3}{w})$ 。
- 对于稀疏有向图，缩点后拓扑排序可做到 $\mathcal{O}(\frac{nm}{w})$ 求传递闭包。

1.5 扩展问题

图（弱）连通且无负环。

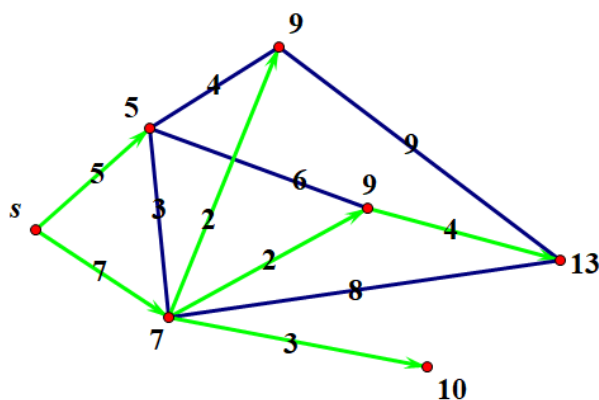
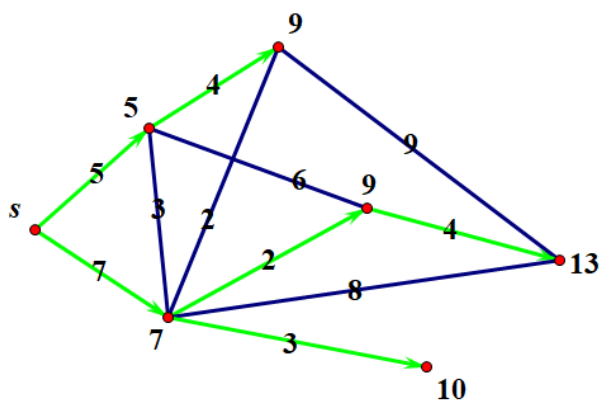
1.5.1 最短路树

从 s 开始求图上每个点的单源最短路径。对于 $i \neq s$ ， i 的最短路由点 j 扩展而来，满足 $dis_j + w(j,i) = dis_i$ ，即 $s \rightsquigarrow i$ 的最短路是 $s \rightsquigarrow j$ 的最短路加入边 (j,i) 得到。我们希望用一个结构刻画 s 到每个点的最短路。

在单源最短路径过程中，记录每个点最后一次被更新最短路的对应前驱 f_i 。即若当前松弛边 (j,i) 且 $dis_j + w(j,i) < dis_i$ ，则令 $f_i \leftarrow j$ 。除了源点 s ，从 f_i 向 i 连边，则形成的有向图无环。

这样，得到一棵有根叶向树，称为 **从 s 出发的最短路树**，每个点的 f_i 为其父亲。最短路树上从 s 出发到 i 的简单路径就是原图 $s \rightsquigarrow i$ 的最短路。

一张图的最短路树不唯一，如下图所示（无向图）。



洛谷@Alex_Wei

最短路树以外的所有边不影响 s 到各个点的最短路，可以忽略。

类似地，将图上所有边翻转方向后求得的以 s 为根的最短路树，就是原图 **到达 s 的最短路树**，它是一棵根向树。无向图不区分边的方向，所以从 s 出发的最短路树也可以是到达 s 的最短路树，但有向图并不一定。

最短路树在求解单源最短路径问题的变形时发挥了很大的作用，如接下来介绍的删边最短路。

扩展：在 **不含零环的图上** 将 $dis_j + w(j, i) = dis_i$ 的 $j \rightarrow i$ 加入结构，得到一张有向无环图，称为 **最短路 DAG**。最短路计数相当于 DAG 路径计数，拓扑排序后 DP。最短路 DAG 的任意一棵生成树均为最短路树。最短路树计数相当于 DAG 生成树计数，使用矩阵树定理求解。

例题：P6880。

*1.5.2 删边最短路

问题描述：给定一张 **无向正权图**，对图上的每条边，求删去该边后 $1 \rightsquigarrow n$ 的最短路。

首先求出 $1 \rightsquigarrow n$ 的最短路 $P = p_0(1) \xrightarrow{e_1} p_1 \xrightarrow{e_2} \dots \xrightarrow{e_L} p_L(n)$ 。设 e 表示当前考虑的边 (i, j) 。

若 $e \notin P$ ，答案是显然的：删去一条边后最短路不可能变短，故 $1 \rightsquigarrow n$ 的最短路仍为 $w(P)$ 。

若 $e \in P$ ，考虑求出 **从 1 出发** 和 **到达 n** 的最短路树 T_1 和 T_n ，要求 T_1 上 $1 \rightsquigarrow n$ 的路径等于 T_n 上 $1 \rightsquigarrow n$ 的路径等于 P 。枚举每一条边 (u, v) （注意一条边可以从两个方向经过）。若 $T_1(1 \rightsquigarrow u)$ 和 $T_n(v \rightsquigarrow n)$ 均不包含 (i, j) ，那么用 $w(T_1(1 \rightsquigarrow u)) + w(u \rightarrow v) + w(T_n(v \rightsquigarrow n))$ 更新最短路。这个做法本质上是 **存在最短路只走一条“非树边”**，指：这条非树边之前的路径全部在 T_1 上，之后的路径全部在 T_n 上。

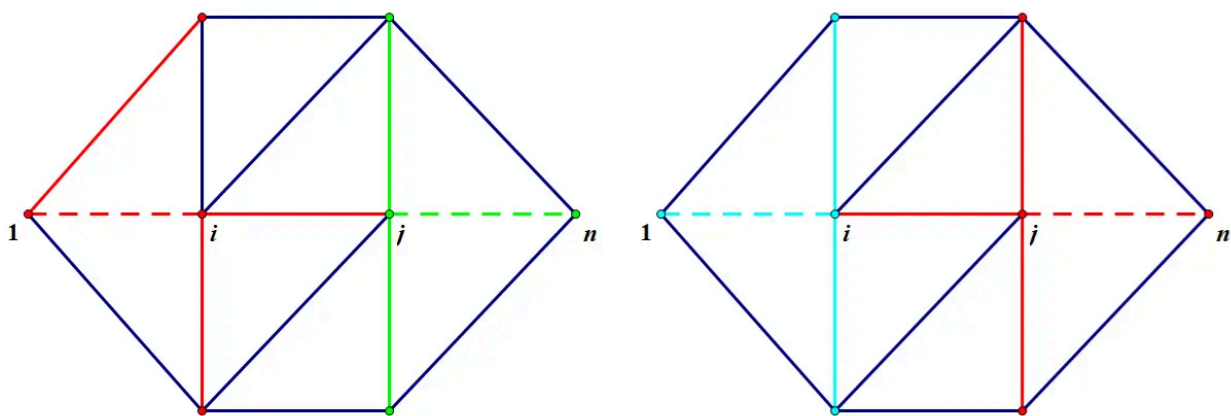
证明

不妨设 i, j 在 P 上的顺序为 $i \rightarrow j$ 。

设 A_j 和 A_i 分别表示 T_1 上 j 的子树和 j 的子树以外的部分。设 B_i 和 B_j 分别表示 T_n 上 i 的子树和 i 的子树以外的部分。

引理

$$A_j \cap B_i = \emptyset.$$



洛谷@Alex_Wei

如上图，绿色部分的 A_j 和蓝色部分的 B_i 无交。

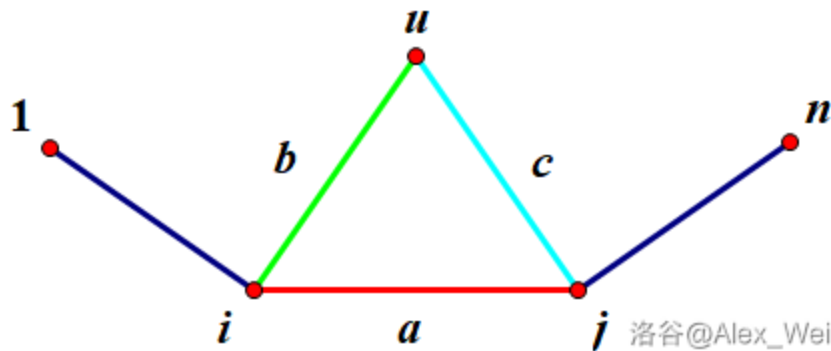
证明

假设存在 $u \in A_j \cap B_i$ 。因为 $e \in P \subseteq T_1$ ，所以 $1 \rightsquigarrow u$ 的最短路形如 $1 \rightsquigarrow i \rightarrow j \rightsquigarrow u$ （在 T_1 上）。因为 $e \in P \subseteq T_n$ ，所以 $u \rightsquigarrow n$ 的最短路形如 $u \rightsquigarrow i \rightarrow j \rightsquigarrow n$ （在 T_n 上）。

根据最短路的性质，有 $w(T_1(i \rightarrow j \rightsquigarrow u)) \leq w(T_n(i \rightsquigarrow u))$ 以及 $w(T_n(u \rightsquigarrow i \rightarrow j)) \leq w(T_1(u \rightsquigarrow j))$ 。注意这里将最短路树上的路径进行了翻转，这用到了 **原图为无向图** 的性质（ $w(T_n(u \rightsquigarrow i)) = w(T_n(i \rightsquigarrow u))$ ）。对于有向图，不等式右侧不一定存在。

将左侧路径拆开，右侧路径翻转，得 $w(i \rightarrow j) + w(T_1(j \rightsquigarrow u)) \leq w(T_n(u \rightsquigarrow i))$ 以及 $w(T_n(u \rightsquigarrow i)) + w(i \rightarrow j) \leq w(T_1(j \rightsquigarrow u))$ 。

两式相加，得 $2w(i \rightarrow j) \leq 0$ ，与正权图矛盾。□



简单地说，根据 $u \in A_j \cap B_i$ 可以推出 $a + c \leq b$ 和 $a + b \leq c$ ，得 $2a \leq 0$ 。如上图。

考虑删去 $e = (i, j)$ 后任意一条 $1 \rightarrow n$ 的最短路 P' 。因为 $1 \in A_i$, $n \in A_j$ ，所以存在 $e' = (u, v) \in P'$ 使得 $u \in A_i$, $v \in A_j$ 。不妨设 e' 是路径上最后一条这样的边。

将 P' 上 $1 \rightsquigarrow u$ 的部分替换成 $T_1(1 \rightsquigarrow u)$ 一定不劣，因为后者是 $1 \rightsquigarrow u$ 的最短路。又因为 $u \in A_i$ ，所以这部分一定不会包含 $i \rightarrow j$ 。

将 P' 上 $v \rightsquigarrow n$ 的部分替换成 $T_n(v \rightsquigarrow n)$ 一定不劣，因为后者是 $v \rightsquigarrow n$ 的最短路。又因为 $v \in A_j$ ，根据引理， $v \notin B_i$ ，即 $v \in B_j$ ，所以这部分同样不会包含 $i \rightarrow j$ 。

这样，对于删去 e 后的任意最短路，总存在调整它的方案使得恰好只经过 e' 一条非树边。而枚举每条非树边计算最短路长度并更新的过程考虑到了所有恰好只经过一条非树边的路径，相对应的也就考虑到了所有最短路。算法正确性得证。

尽管如此，对一条边求答案仍需要 $\mathcal{O}(m)$ 的时间。

枚举 $e = (u, v)$ ，求出 T_1 上 n 和 u 的 LCA，即最短路 $1 \rightsquigarrow n$ 和 $1 \rightsquigarrow u$ 的最后一个公共交点，设为 u' 。类似地，设 $v' = T_n(\text{lca}(v, 1))$ 。则 $w(T_1(1 \rightsquigarrow u)) + w(u \rightarrow v) + w(T_n(v \rightsquigarrow n))$ 可以用于更新 P 上 $u' \rightsquigarrow v'$ 之间所有边的答案。正确性留给读者思考。

线段树维护区间取 min 或 multiset 离线扫描线，时间复杂度 $\mathcal{O}(m \log m)$ 。

扩展：一开始我们要求 $P \in T_1 \cap T_n$ ，实际上这是不必要的。根据 T_1 求出 P 之后，可以将 v' 的定义改为 $T_n(v \rightsquigarrow n)$ 和 P 的第一个交点而不影响答案，因为将 $P(v' \rightsquigarrow n)$ 接在 $T_n(v \rightsquigarrow v')$ 之后一定不劣，相当于手动钦定了 $P \in T_1 \cap T_n$ 。

扩展：对于存在零权边的非负权图，导出矛盾的 $2w(i \rightarrow j) \leq 0$ 失效了。做出修正：求出 u' 后，若 u' 的父边权值为 0，则不断向上跳父亲。对于 v' 同理。这样我们的调整法又可以用了，具体证明就略去了。也可以将边权为 0 的边两侧缩成一个点，不过这样处理起来麻烦一点。

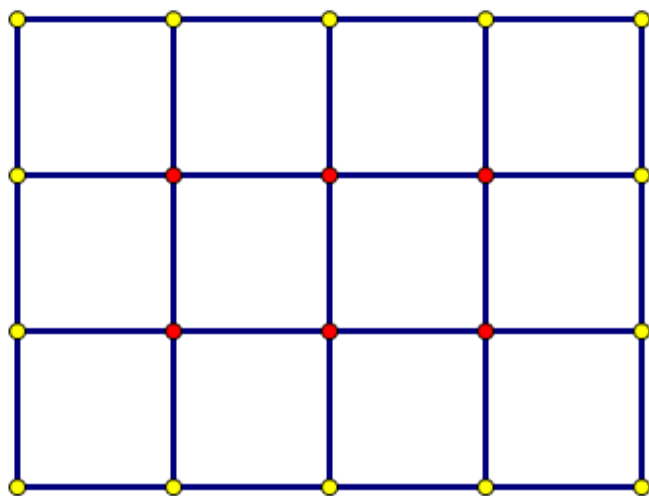
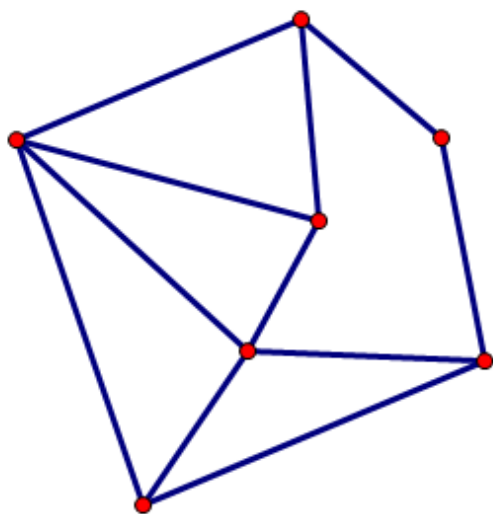
例题：P1186, P2685, P3573, CF1163F。有向图删边最短路 P3238 是假题，不可做。

1.5.3 平面图最小割

前置知识：割的定义。

- 平面图：能画在平面上，满足除顶点处以外无边相交的图称为 **平面图**。
- 网格图：形成一个网格的平面图称为 **网格图**。形式化地，对于点数为 $n \times m$ 的网格图，存在一种为每个点赋互不相同的标号 (i, j) ($1 \leq i \leq n, 1 \leq j \leq m$) 的方案，满足两点之间有边当且仅当它们的 j 相同且 i 相差 1，或 i 相同且 j 相差 1。

一般平面图最小割问题都在网格图上，所以我们先介绍网格图最小割，满足 **边权非负** 且 **割的源点和汇点在边界** 上。称一个点在边界上，当且仅当它的 i 等于 1 或 n ，或 j 等于 1 或 m 。



洛谷@Alex_Wei

如上图，左边是一张平面图，右边是一张网格图，边界上所有点用黄色标注。

考虑求网格图左上角 s 到右下角 t 的最小割，即一种划分点集的方式 $V = S \sqcup T$ ($\sqcup$ 表示无交并，即 $S \cup T = V, S \cap T = \emptyset$)，满足 $s \in S, t \in T$ 且两端分别在 S, T 的边（称为割边）的权值之和（称为割的权值）最小。

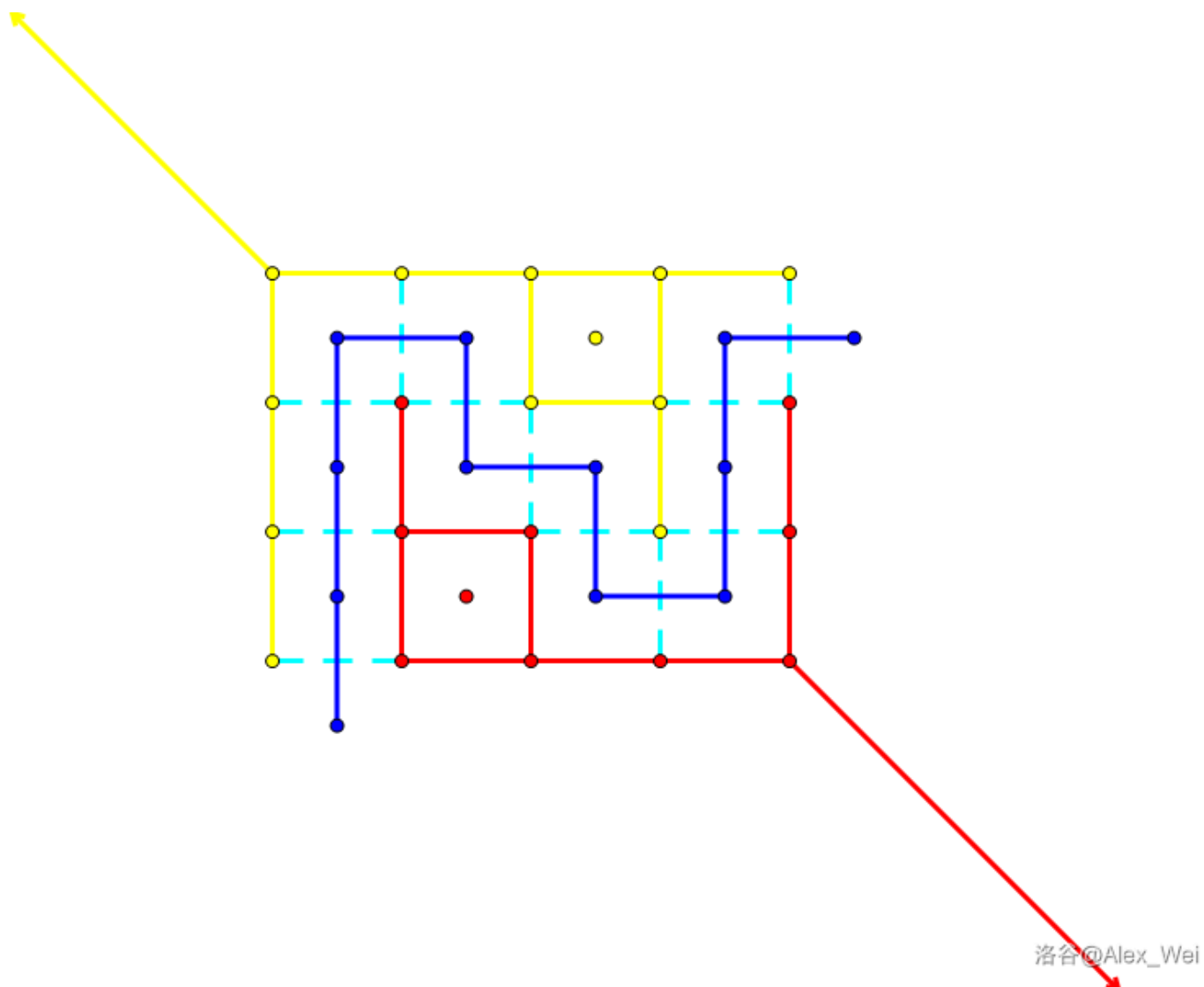
引理

总存在权值最小的割满足 S 的所有点连通， T 的所有点连通。

证明

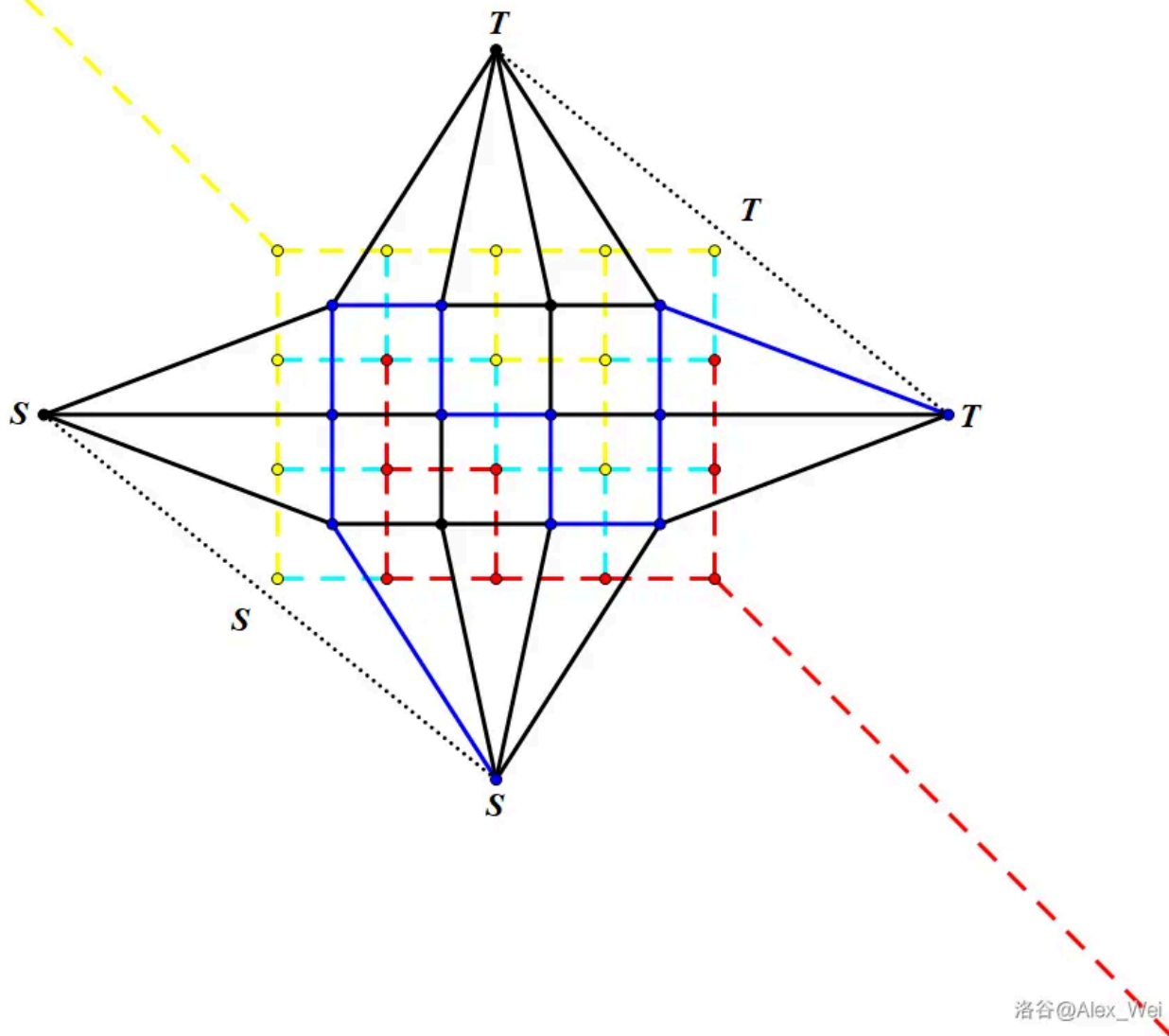
假设 S 的点不连通，那么存在不包含 s 的连通块 S' 。将 S' 并入 T ，原来的非割边不会变成割边，而一些割边变成非割边，所以割的权值不会变大。□

考虑 S 和 T 的边界。这个边界对应了一条从右上到左下的路径。在两个相邻的面之间移动时，需要花费对应边权的代价，那么路径长度就对应了割的权值。如下图。



考虑从左上角和右下角向外引出两条权值无穷大的射线，将网格图外的平面分成左下和右上两部分。现在我们要建出网格图的 **对偶图**：将所有面抽象成点，对于通过一条边相邻的两个面，在它们对应的点之间连权值为这条边的权值的边（这不是对偶图的严谨定义）。

一个满足 S 连通， T 连通的割，恰对应了对偶图上从右上无穷面代表的点到左下无穷面代表的点的路径，因此 **网格图最小割等于对偶图最短路**，如下图。



洛谷@Alex_Wei

实边表示对偶图的边，虚边表示原图的边。注意左侧和下侧，上侧和右侧分别是同一个点。从右上到左下的最短路即为左上到右下的最小割。上图给出了一条可能的最短路以及它对应的最小割。

扩展：不需要源汇在左上角和右下角。只要源汇在边界上，就可以通过转对偶图的方式求最小割。

扩展：不需要是网格图。平面图也可以转对偶图求最短路，整个过程没有用到网格图的性质，只是便于理解。

例题：P4001，P7916。

*1.5.4 k 短路

前置知识：可持久化线段树或其它可持久化结构。

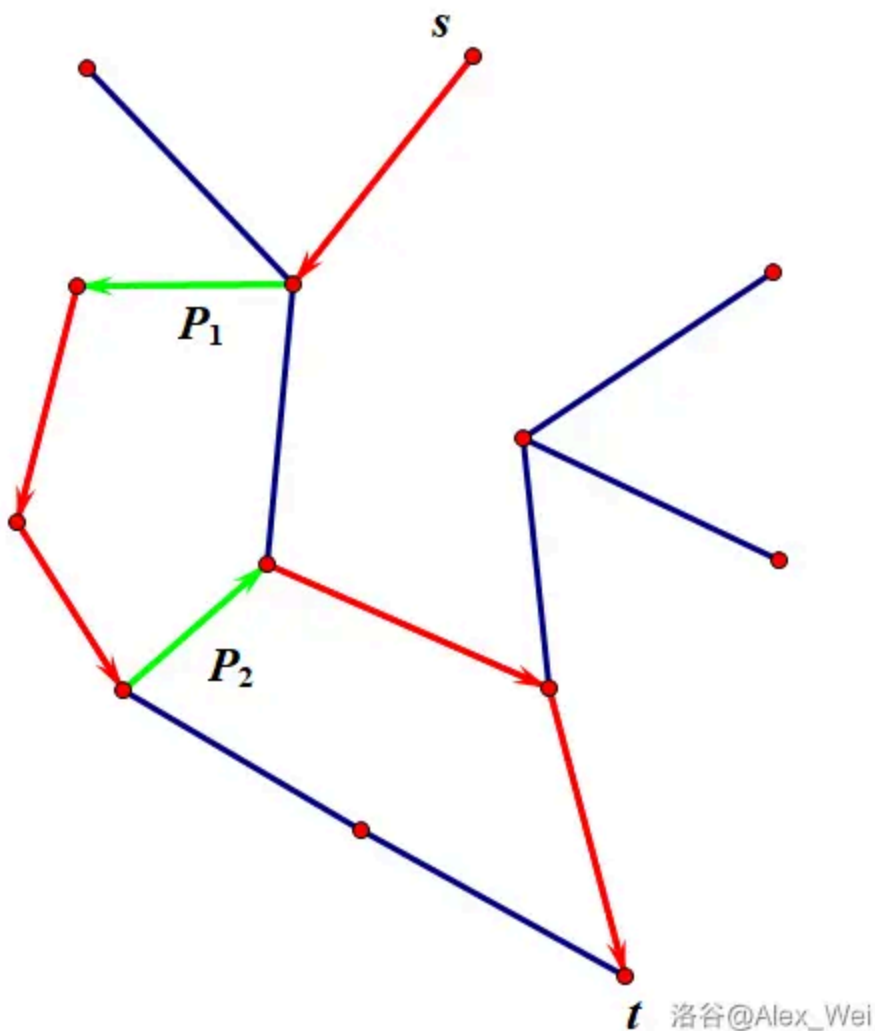
问题描述：给定一张 **有向非负权** 图，求 $s \rightsquigarrow t$ 的前 k 短路径长度。对于无向图，将无向边拆成两条有向边即可。

k 短路严格强于最短路，所以无论如何都需要跑一次最短路算法。

一个解决问题的有效思路是：找一种好的方式刻画研究对象，这里“好的”指便于考虑和解题。 k 短路问题的研究对象是 $s \rightsquigarrow t$ 的路径，所以我们希望找到一种好的刻画路径的方式。我们知道刻画最短路的方式是最短路树，而 k 短路在一定程度上也是最短路问题，所以：在反图上跑最短路，建出到达 t 的最短路树 T （建从 s 出发的最短路树也可以做），有多棵选任意一棵。

考虑一条 $s \rightsquigarrow t$ 的路径的形态。因为一条边要么是树边，要么是非树边，所以路径上会有一些非树边和树边形成的连续段。每个树边连续段对应一条 T 上的路径，满足连续段结束的端点在起始端点到 t 的路径上。

进一步地，发现一个 **有序** 的非树边序列 $P = [e_1, e_2, \dots, e_p]$ 可以 **唯一确定** 一条 $s \rightsquigarrow t$ 的路径。其中，根据上述分析，需要满足对于任意相邻非树边 e_i, e_{i+1} ， $u(e_{i+1})$ 在 T 上是 $v(e_i)$ 的祖先，此外 $u(e_1)$ 是 s 的祖先。如下图。容易证明 P 和路径的一一对应关系。



这很优秀，因为直觉告诉我们 k 短路相较于最短路相差不会太大，而导致它们差异的地方就在于经过了一些不太优的非树边，我们精确刻画了这些非树边的形态，且和原路径一一对应了。那么一个思想是不断添加非树边，通过调整法得到次短路，第三短路，以此类推。

还有一个问题：在研究 P 的过程中，我们丢掉了树边的权值。这对路径长度的统计是致命的。解决方法：当序列末尾加入非树边 $e = (u, v, w)$ 时，本来从 u 沿最短路走到 t 变成先经过 e ，再从 v 沿最短路走到 t ，有 $\delta(e) = (dis_v + w) - dis_u$ ，其中 dis_i 表示 $i \rightsquigarrow t$ 的最短路长度。进一步地，可以证明 P 对应路径的长度等于 $s \rightsquigarrow t$ 的最短路加上所有非树边的 δ 之和，即：

$$dis_s + \sum_{e \in P} \delta(e)$$

注意到 dis_s 是定值，这样路径长度也只和 P 有关了。问题转化为求权值前 k 小的合法的 P 。

这类“不断扩展结构，取出权值前 k 小”的问题非常经典，解法如下：对每个除了权值最小的结构，为其钦定一个权值不比它大的前驱，满足前驱关系不成环。这样，任何结构都可以从唯一的无前驱的权值最小的前驱开始扩展得到，满足每次扩展一个前驱为当前结构的结构，即每次扩展一个后继。用优先队列维护所有被扩展但未被考虑的结构，第 i 次取出队列中权值最小的结构就是权值第 i 小的结构，然后将其所有后继加入优先队列。

证明

假设第 i 次取出队列的结构 Z 不是权值第 i 小的结构，且 i 是所有这样的操作中编号最小的。考虑第 i 小的结构 Y ，其前驱 X 一定被考虑过： X 的排名小于 i ，而根据假设，前 $i - 1$ 次操作均取出了正确的结构，所以 X 被正确取出了。所以 Y 一定在优先队列中且 Y 小于 Z 。这和当前取出 Z 矛盾了。□

只要每个点的后继数量不大，那么时间复杂度为 $\mathcal{O}(k(a \log k + b))$ ，其中 a 表示每个点的后继数量， b 表示查询所有后继的复杂度。

回归原问题，一个设置前驱的方案为：令 P 的前驱为删去 e_p 得到的序列，但这将导致一个序列有 $\mathcal{O}(m)$ 个后继：从 $v(e_p)$ 在 T 上任意祖先出发的所有边均可加入 P 。

解决方法也是经典的：前驱不一定要减少元素个数，也可以减小最后一个元素的权值。对此，加入限制：设 $v(e_{p-1})$ 在 T 上的祖先集合为 A ，若 e_p 不是从 A 出发的 δ 最小的边，那么删去 e_p ，加入它的从 A 出发的 δ 前驱，否则删去 e_p 。即设从 A 出发的所有边按 δ 从小到大排序后分别为 $c_1, c_2, \dots, c_d$ ，将 e_p 替换为它在该序列的前驱。若前驱不存在则直接删去 e_p 。

这样，一个序列至多有两个后继：

- 如果 e_p 不是从 A 出发的 δ 最大的边，那么将其删去，改成后继。

- 加入从 $v(e_p)$ 在 T 上任意祖先出发的 δ 最小的边。

容易证明这是我们钦定的前驱关系对应的后继关系。用可持久化权值线段树维护最短路树上从一个点的所有祖先出发的所有边，查后继和权值最小的边均可线段树二分。

综上，时间和空间复杂度是关于 n, m, k 的线性对数。具体分析可知时间为 $\mathcal{O}((m+k)\log m + k\log k)$ ，空间为 $\mathcal{O}(m\log m + k)$ 。

注意 $|P| = 0$ 或 1 时需要特殊讨论，此时 e_{p-1} 和 e_p 可能不存在。不要忘记 $u(e_1)$ 需要是 s 的祖先。

扩展：存在负权边会使得一开始求最短路的复杂度变成 $\mathcal{O}(nm)$ ，而求 k 短路的复杂度不受影响。

扩展：对于 k 短路，使用可持久化可并堆可以做到更优秀的复杂度。见参考资料。

例题：P2483。

1.5.5 同余最短路

同余最短路算法可以求出在给定范围内有多少重量可由若干物品做完全背包得到，即一段区间内有多少数可由给定正整数进行 **系数非负** 的线性组合得到。

问题描述：给定 n 个正整数 a_i ，求有多少个整数 $y \in [L, R]$ 可以表示为 $\sum c_i a_i$ ，其中 a_i 较小， c_i 是非负整数， L, R 较大。

其核心在于观察到，如果一个数 r 可以被表出，那么任何 $r + xa_i$ ($x \geq 0, 1 \leq i \leq n$) 也可以被表出。选出一个 a_i ，求出每个模 a_i 同余的同余类 K_j 当中最小的能被表出的数 f_j ，即可快速判断 y 能否被表出：当且仅当 $y \geq f_{y \bmod a_i}$ 。

一种理解方式：考虑每个数能否被表出，设为 $g(r)$ 。对于某个 a_i ，如果 $r_1 \equiv r_2 \pmod{a_i}$ 且 $r_1 < r_2$ ，那么 $g(r_1) \leq g(r_2)$ 。这样，根据定义域值域互换的技巧，设 f_j 表示使得 $g(r) = 1$ 且 $r \bmod a_i = j$ 的最小的 r 。

为减小常数，一般选择使得同余类个数最少的 a_i ，即 a 的最小值，设为 a_1 。得到模 a_1 余 j 的数中最小的能被表出的数 f_j 后，加上 $a_2 \sim a_n$ 转移到其它同余类 $f_{(j+a_i) \bmod a_1}$ 。

上述过程非常像一个最短路：对于每个点 j ，它向 $(j + a_i) \bmod a_1$ 连长度为 a_i 的边，求从源点 0 开始到每个点的最短路。使用 Dijkstra 或 SPFA 求解。

求出 f_j 后容易求答案。差分转化为求 $[0, R]$ 有多少个数能被表出，即

$$\sum_{i=0}^{a_1-1} \max \left(0, \left\lfloor \frac{R - f_i}{a_1} \right\rfloor + 1 \right)$$

用 $[0, R]$ 的答案减去 $[0, L - 1]$ 的答案即可。

时间复杂度为 Dijkstra 的 $\mathcal{O}(na_1 \log a_1)$ 或 SPFA 的 $\mathcal{O}(na_1^2)$ 。

存在 $\mathcal{O}(na_1)$ 的做法，见 [同余最短路转圈技巧](#)（含例题 P2371，P9140）。

例题：ARC084B，P4156（见“字符串进阶 I”）。

1.6 例题

P1462 通往奥格瑞玛的道路

最小化最大值，考虑二分答案并使用 Dijkstra 检查 $1 \rightarrow n$ 的最短路是否不大于 b 。

时间复杂度 $\mathcal{O}(n \log n \log c)$ 。[代码](#)。

P4568 [JLOI2011] 飞行路线

注意到 k 很小，跑最短路时记录当前用了几条免费边即可。相当于在分层图（将原图复制若干次）上跑最短路。

时间复杂度 $\mathcal{O}(mk \log(mk))$ 。[代码](#)。

P6880 [JOI 2020 Final] 奥运公交

不翻转的情况是平凡的。考虑翻转，将贡献拆为 $1 \rightsquigarrow n$ 和 $n \rightsquigarrow 1$ 。

对于 $1 \rightsquigarrow n$ ，建出 1 开始的最短路树。

- 若边 i 不在最短路树上，因为最短路要么经过 e_i ，要么不经过。对于前者，即 $1 \rightsquigarrow v_i$ 的最短路，加上 w_i ，再加上 $u_i \rightsquigarrow n$ 的最短路。对于后者，最短路即原最短路。两种情况都可以预处理后 $\mathcal{O}(1)$ 计算。
- 如果边 i 在最短路树上，可以重新跑最短路，因为总共只有 $\mathcal{O}(n)$ 条最短路树上的边。稠密图，用 $\mathcal{O}(n^2)$ Dijkstra 跑最短路。

对于 $n \rightsquigarrow 1$ ，同理。

时间复杂度 $\mathcal{O}(n(n^2 + m))$ 。做到 $\mathcal{O}(m + n^3)$ 需要一些细节处理。[代码](#)。

P4001 [ICPC-Beijing 2006] 狼抓兔子

网格图最小割。建出网格图的对偶图，每条边与其对偶边权值相等。从右上到左下的最短路即为所求。

代码。

CF1765I Infinite Chess

直接 BFS，复杂度高达 8×10^8 ，无法接受。

注意到行数很少，这样斜向攻击只会影响到半径为 8 的列邻域。而对于横向攻击，我们可以把一段每行被攻击状态相同的列缩成一列，相当于离散化。注意国王可以斜向走，所以两侧还要保留 8 列。

离散化后模拟得到每个格子是否被攻击，跑最短路即可。时间复杂度 $\mathcal{O}(n \log n)$ 。代码。

P4926 [1007] 倍杀测量者

化乘法为加法不难想到取对数。

考虑对没有女装的限制建出差分约束图，若出现负环则一定有人女装。对于固定分数的人 i ，让 $0, i$ 之间互相连边权分别为 $\log x_i$ 和 $-\log x_i$ 的边即可。

答案显然满足可二分性。

时间复杂度 $\mathcal{O}(ns \log V)$ 。代码。

*P5304 [GXOI/GZOI2019] 旅行者

$\mathcal{O}(m \log m \log k)$ 做法的核心思想是 **二进制分组** 后跑最短路。两个不同的数的二进制必定至少有一位不同，所以每两个关键点之间的最短路均被统计到。代码。

随机分组也可以，做 k 次的正确性是 $1 - \left(\frac{3}{4}\right)^k$ 。类似题目如 CF1314D。

$\mathcal{O}(m \log m)$ 做法的思想很巧妙。预处理每个特殊城市到点 i 的最短距离 f_i 和对应城市 fr_i ，以及点 i 到所有特殊城市的最短距离 g_i 和对应城市 to_i 。对于每条边 $u \rightarrow v$ ，若 $fr_u \neq to_v$ ，则用 $f_u + w_{u,v} + g_v$ 更新答案。

考虑任意最优路径 $i \rightsquigarrow j$ ，因为 $fr_i = i$ 且 $to_j = j$ ，所以路径上必然至少有一条边 $u \rightarrow v$ 满足 $fr_u \neq to_v$ ， $fr_u \neq j$ 且 $to_v \neq i$ ，反证法易证。那么 f_u 一定等于 $i \rightsquigarrow u$ 的最短路， g_v 一定等于 $v \rightsquigarrow j$ 的最短路，否则将 i 替换为 fr_u 或 j 替换为 to_v 一定更优。

代码。

*P5590 赛车游戏

好题。

转换思路，与其设置边权使路径长度相等，不如设置路径长度去拟合边权的限制。

设 d_i 为 $1 \rightarrow i$ 的最短路，只需保证对于所有边 (u, v) 有 $w_{u,v} = d_v - d_u$ 即可使任意一条简单路径长相等。于是 $1 \leq d_v - d_u \leq 9$ ，转化为 $d_u + 9 \geq d_v$ 与 $d_v - 1 \geq d_u$ ，差分约束求解即可。注意不在 $1 \rightarrow n$ 的任意一条路径上的边没有用，这些边不应计入限制，随便标权值。

时间复杂度 $\mathcal{O}(nm)$ 。代码。

*ABC232G Modulo Shortest Path

将所有点按照 b 值从小到大排序。对于一个点连出去的所有边，连向一段前缀的点的权值形如 $a_i + b_j$ ，连向一段后缀的点的权值形如 $a_i + b_j - m$ ，容易找到分界点。

考虑朴素 Dijkstra 的过程，发现操作本质上是：区间 c_i 对 v 取 $\min$ ，单点修改 d_i ，全局查询 $\min c_i + d_i$ 以及对应位置。线段树区间维护 d_i 和 $c_i + d_i$ 的最小值及其位置即可。

时间复杂度 $\mathcal{O}(n \log n)$ 。代码。

本题的官方解法非常精妙：取模操作可以想象成在环上走。考虑建立虚点 $x_{0 \sim m-1}$ ，从 $x_i \rightarrow x_{(i+1) \bmod m}$ 连权值为 1 的边，让它们连成一个环。从 i 向 $x_{-a_i \bmod m}$ ，从 $x_{b_i \bmod m}$ 向 i 连权值为 0 的边，可准确表示出所有边。环上点数为 $\mathcal{O}(m)$ ，但可以将环上的若干段点缩起来，做到点数 $\mathcal{O}(n)$ 。跑 Dijkstra 即可，时间复杂度 $\mathcal{O}(n \log n)$ 。

P3573 [POI2014] RAJ-Rally

本题可以借助删边最短路的思想，枚举除了最长链以外的所有点，用线段树维护它对最长链上一段区间的贡献。因为保证有向图无环，所以可行。需要添加虚点保证弱连通。

另外一种思路：删去点 u 后有三种路径：拓扑序全部比 u 小，拓扑序全部比 u 大，或拓扑序跨过 u 。前两种好处理，对于第三种，在跨过拓扑序的边上统计答案。按拓扑序扫描，每次加入从拓扑序前驱出发的边的贡献，删去到达当前点的边的贡献，用 `multiset` 维护。

时间复杂度 $\mathcal{O}(m \log m)$ 。代码。

*P3530 [POI2012] FES-Festival

一道加深对差分约束理解的好题。

首先建出差分约束图。跑一遍负环判无解。

但如何满足不同的 t 值数量最大呢？差分约束无法解决这样的问题。

首先强连通分量缩点（见第四章强连通分量），不同 SCC 之间独立，因为可以将两个 SCC 之间的距离任意拉远。

考虑 SCC 内部的贡献。不妨设 t_u 取到了下界, t_v 取到了上界, 根据差分约束的实际意义, 需要满足 $t_u + d_{u,v} \geq t_v$, 其中 $d_{u,v}$ 表示 $u \rightsquigarrow v$ 的最短路。令 $t_v = t_u + d_{u,v}$, 那么 $t_u \sim t_v$ 是否都能取到呢? 因为正权边权值均为 1, 所以最短路路径上所有点必然取遍了 $t_u \sim t_v$ 这 $d_{u,v} + 1$ 个值。

考虑最短路的最大长度 $d_{u,v}$ 。首先容易证明 SCC 内部的答案可以取到 $d_{u,v} + 1$: 根据三角形不等式, 令 $t_i = t_u + d_{u,i}$ 一定合法, 且上文说明了 $t_u \sim t_u + d_{u,v}$ 所有取值均出现过。若答案更大, 设 t'_u 取到下界, t'_v 取到上界, 则 $d_{u',v'}$ 无论如何都得大于 $d_{u,v}$, 与 $d_{u,v}$ 最大矛盾。因此答案为 $\max d_{u,v} + 1$ 。

因为图是稠密图, 所以使用 Floyd 求解任意两点之间的最短路。答案即每个 SCC 的答案之和。

时间复杂度 $\mathcal{O}(n^3)$ 。代码。

*P7916 [CSP-S 2021] 交通规划

$k = 2$ 是经典“狼抓兔子”, 平面图最小割转对偶图最短路。

$k > 2$ 类似。平面图转对偶图后, 最外层会出现 k 个结点。对于一个点 i , 设其逆时针方向的射线颜色和顺时针方向的射线颜色分别为 L_i 和 R_i , 取值为 0 或 1, 表示白或黑。若 $L_i = R_i$, 那么 i 相邻的两个射线处于同一连通块是最优的。因为任何将它们分割开的方案, 通过调整使得它们在同一连通块后代价总会减少。

显然, (L_i, R_i) 等于 $(0, 1)$ 和 $(1, 0)$ 的结点个数相同。每个 $(0, 1)$ 和 $(1, 0)$ 匹配, 路径经过的所有边两端颜色不同。因此这样的匹配方案总能给出一个划分方案及其代价。

关键性质是任意两点的匹配不会相交, 即不会出现 $a < b < c < d$ 且 a 匹配 c , b 匹配 d 。若相交, 则调整法可证不交时不劣。

因此, 首先求出所有 $(0, 1)$ 和 $(1, 0)$ 点之间两两最短路。求匹配最小代价可以类似括号匹配, 破坏成链后区间 DP。

时间复杂度 $\mathcal{O}(knm \log(nm) + k^3)$ 。细节较多, 代码。

*ARC084B Small Multiple

好题。

所有正整数都可以从 1 开始经过若干次 $+1$ 和 $\times 10$ 得到。从 i 向 $(i + 1) \bmod K$ 连权值为 1 的边, 向 $10i \bmod K$ 连权值为 0 的边, 求 $1 \rightarrow 0$ 的最短路即可。

注意不可以将 0 作为源点, 因为题目要求正整数倍。

时间复杂度 $\mathcal{O}(K)$ 。代码。

*AGC056C 01 Balanced

将 0 看成 1, 1 看成 -1 , 问题限制是差分约束的形式。

对于限制 $L - 1, R$, 互相连长度为 0 的边。

对于相邻的两个位置 $i - 1$ 和 i , 要求 $|v_i - v_{i-1}| = 1$ 。看似没有办法描述, 不过当限制变为 $|v_i - v_{i-1}| \leq 1$ 时, 可以在 i 和 $i - 1$ 之间互相连长度为 1 的边, 并最大化 v 的字典序。回忆差分约束本身自带字典序极值性质。

我们惊讶地发现, 在保证字典序最大的前提下, 不可能出现 $v_i = v_{i+1}$ 的情况。模拟最短路的过程, 可以证明 v_i 的奇偶性是固定的, 并且相邻两个 v 的奇偶性一定不同: 0 边连接的两个点的下标奇偶性相同 (题目保证了这一点), 1 边则奇偶性不同。

这题的思想很高妙, 虽然解法是差分约束但是形式新颖, 需要猜性质 (不要被固有思维限制) 才能做出本题。

边权只有 0/1, 01 BFS 的时间复杂度为 $\mathcal{O}(n + m)$ 。代码。

若为了避免出现 $|v_i - v_{i-1}| = 1$ 的限制而直接求前缀和, 我们会建出带有负权边的差分约束网络。用 SPFA 求解会 TLE。

*P7516 [省选联考 2021 A/B 卷] 图函数

设一条边的边权为它的编号。

题面描述花里胡哨, 我们先推些性质。

- 将 h 的值摊到每个点对 (i, j) 上。
- 点对 $(i, j) (i \leq j)$ 最多只会贡献一次, 因为计算 $f(i, G)$ 时, 枚举 $v = j$ 时 i 已经被删去了, 所以只可能在计算 $f(j, G)$ 且枚举 $v = i$ 时产生贡献。
- 模拟图函数的计算过程, 点对 $(i, j) (i \leq j)$ 产生贡献当且仅当存在 $i \rightarrow j$ 的路径 P 和 $j \rightarrow i$ 的路径 Q 使得路径上所有点的编号不小于 i 。
- 设 $T(i, j)$ 表示所有这样的路径 P 和 Q 中, 边权最小值的最大值。注意是 P 和 Q 的边权最小值而不是 P 或 Q 。若 $i = j$ 则 $T(i, j) = m$ 。则 (i, j) 会对所有 $h(G_t) (0 \leq t < T(i, j))$ 产生贡献。

至此, 存在 n^3 的 Floyd 做法。从大到小枚举中转点 k , 枚举 $i, j \in [1, n]$, 令 $d_{i,j}$ 和 $\min(d_{i,k}, d_{k,j})$ 取 $\max$, 初始值 $d_{i,i} = m + 1$, $d_{i,j} (i \neq j)$ 为 $i \rightarrow j$ 的编号, 若不存在则为 0。则 $d_{i,j}$ 表示仅考虑编号不小于 k 的节点时 (不包括两端), $i \rightarrow j$ 的边权最小值的最大值。因此, 在转移 k 之前 (或之后), 先枚举一遍 $j \in [k, n]$, 则 $T(k, j) = \min(d_{k,j}, d_{j,k})$ 。

这个做法严重卡常，需要加一些优化：若 $d_{i,k} = 0$ 则不用再枚举 j ，若 $i \geq k$ 则不用再枚举。代码，可以在 LOJ 和洛谷通过。

考虑对每个 i 求出所有 $T(i, j) (i < j)$ 。将所有边按权值从大到小加入，并跳过至少一个端点编号小于 i 的边。在加入权为 w 的边时，若 $i \rightsquigarrow j$ 且 $j \rightsquigarrow i$ ，说明 $T(i, j) = w$ 。

$j \rightsquigarrow i$ 说明 i 在反图上可达 j 。往当前图加入边 $u \rightarrow v$ 时，若 u 可达且 v 不可达，则从 v 出发开始 bfs，将经过的点标记为可达。否则若 v 可达，则 $u \rightarrow v$ 对连通性没有帮助，直接忽略。否则 u, v 均不可达，这条边在 u 可达时起作用，但当前没有用，所以我们保留它，将其加入 u 的邻边集合。对于反图同理，加入边 $v \rightarrow u$ 。若 j 在加入边权为 w 的边时第一次同时在原图和反图上可达，则 $T(i, j) = w$ 。每条边只会被遍历一次，时间复杂度 $\mathcal{O}(nm)$ 。代码，可以在 LOJ 和洛谷通过。

类似 ARC092F，考虑 bitset 优化。

肯定不能先枚举 i 再枚举 (u, v) ，这样已经 $\mathcal{O}(nm)$ 了。考虑按权值 w 大到小添加每条边 $u \rightarrow v$ ，并批量处理每个 i 。设 $a_{i,j}$ 表示当前 i 是否可达 j 。若某个 i 需要从 v 开始 bfs，易知有 $a_{i,u} = 1$ ， $a_{i,v} = 0$ 且 $i \leq \min(u, v - 1)$ 。因此，还要维护 $r_{j,i}$ 表示 i 是否可达 j ，则 $r_u \wedge \neg r_v$ 即可找到所有 i 。

此外，维护图的邻接矩阵 $e_{i,j}$ 表示当前 $i \rightarrow j$ 是否有边。设 bfs 到 (i, j) ，则需要更新 i 可达 to 当且仅当 $a_{i,to} = 0$ 且 $e_{j,to} = 1$ 。 $\neg a_i \wedge e_j$ 即可找到所有 to 。反图类似处理。

用 `_Find_first` 和 `_Find_next` 找 bitset 中第一个 1 和下一个 1，时间复杂度 $\mathcal{O}(n^3/w)$ ，是在稠密图上表现最优秀的算法。代码。

*CF1163F Indecisive Taxi Fee

修改一条边的权值相当于删去这条边的最短路和强制经过这条边的最短路的较小值。前者是删边最短路，后者也是容易的：

- 对于 $e \in P$ ， $dis(1, n) - w_e + x$ 。
- 对于 $e \notin P$ ， $\min(dis(1, u_e) + w_e + dis(v_e, n), dis(1, v_e) + w_e + dis(u_e, n))$ 。

时间复杂度 $\mathcal{O}(m \log n + q)$ 。代码。

类似题目：P1186 玛丽卡，P2685 桥。

*P7515 [省选联考 2021 A 卷] 矩阵游戏

神题。

nm 个未知数， $(n - 1)(m - 1)$ 个方程，还有不等式限制，一看就没法高斯消元。

既然如此，本题的突破口一定在 $n + m - 1$ 个自由元上。首先让自由元的形态比较好考虑：钦定第一行和第一列上所有元素为自由元，固定它们为 0，根据 b 的限制推出剩下来整个矩阵 $a_{i,j}$ 。

考虑 $a_{1,j}$ ($j \geq 2$)，将其加上 1，发现会令所有 $a_{i,j}$ 加上 $(-1)^{i-1}$ 。

考虑 $a_{i,1}$ ($i \geq 2$)，将其加上 1，发现会令所有 $a_{i,j}$ 加上 $(-1)^{j-1}$ 。

观察到 $a_{i,j}$ 对应的不等式形成了 $a_{1,j}$ 和 $a_{i,1}$ 之间的约束，很好。

但是对于 $a_{1,1}$ ，它对整个矩阵产生的影响是这样的：

$$\begin{bmatrix} 1 & 0 & 0 & 0 & \cdots \\ 0 & -1 & 1 & -1 & \cdots \\ 0 & 1 & -1 & 1 & \cdots \\ 0 & -1 & 1 & -1 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

这样一个约束有三个元素，无法处理。

但是根据之前的经验，将 $a_{1,1}$ 加上 1 可以理解为将所有 $a_{1,j}$ 加上 $(-1)^{j-1}$ ，或将所有 $a_{i,1}$ 加上 $(-1)^{i-1}$ 。

这样，得到如下思路：设 r_i 表示将 $a_{i,j}$ 加上 $(-1)^{j-1}r_i$ ，设 c_j 表示将 $a_{i,j}$ 加上 $(-1)^{i-1}c_j$ ，那么调整后的 $a'_{i,j} = a_{i,j} + (-1)^{j-1}r_i + (-1)^{i-1}c_j$ 。当 i, j 奇偶性相同时，变量前的符号相同，无法处理。考虑写成 $a_{i,j} + (-1)^{i+j}r_i - (-1)^{i+j}c_j$ ，这样每个约束的两个变量符号相反，使用差分约束处理。

题目卡常，注意常数。SPFA 用最短路长度判无解会跑得快一些。

时间复杂度 $\mathcal{O}(nm(n + m))$ 。代码。

思考： r, c 共有 $n + m$ 个变量，但自由元的数量只有 $n + m - 1$ 。多出来的自由度在哪里呢？注意到将 r 交替加减 x ， c 交替加减 $-x$ ，得到的 a 相同。 x 就是多出的自由度。

P2483 【模板】k 短路 / [SDOI2010] 魔法猪学院

k 短路模板题。不断求 k 短路，判断是否有累计权值大于 E 。

代码。

2. 无向图最小生成树

本章讨论的图为 **无向连通图**。

关于有向图最小生成树（最小树形图），见“图论 II”。

2.1 相关定义

- 生成树：对于连通图，形态是一棵树的生成子图称为 **生成树**。

通俗地说，生成树就是连通了图上所有点的树。**非连通图不存在生成树**。

- 生成森林：由每个连通分量的生成树组成的子图称为 **生成森林**。
- 非树边：对于某棵生成树，原图的不在生成树上的边称为 **非树边**。

生成树算法的核心思想：**往生成树加入一条边形成环，考察环的性质**。

给定一张带权连通图，求其边权和最小的生成树，称为 **最小生成树**（Minimum Spanning Tree，MST）。对于非连通图，对每个连通分量求最小生成树即得最小生成森林。

注意：连通图有最小生成森林，等于它的最小生成树。非连通图没有最小生成树。对任意图都可以求最小生成森林，但只有连通图才能求最小生成树。

2.2 最小生成树问题

介绍三种求最小生成树的算法。

2.2.1 Kruskal

考虑最终求得的生成树 T ，考察其性质。

考虑一条非树边 $e = (u, v, w)$ ，以及生成树上连接 u, v 的简单路径。若 w 小于其中任意一条边的边权，将这条边从 T 中删去，再加入边 e ，所得仍为生成树且权值变小。

这说明 T 满足：对于任意非树边 (u, v, w) ，树上连接 u, v 的简单路径上的每条边的权值不大于 w 。这是 **最小生成树的基本性质**。

这启发我们将边按照边权从小到大排序，依次枚举每一条边，若当前边两端不连通则往生成树中加入该边。用并查集维护连通性，时间复杂度 $\mathcal{O}(m \log m)$ 。

若存在非树边 $e = (u, v, w)$ 不满足上述性质，则考虑被替换的边 $e' = (u', v', w')$ 其中 $w' > w$ 。因为考虑到 e' 时 u, v 不连通，所以考虑到 e 时 u, v 不连通，与 $e \notin T$ 矛盾。

借助该工具，可以归纳证明加入的每一条边均在原图的最小生成树中。

并查集合并时新建虚点维护合并关系，得到 **Kruskal 重构树**。它刻画了在仅考虑权值不超过某个阈值的边时整张图的连通情况。见“树论”。

2.2.2 Prim

维护当前点集 V 和边集 E ，每次找到 $\notin E$ ，且一端 $\in V$ ，另一端 $\notin V$ 的权值最小的边 (u, v) ，将 v 加入 V ， (u, v) 加入 E 。初始 V 可以包含任意一个点， E 为空。

另一种理解方式：维护每个点的权值 c_v 表示 v 的所有邻边 (v, u) 中，使得 $u \in V$ 的最小权值。每次取出不在 V 中的权值最小的点加入 V ，将最小生成树的边权加上 c_v （将对应边加入边集 E ），并用 v 的邻边更新不在 V 中的点的权值。

“取出权值最小的点”的过程类似 Dijkstra 算法借助优先队列实现。算法的时间复杂度为 $\mathcal{O}(m \log m)$ 。

Prim 算法的正确性证明和 Kruskal 差不多。

2.2.3 Boruvka

对于每个点 i ，其权值最小的邻边必然在生成树上（若有多条，则任选一条）。否则将其加入 T ，出现过点 i 的环，将环上对应的另一条邻边删去，得到权值更小的生成树。

对于每个点，求出边权最小的邻边。将这些边删去后加入最小生成树。除去至多 $\lfloor \frac{n}{2} \rfloor$ 条重边，每条边使连通分量数减 1，因此一轮这样的过程使得连通分量数严格减半。这样，对剩余连通分量继续上述操作至多 $\log_2 n$ 轮，连通分量数为 1，即得最小生成树。

注意在选择边权最小的边时，不能选择两端已经在同一连通块的边：

- 对每个点，选择一端为该点，另一端和它不在同一连通块的权值最小的边。
- 对每个连通块，选择一端在该连通块，另一端不在该连通块的权值最小的边。

两种选边方式均正确，视情况使用更方便的一种。对于前者，总边数为 $\mathcal{O}(n \log n)$ ；对于后者，总边数为 $\mathcal{O}(n)$ 。

算法的时间复杂度为 $\mathcal{O}(m \log n)$ 。

Boruvka 在解决一类 MST 问题上非常有用：给定 n 阶完全图（任意两点间恰有一条边的无向图），边权通过某种方式计算。当 n 过大时，无法使用朴素 MST 算法，此时根据特殊的边权计算方式可以快速求出每个点权值最小的另一端不在该连通块的邻边，从而借助 Boruvka 算法做到复杂度与 m 无关。如例题 CF888G。

*2.3 拟阵和生成树

注：不保证本小节内容的严谨性。可跳过 2.3.1 和 2.3.2 直接阅读 2.3.3 小节的结论。

线性代数是研究图论问题的常见方法。当生成树和线性代数相结合，又会碰撞出怎样的火花呢？本小节简要介绍了拟阵，这个从线性代数中抽象出的概念，在生成树上的应用。

2.3.1 拟阵的定义与性质

拟阵的定义：记 $M = (S, \mathcal{I})$ 表示拟阵，其中 $\mathcal{I} \subseteq 2^S$ ，即 $\mathcal{I}$ 是 S 的所有子集构成的集合的子集。 S 中的元素称为 **边**， $\mathcal{I}$ 中的元素称为 **独立集**，注意区分拟阵独立集和图独立集。 M 需要满足以下两条公理：

- **遗传性**：独立集的所有子集也是独立集。若 $I \in \mathcal{I}$ ， $J \subseteq I$ 则 $J \in \mathcal{I}$ 。一般认为 $\emptyset \in \mathcal{I}$ 。
- **交换性**：对于两个大小不同的独立集，存在仅属于较大的独立集的元素，使得较小的独立集加入该元素后仍为独立集。对于 $I, J \in \mathcal{I}$ ，若 $|I| < |J|$ ，那么存在元素 $z \in J \setminus I$ 满足 $I + \{z\} \in \mathcal{I}$ 。

拟阵是对线性代数中“线性无关”的关系的抽象：对于向量集合 S ，令 $\mathcal{I}$ 为所有线性无关的向量集。那么遗传性可以理解为：任意线性无关向量集的子集也是线性无关的；交换性可以理解为：对于大小不同的向量集，较大的向量集的秩大于较小的向量集的秩，所以较大的向量集中总存在向量不能被较小的向量集表示。

拟阵和生成树的关系：令 S 为图的边集，它的子集 I 是独立集当且仅当 I 不存在环。证明 $(S, \mathcal{I})$ 是拟阵：

- 遗传性：一个边集无环，则该边集的子集显然无环。
- 交换性：设两个独立集 $|I| < |J|$ ，那么 I 形成的连通块数量大于 J 。因此，总存在 J 形成的连通块，使得它在 I 上不连通。因此存在 J 的一条边在加入 I 时不会产生环。

由此构造的拟阵 $M = (S, \mathcal{I})$ 被称为 **图拟阵**。

给出拟阵相关的若干重要定义：

- **基**：对于独立集 I ，若加入任何 $S \setminus I$ 的元素都会变成非独立集，则称 I 是拟阵的一个 **基**，也称 **极大独立集**。
- **环**：对于非独立集 I' ，若删去其任何元素都会变成独立集，则称 I' 是拟阵的一个 **环**，也称 **极小非独立集**。

定理 1

基的大小相同。

证明

若存在两个大小不同的基 $|A| < |B|$ ，根据交换性，存在 $z \in B \setminus A$ 使得 $A + \{z\} \in \mathcal{I}$ ，与基的定义矛盾。□

定理 2（基交换定理）

设两个不同的基 A, B ，对于任意 $z \in A \setminus B$ ，存在 $y \in B \setminus A$ 满足 $A - \{z\} + \{y\}$ 为 M 的基。

证明

因为 $|A - \{z\}| < |B|$ 且 $z \notin B$ ，根据交换性，存在 $y \in B \setminus (A - \{z\}) = B \setminus A$ 使得 $A - \{z\} + \{y\} \in \mathcal{I}$ 。□

任意大小相同的不同独立集 I, J 之间都有交换定理（替换后仍为独立集），证明方法类似。笔者将对应的操作称为 $I \rightarrow J$ 的单步替换。

图拟阵的每个基对应了 G 的一棵生成树，而基交换定理告诉我们，对于两棵不同的生成树 T, T' ，删去 $T \setminus T'$ 的任意一条边后，可以加入 $T' \setminus T$ 的一条边使得它仍是一棵生成树。这给出了任意两个生成树之间的替换方案，其中单步替换为删去并加入一条边，满足替换过程中恒为生成树。

- 秩：基的大小称为拟阵的 **秩**，对于任意 S 的子集 U ，定义 **秩函数** $r(U)$ 表示 U 的极大独立集（基）大小，即 $\max_{I \in \mathcal{I} \wedge I \subseteq U} |I|$ 。

可知 I 是 U 的基当且仅当 $I \subseteq U$ 且 $|I| = r(U)$ 。容易证明 U 的基仍满足基交换定理。

秩函数 r 有如下性质：

- **有界性**： $\forall U \subseteq S, 0 \leq r(U) \leq |U|$ 。
- **单调性**： $\forall A \subseteq B \subseteq S, r(A) \leq r(B)$ 。

r 的次模性在接下来研究的问题中并不重要，故略去。感兴趣的读者可自行查阅资料（专门学习拟阵时会写新的学习笔记）。

2.3.2 拟阵上的最优化问题

基交换定理很有用，它告诉我们拟阵的两个基可以通过替换元素相互得到。

通过上一小节的铺垫，我们可以解决拟阵上的最优化问题：给出定义在 S 上的函数 $w : S \rightarrow \mathbb{R}$ ，即 S 的每个元素 x 有权值 $w(x)$ ，求所含元素权值和 $w(I) = \sum_{x \in I} w(x)$ 最大的独立集 I 。

如果 I 包含权值非正的元素，根据遗传性，将这些元素删去后 I 仍为独立集，且权值不变小。因此 I 只包含权值为正的元素。

不妨设 S 的所有元素的权值均为正数，则 I 一定是 M 的一组基。否则，根据交换性， I 可以再添加元素使得权值增大。

为方便讨论，以下设元素权值均为 **正整数**。

拟阵的优秀性质启发我们尝试直接贪心：将 S 的元素按权值不升的顺序排序，得到 $w(s_1) \geq w(s_2) \geq \cdots \geq w(s_{|S|})$ 。依次考虑每个 s_i ，若 $I + \{s_i\} \in \mathcal{I}$ ，则将 s_i 加入 I 。

证明

设 I_i 表示考虑 s_i 之后 I 的形态， I'_i 表示最优解包含的下标不大于 i 的元素。设贪心结果 I 和最优解 I' 选择的元素下标分别为 $\{x_i\}$ 和 $\{y_i\}$ 。

若存在 $y_i < x_i$ ，那么 $I'_{y_i} \setminus I_{x_i}$ 只包含下标不大于 y_i 的元素（ I'_{y_i} 只包含下标不大于 y_i 的元素）。进行 $I_{x_i} \rightarrow I'_{y_i}$ 的单步替换，存在 $s_z \in I'_{y_i} \setminus I_{x_i}$ 使得 $I_{x_i} - \{s_{x_i}\} + \{s_z\} \in \mathcal{I}$ 。则 $z \leq y_i < x_i$ 且 $z \notin I_z$ 。而 $(I_z + \{s_z\}) \subseteq (I_{x_i} - \{s_{x_i}\} + \{s_z\})$ ，根据遗传性， $I_z + \{s_z\} \in \mathcal{I}$ ，这与“能加入就加入”的贪心算法矛盾。

故 $x_i \leq y_i$ ，即 $w(s_{x_i}) \geq w(s_{y_i})$ ，所以 $w(I) \geq w(I')$ ， I 一定是最优解。□

扩展：如果要求权值最小，只需将元素按权值不降排序再贪心。

扩展：如果要求权值最大的 **基**，不要删去任何元素，直接做就行了。因为任意时刻 I 都是当前考虑到的所有元素的一组基。

证明

设 $U_i = \{s_1, s_2, \cdots, s_i\}$ 。

假设考虑到 s_i 时， I_i 不是 U_i 的基且 i 最小。因为 $|I_i|$ 不会变小且 $r(U_i)$ 至多增加 1，故只可能是 $r(U_i) = r(U_{i-1}) + 1$ 且 $|I_i| = |I_{i-1}|$ 。

设 I' 为 U_i 的基，则 $|I'| = r(U_i) > r(U_{i-1}) = |I_{i-1}|$ 。根据交换性，存在 $s_j \in I' \setminus I_{i-1}$ 可以加入 I_{i-1} 。而 s_i 无法加入 I_{i-1} ，所以 $j < i$ 。因此 $I_{i-1} + \{s_j\} \in \mathcal{I}$ ， I_{i-1} 不是 U_{i-1} 的基，与 i 最小的假设矛盾。□

从以上两条证明中可以总结出一个小套路：证明拟阵的性质，一般方法是反证 + 交换性和基交换定理。这有助于加深对拟阵的感性认知。

因为一张图的所有生成树也是拟阵，所以拟阵上的最优化问题的解法可以直接解决最小生成树问题。直接套用贪心过程，就得到了 Kruskal 算法。

2.3.3 最小生成树的性质

拟阵是研究最小生成树的强力工具。拟阵的贪心过程可以求得最小权基，但最小权基并不一定唯一。若要刻画所有最小权基的形态，研究它们共同具有的性质，就需要对贪心过程进行更深层次的探索。

一些问题需要将最小生成树的边权不超过某个值的边单独拎出来。请读者思考：在任意最小生成树中，所有权值不大于 w 的边具有什么样的共同特征？所有权值等于 w 的边呢？在贪心过程中，如果跳过了一些本可以加入生成树的边，是否仍能求出最小生成树？

如果你觉得这个问题比较困难，可以先思考特殊情况：所有元素的权值互不相同。

结论

若所有元素的权值互不相同，则最小权基唯一。

证明

假设存在两个最小权基 I, I' 。按权值从小到大排序后，设 I 和 I' 选择的元素下标分别为 $\{x_i\}$ 和 $\{y_i\}$ 。

因为 $I \neq I'$ ，所以存在 i 使得 $x_i \neq y_i$ 。找到最大的 i ，不妨设 $y_i < x_i$ ，那么 $I' \setminus I$ 包含的元素下标不大于 y_i 。进行 $I \rightarrow I'$ 的单步替换，存在 $z \leq y_i < x_i$ 使得 $J = I - \{s_{x_i}\} + \{s_z\} \in \mathcal{I}$ 。因为元素权值互不相同，所以 $w(s_z) < w(s_{x_i})$ ，故 $w(J) < w(I)$ ，与 I 是最小权基矛盾。□

借助相同的思想，可以证明如下性质：

结论

对于权值 w ，设权值不大于 w 的元素集合为 U_w ，则对于任意最小权基 I ， I 包含的权值不大于 w 的元素个数为 $r(U_w)$ 。

证明

设两个最小权基 $I \neq I'$ 。对它们不断应用最小权唯一时的替换方法（可能是 $I \rightarrow I'$ 的单步替换，也可能是 $I' \rightarrow I$ 的单步替换，取决于 x_i 和 y_i 的大小关系），则 $w(I), w(I')$ 均不增加。又因为 $w(I), w(I')$ 是最小权基，所以 $w(I), w(I')$ 不减少。所以每次替换时删去和加入的元素权值相同。这说明任意两个最小权基在相互替换直到相等的过程中，权值等于某个值的元素数量不变，所以任意最小权基的权值等于某个值的元素个数相等。

再根据贪心过程得到的最小权基包含的权值不大于 w 的元素个数为 $r(U_w)$ ，得证。□

推论

对于每个权值 w ，任意最小权基含有的权值不大于 w 和等于 w 的元素个数为定值。

对于每个权值 w ，任意最小生成树含有的权值不大于 w 和等于 w 的边的个数为定值。

在图拟阵中，集合 U 的一组基表示在仅保留 U 对应的边时，整张图的一棵生成森林。那么上述性质告诉我们：在任意最小生成树中，仅保留权值不大于 w 的边时，整张图的连通性相同。即若 u, v 通过权值不大于 w 的边连通，那么在任意最小生成树中，它们也可以通过权值不大于 w 的边连通。

可以感受到，在最小生成树中，每个权值的边相对独立。求最小生成树的过程可以看做：从小到大枚举所有权值。将已经加入的边的两端缩成一个点，求当前权值的边的任意一棵生成森林，并将这些边加入最小生成树。所有最小生成树都可以通过该过程生成，且每层“当前权值的边”的生成森林的多样性导致了最小生成树的多样性。

重要事实：将最小生成树 T 的权值不等于 w 的边的两端缩成一个点，最小生成树的权值等于 w 的边的形态，就是这张图的任意一棵生成树。注意这不意味着两个最小生成树 T 和 T' 关于权值不等于 w 的边的缩点图相等。反例： $G = \{(1, 2, 1), (1, 3, 2), (2, 3, 2)\}$ ， $T = \{(1, 2, 1), (1, 3, 2)\}$ ， $T' = \{(1, 2, 1), (2, 3, 2)\}$ ，它们关于权值 1 的缩点图，一张 1, 3 缩在一起，另一张 2, 3 缩在一起。

实际上，将权值大于 w 的边的两端也缩在一起，只是为了把权值为 w 的边关于权值小于 w 的边的缩点图的生成森林连起来变成生成树。这样在最小生成树计数时方便些：对最小生成森林计数，需要对每个连通块分别求最小生成树的数量并相乘，较麻烦。处理后只要求缩点图的生成树，若缩点图不连通说明原图不连通，生成树数量为 0，不必对每个连通块单独求最小生成树的数量。

此外，点对 (u, v) 第一次连通时对应的权值 w ，就是它们之间所有路径的边权最大值的最小值，即 u, v 之间必须经过权值不小于 w 的边，且恰存在一条路径使得边权不大于 w 。Kruskal 重构树刻画了像这样的关于边权的连通性，见“树论”。

2.4 扩展问题

除了以下提到的扩展问题，还有使用 LCT 维护生成树的经典手段：在生成树上加入一条边，先求出这条边两端的路径上的权值最大的边，将最大边从生成树中删去，再加入当前边。

2.4.1 次小生成树

注：可以不看证明只看结论。

问题描述：求次小生成树的权值。次小生成树即权值第二小的生成树。

感性理解，如果次小生成树和最小生成树相差很多边，可以用基交换定理调整成只相差一条边。而只相差一条边是好做的：枚举每条非树边，求出其两端对应路径上权值最大的边，得到替换后生成树的最小权值。所有替换方案的最小权值对应的生成树就是次小生成树。

倍增求路径最大边权，时间复杂度 $\mathcal{O}(n \log n)$ 。

结论

对于任意最小生成树 T ，若存在次小生成树 T' ，则存在次小生成树 T' 使得 T 和 T' 只相差一条边。

证明

设 T 和 T' 包含的边的编号为 $\{x_i\}$ 和 $\{y_i\}$ ，其中边按权值不降排序。存在最大的 i 使得 $x_i \neq y_i$ 。

如果 $w(y_i) < w(x_i)$ ，进行 $T \rightarrow T'$ 的单步替换，得到权值更小的生成树，矛盾。

否则 $w(y_i) \geq w(x_i)$ ，进行 $T' \rightarrow T$ 的单步替换，得到权值不大于 $w(T')$ ，且与 T 相差边数减少 1 的生成树。

因此，如果 T' 与 T 相差大于一条边，总可以调整使得权值不增，且恰与 T 相差一条边。□

这个结论对严格次小生成树仍然适用。严格次小生成树即权值严格大于 $w(T)$ 的权值最小的生成树。

结论

对于任意最小生成树 T ，若存在严格次小生成树 T' ，则存在严格次小生成树 T' 使得 T 和 T' 只相差一条边。

证明

次小生成树的证明不再适用，因为调整过程中可能出现 $w(T') = w(T)$ 。

欲证结论，只需证明对任意非最小生成树 T' ，可以从 T' 不断调整，每次调整不增加权值，得到与 T 相差一条边的生成树。结论的证明较复杂，但这是笔者能想到的最简单的证明。如果有更简洁的证明欢迎交流。

引理（强基交换定理）

设两个不同的基 A, B ，对于任意 $x \in A \setminus B$ ，存在 $y \in B \setminus A$ ，使得 $A - \{x\} + \{y\}$ 和 $B - \{y\} + \{x\}$ 均为 M 的基。

证明较复杂，略去。

设 T 和 T' 包含的权值为 i 的边集分别为 $\{X_i\}$ 和 $\{Y_i\}$ 。不断应用次小生成树的证明，直到某一步 $y_p \rightarrow x_p$ 使得 $w(T') = w(T)$ 。在这一步之前停下。因为最小生成树的每个权值的边数固定，所以：

- 存在 $j < i$ 满足 $|Y_j| + 1 = |X_j|$, $|Y_i| - 1 = |X_i|$ 。
- 对于任意 $k \neq i, j$, 有 $|X_k| = |Y_k|$ 。
- 对于 $k > i$, 有 $X_k = Y_k$ (否则不会选中 y_p)。

考虑 $k = i$: 对 $x \in X_i \setminus Y_i$ 用强基交换定理, 存在 $y \in T' \setminus T$ 使得 $T - \{x\} + \{y\} \in \mathcal{I}$ 。因为 $X_k = Y_k$ ($k > i$), 所以 $w(y) \leq w(x)$ 。因为 T 是最小生成树, 所以 $w(y) \geq w(x)$ 。因此 $w(y) = w(x)$, 即 $y \in Y_i \setminus X_i$ 。根据强基交换定理, 令 T' 变成 $T' - \{y\} + \{x\}$ 。不断进行该操作直到 $X_i \setminus Y_i$ 为空, 此时 Y_i 等于 X_i 加上一个元素。

从 $i - 1$ 到 $j + 1$ 降序枚举 k : 归纳假设对于所有 $k' > k$, 恰存在一个 k' 满足 $Y_{k'}$ 等于 $X_{k'}$ 加上一个元素, 设为 p , 且对于其它 k' 有 $Y_{k'} = X_{k'}$ 。类似 $k = i$ 的情况, 对 $x \in X_k \setminus Y_k$ 使用强基交换定理直到 $X_k \setminus Y_k$ 为空。过程中可能删去了 Y_p 多出来的元素, 此时 Y_k 等于 X_k 加上一个元素, 新的 p 变成当前的 k ; 也可能没有删掉, 此时 $Y_k = X_k$ 。归纳假设仍然成立。

经过上述调整, 有:

- 存在 $j < i$ 使得 $|Y_j| + 1 = |X_j|$, $|Y_i| - 1 = |X_i|$, 且 Y_i 为 X_i 加上一个元素 (即 $X_i \subset Y_i$)。
- 对于 $k < j$, 有 $|X_k| = |Y_k|$ 。
- 对于 $i < k < j$ 和 $k > j$, 有 $X_k = Y_k$ 。

此时 $T \setminus T'$ 不存在权值大于 j 的元素。

从 j 到 1 降序枚举 k : 归纳假设 $T \setminus T'$ 不存在权值大于 k 的元素, 且对于 $k' < k$, 有 $|X_{k'}| = |Y_{k'}|$ 。对 $y \in Y_k \setminus X_k$ 进行 $T' \rightarrow T$ 的单步替换。假设选择 $x \in T \setminus T'$ 。因为 $T \setminus T'$ 不存在权值大于 k 的元素, 所以 $w(x) \leq k$ 。而如果 $w(x) < k$, 则根据遗传性, 替换后 T' 的所有权值不大于 $w(x)$ 的元素形成独立集, 且元素个数为 T 的权值不大于 $w(x)$ 的元素的个数加 1。这和 “ T 的所有权值不大于 $w(x)$ 的元素为 S (拟阵的边集) 的所有权值不大于 $w(x)$ 的元素的基” 矛盾。因此 $w(x) = k$ 。不断进行操作直到 $Y_k \setminus X_k$ 为空, 此时 $X_k = Y_k$ ($k < j$) 或 X_k 为 Y_k 加上一个元素 ($k = j$)。归纳假设仍然成立。

最终我们得到:

- 存在 $j < i$ 使得 X_j 为 Y_j 加上一个元素, Y_i 为 X_i 加上一个元素。
- 对于任意 $k \neq i, j$, $X_k = Y_k$ 。

这说明 T 和 T' 之间只相差一条边, 且 $w(T) < w(T')$ 。而显然地, 调整的每一步 $w(T')$ 不增。□

如果从 “最小生成树的权值不大于 w 的边的连通性相同” 入手, 可以在一开始将 $Y_{1 \sim j-1}$ 全部替换为 $X_{1 \sim j-1}$, 替换后的 T' 仍为生成树。因为 $\bigcup X_{1 \sim j-1}$ 是拟阵的权值不大于 $j - 1$ 的元素的基, 而

$X_k = Y_k$ ，所以 $\bigcup Y_{1 \sim j-1}$ 也是一组基。基就意味着它们的连通性和拟阵的权值不大于 $j - 1$ 的所有边的连通性相同。

可以类似推导出任意拟阵存在次小权基和严格次小权基（若存在）与最小权基只有一个元素不同。实际上我们已经证明了这一点：上述两条结论的证明用到的所有关于最小生成树的结论都是由拟阵本身得出的，故适用于所有拟阵。将证明中的“最小生成树”改成“独立集”即可。

例题：P4180。

2.4.2 最小生成树计数

问题描述：求一张图的最小生成树的数量。

前置知识：生成树计数（矩阵树定理）。

根据 2.3.3 小节提到的性质，从小到大枚举边权 w ，求出将所有权值小于 w 的边的两端缩起来后，边权为 w 的边的生成森林个数，然后将属于同一连通块的点缩起来。

求生成森林个数有些麻烦。2.3.3 小节最后提到，可以先求出任意最小生成树，将生成树上权值不等于 w 的边的两端缩起来，再使用矩阵树定理求出所有权值等于 w 的边关于缩点图的最小生成树的数量。

注：最小生成树计数和最小生成森林计数的不同点在于，对于非连通图，前者的答案为 0，后者的答案为每个连通块的最小生成树的数量之积。

复杂度分析：设最小生成树有 c_w 个权值为 w 的边，那么考虑边权 w 时，点的数量为 $c_w + 1$ 。而 $\sum c_w \leq n$ ，所以 $\sum (c_w + 1)^3 \leq \sum (c_w + 1)n^2 = (\sum c_w + 1)n^2 \leq 2n^3$ 。时间复杂度为 $\mathcal{O}(n^3)$ 。

例题：P4208。

*2.4.3 k 小生成树

和 k 短路是类似问题，学会其中一个之后容易理解另一个。

问题描述：求无向带权简单图前 k 小生成树的权值。[例题](#)。

首先求出最小生成树。

已知次小生成树可由最小生成树加入一条非树边并删去一条树边得到。枚举非树边 (u, v) ，求出 u, v 在最小生成树的简单路径上权值最大的边，如果加入 (u, v) ，则删去这条边。对每条非树边求出加入它对权值的影响是多少，取权值增加量最小的方案。

对最小生成树求出权值增加量最小的非树边，则有两种选择：强制选择这条边，和强制不选择这条边。对于前者，给出了一棵新的生成树；对于后者，没有给出新的生成树，但边的状态改变了。

如果强制不选择这条边，那么接下来就是决策权值增加量次小的边是否强制选择，依次类推。于是最小生成树给出了若干棵新的生成树，其中第 i 棵生成树钦定了权值增加量前 $i - 1$ 小的非树边不选择，且第 i 小的非树边强制选择。将所有生成树加入优先队列。

不断从优先队列取出权值最小的生成树 k 次做上述扩展，注意已经钦定状态的边不可被改变。一条边有四种状态：树边且可被替换；非树边且可加入生成树；树边且不可被替换；非树边且不可加入生成树。在枚举所有非树边时需要跳过所有不可加入生成树的非树边，求一条非树边能替换掉哪条树边时不能替换不可被替换的树边。

为什么这样做是对的：钦定权值增加量最小的非树边是否选择，本质是将当前生成树可扩展到的生成树集合分裂为了两半，一半含这条非树边，另一半不含这条非树边。每棵可扩展到的生成树恰属于其中一半。对于含非树边的，替换后的生成树一定是这些生成树的权值最小值，因为替换后的生成树是相对于当前生成树而言的次小生成树，即不存在可扩展到的生成树的权值小于替换后的生成树（否则和替换后的生成树为次小生成树矛盾了）。对于不含非树边的，对应集合的最小值依然是当前生成树，但一条非树边的状态从可加入变成了不可加入。

如果将每棵生成树抽象为一个点，用一棵有根树描述整个扩展过程（根节点是最小生成树），那么每个非叶子结点有两个儿子，一个是实点，从当前点向下走到实点表示用一条非树边替换了一条树边得到新的生成树；另一个则是虚点，从当前点向下走到虚点表示当前生成树的一条非树边从可加入变成了不可加入。由于实点一定是当前生成树能扩展到的最小生成树，所以整个过程是不重不漏的。

支持对一个“点”求权值增加量最小的非树边：直接求树链最值需要倍增，但我们只需求 $\mathcal{O}(k)$ 次权值增加量最小的非树边。考虑对每条可被替换的树边求出能替换它的权值最小的非树边，操作方式为按权值从小到大的顺序枚举可以加入的非树边，将对应路径上所有还没被标记的边标记。可以用并查集实现该过程。

优先队列中不需要存储生成树的具体形态，只需记录每棵生成树由哪棵生成树如何替换一条边扩展得到，以及权值。等到真正取出这棵生成树之后再对其 DFS 预处理相关信息（以支持求权值增加量最小的非树边）。

此外，为了避免从优先队列中不断取出虚点导致求了 $\mathcal{O}(m)$ 次后继状态（相当于去掉了所有虚点，将二叉树变成了多叉树，一次扩展所有儿子），考虑只扩展一次且只维护实点，并在取出实点后加入它的“兄弟虚点”的实儿子。

时间复杂度 $\mathcal{O}(m \log m + mk\alpha(n) + k \log k)$ 。代码。

*2.4.4 最小度限制生成树

问题描述：给定 k ，求点 1 的度数为 k 的最小生成树。

该问题有 $\mathcal{O}(m \log V \alpha(m))$ 的 wqs 二分做法，不在本文讨论范围内，感兴趣的同学可自行查阅资料。

先求出不含点 1 的最小生成森林 T ，则不在森林上的边一定无用。

证明

设 $e = (u, v, w)$ 为非树边，根据最小生成树的基本性质，存在连接 u, v 的树边构成的路径 P ，且路径上每条边的权值不大于 w 。

假设 e 在最小 k 度生成树上，断开 e ， u, v 不连通。因此 P 上存在一条边使得其两侧不连通，加入该树边即可。□

设 $2 \sim n$ 每个点的点权为它和 1 之间的边权，若不存在则为 $+\infty$ 。考虑最终生成树，将 1 删去后整张图裂成 k 个连通块，每个连通块会选择与 1 相连的权值最小的边，即点权最小值。故一棵最小生成树可理解为：从 T 删去若干条边，使图分裂为 k 个连通块。在每个连通块中选择权值最小的点与 1 相连。这样，可以用只含 $2 \sim n$ 的森林描述最小 k 度生成树。

T 初始的连通块数量决定了度数的下界，而点权不为无穷大的点的数量决定了度数的上界。为方便讨论，以下设 **T 连通且 1 和每个点均有边**，则最小 $1 \sim n - 1$ 度生成树均存在。

最小生成树可以贪心，那么最小 k 度生成树是否也可以贪心呢？能否每次贪心地删去一条树边，使得 $w(T') - w(T)$ 最小？

考虑删去树边 e 后树的权值如何变化：设删去 e 前 e 所在连通块为 C ，最小权点为 x 。删去 e 后 C 分裂为两个连通块 C_1, C_2 ，不妨设 x 是 C_1 的最小权点，同时设 C_2 的最小权点为 y ，则权值变化量为 $f(e) = w(T') - w(T) = w(y) - w(x)$ 。

结论

对于初始最小生成树 T ，考虑使得 $f(e)$ 最小的边 e 。对于任意 $2 \leq k < n$ ，存在最小 k 度生成树 T_k 删去了 e 。

证明

设 x, y 分别为删去 e 后两个连通块 C_1, C_2 的最小权点，其中有一个是 T 的最小权点，设为 x ，则 x 在任何连通块内都是最小权点，所以 x 一定被选中。

根据 $f(e)$ 的最小性， e 一定是 T 的连接 x, y 的路径 P ($P = p_0(x) \rightarrow p_1 \rightarrow \cdots \rightarrow p_c(y)$) 上的最大权边。

假设 $e \in T_k$ 并从 T_k 中删去点 1，设包含 e 的连通块为 C 。

- 若 y 被选中：因为 x, y 均被选中，所以它们在 T_k 上不连通。设 P 和 C 的交集为路径 Q 。因为 $e \in Q$ ，所以 Q 非空，设为 $p_l \rightsquigarrow p_r$ ($0 \leq l < r \leq c$)。
 - 若 C 被选中的点在 e 左侧（含有 x 一侧的子树），则 $y \notin C$ （因为 y 也被选中），即 $r < c$ 。加入边 (p_r, p_{r+1}) ，删去 e ，考虑权值变化：
 - 因为 e 是 P 的最大权边，所以 $w(e) \geq w(p_r, p_{r+1})$ 。
 - 因为 C 被选中的点不变，其它连通块没有缩小，所以被选中的点的权值之和不增。这说明调整后权值不增。
 - 若 C 被选中的点在 e 右侧（含有 y 一侧的子树），则 $x \notin C$ ，即 $0 < l$ 。加入边 (p_{l-1}, p_l) ，删去 e ，类似可证权值不增。
- 若 y 未被选中：因为 y 是 e 右侧的最小权点，所以 y 所在连通块一定包含 e 左侧的点，因此 $y \in C$ 。设任意一条与 x 所在连通块 C_x 相连的被删去的边为 e' （因为 $k \geq 2$ ，所以 e' 存在），它连接的另一个连通块 C' 的最小权点为 y' 。在原树 T 中删去 e 后，另一个连通块 C'' 包含 C' ，所以 C'' 的最小点权不大于 $w(y')$ ，因此 $f(e') \leq w(y') - w(e')$ 。加入边 e' ，对权值的影响为 $w(e') - w(y')$ 。然后删去 e 。因为 C 的最小权点在 y 左边，所以 C 分裂出的两个连通块 C_1, C_2 ，一个的最小权点为 C 最小权点，另一个包含 y ，最小点权不大于 $w(y)$ ，对权值的影响不大于 $w(y) - w(e) = f(e)$ 。而 $f(e) \leq f(e') \leq w(y') - w(e')$ ，因此权值不增。

综上，总可以调整使得 T_k 权值不增且 $e \notin T_k$ 。□

因此，在求最小 $2 \leq k < n$ 度生成树之前，可直接将 $f(e)$ 最小的 e 删去。注意到删去后两个子树的贡献独立，结合上述结论，我们猜测每个点的贡献都是独立的。

设 $f_k(T)$ ($1 \leq k \leq |T|$) 表示当初始树为 T 时（不含点 1），它的最小 k 度生成树的权值。设 T_k 表示对应连通块形态（不含点 1）。即从 T 删去 $k - 1$ 条边，使其裂成 k 个连通块，所有边权加上每个连通块最小点权的最小值，以及取到最小值时的形态。

结论

对于 $2 \leq k \leq |T|$ 和任意 T_{k-1} ，存在 T_k 为 T_{k-1} 删去一条边，且 $f_k(T)$ 关于 k 在 $[1, |T|]$ 上下凸。

证明

当 $|T| = 1$ 或 2 时显然成立。

若 $|T| > 2$ ，根据上述结论，设使得删去后新的权值最小的边为 e （不一定是权值最小的边），可钦定 $T_{2 \sim |T|}$ 一定删去 e 。

设删去 e 后得到的两棵树为 T' 和 T'' ，归纳假设 T' 和 T'' 满足结论。

因为 T 和 T'' 的贡献独立，所以 $f_k(T)$ 等于 $f_{k_1}(T') + f_{k_2}(T'')$ 的最小值，其中 $1 \leq k_1 \leq |T'|$ ， $1 \leq k_2 \leq |T''|$ ，且 $k_1 + k_2 = k$ 。这是下凸函数的 $\min +$ 卷积，根据经典理论（闵可夫斯基和）， f_k 也是下凸的。而 $f_{k_1}(T')$ 和 $f_{k_2}(T'')$ 的每个差分值都对应“删去一条边产生的贡献”，所以 $f_k(T)$ 的每个差分值都对应“从 T' 或 T'' 中删去一条边产生的贡献”。

另一种理解方式：结论实际上说明了在 $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_{|T|}$ 的过程中，每次删去一条使得新的权值最小的边，且权值变化量（每条边的代价）随着删边而不降（下凸即二阶导非负， $f_{k+1}(T) - f_k(T) \geq f_k(T) - f_{k-1}(T)$ ）。现在 T 分裂成 T' 和 T'' ，因为 T' 和 T'' 独立，所以删去使得新的权值最小的边相当于初始令 $k_1, k_2 = 1$ ，每次选择 $f_{k_1+1}(T') - f_{k_1}(T')$ 和 $f_{k_2+1}(T'') - f_{k_2}(T'')$ 较大的那个差分值，令其为 $f_{k+1}(T) - f_k(T)$ ，然后将 k 和被选中的 k_i 加上 1。

此时只需证明 $f_2(T) - f_1(T) = f(e) \leq f_3(T) - f_2(T)$ 。

可以证明在子图上删去一条边的代价不小于在原图上删去这条边的代价：设在原图删去 e' 之后，不含 x 的连通块为 C ，新增点权为 C 的最小点权 $w(C)$ 。在子图删去 e' 之后，一定有一侧连通块 C' 包含于 C 。新增点权为两侧连通块最小点权的较大值，不小于 C' 的最小点权 $w(C')$ 。而 $C' \subseteq C$ ，所以 $w(C) \leq w(C')$ 。

结合 $f(e)$ 的最小性，得证。□

上述结论告诉我们，随着 k 增大，已经删去的边不会再出现，已经加入的点不会再消失。

至此，存在 n^2 DP 做法：对每个连通块以最小权点为根 DFS，求出每个子树的最小点权，可知删去每条边的权值变化量。取代价最小的边删去即可。

若删去 e 后新增点 y ，则称 e 被 y 删去。根据结论，每条边被固定的点删去。考虑求出 e 被哪个点删去。

设删去 e 之前所在连通块的最小权点为 x ，则 e 在 x, y 的路径上，且 e 是路径边权最大值（对边权相同的边任意钦定大小关系，不影响答案）。设 C 为包含 e 的权值不大于 $w(e)$ 的边形成的连通块，考察 C 的状态。

注意到对于 x', y' 至少一个不属于 C 的操作， C 中不可能有边被删掉，因为 x', y' 之间的路径如果经过 C ，那么一定经过权值大于 $w(e)$ 的边（由 C 的定义可知）。这说明当 C 的第一条边被删去时，一定有 $x', y' \in C$ 且 $x' = x$ 。

设 C_x 为 e 的 x 一侧与 C 的交，另一侧为 C_y 。设 $y_{\min}$ 为 C_y 的最小权点。设 x', y' 删去的边为 e' 。

- 如果 $y' \in C_y$ ，则 x', y' 路径上的最大权边为 e ，得 $e' = e$ ，推出 $y' = y_{\min}$ 。
- 如果 $y' \in C_x$ ，则 $w(e') < w(e)$ （ e 是 C 的最大边权），根据 $w(y') - w(e')$ 的最小性得 $w(y') < w(y_{\min})$ 。新的 x 有可能变为 y' ，但无论何种情况点权均不大于 $y_{\min}$ 。

因此 $y = y_{\min}$ ，即 e 一定被 $y_{\min}$ 删去。

在 Kruskal 的过程中维护连通块的最小点权。用 e 合并两个连通块时，可求出删去 e 产生的贡献 $c(e)$ 等于最小点权的较大值减去 $w(e)$ 。将 e 按 $c(e)$ 从小到大排序得 $e_1 \sim e_{n-1}$ ，则 $f_{k+1}(T) = f_k(T) + c(e_k)$ 。

最后处理遗留的细节：之前钦定了 T 连通且 1 与 $2 \sim n$ 每个点相连。对于 T 不连通的情况是一样的，因为 T 的每个连通块独立，连通块数决定了最小的 k ，对小于 $k_{\min}$ 的 k 无解。如果权值为正无穷，说明 k 超过上界（上界为初始连通块数，加上 c 值不为正无穷的边数），同样无解。

算法在 $\mathcal{O}(m \log m)$ 的时间复杂度内，对每个 $1 \leq k \leq n - 1$ 判定最小 k 度生成树是否存在，若存在则可以求出其权值。

模板题 的 代码。

2.5 例题

P1967 [NOIP2013 提高组] 货车运输

求出最大生成树，那么使得 u, v 第一次连通的边就是 u, v 之间所有路径边权最小值的最大值。它等于 u, v 在最大生成树上的简单路径边权最小值。倍增即可。

设 n, m 同级，时间复杂度 $\mathcal{O}((n + q) \log n)$ 。代码。

P4180 [BJWC2010] 严格次小生成树

因为总可以调整使得严格次小生成树和最小生成树之间只差一条边，所以先求出任意最小生成树，然后枚举非树边，加入该边之后替换掉环上比该非树边权值小的权值最大的边，为原树一条路径上权值最大或严格第二大的边。

倍增即可，时间复杂度 $\mathcal{O}(m \log n)$ 。代码。

P4208 [JSOI2008] 最小生成树计数

最小生成树计数的板子题。因为相同权值的边不超过 10 条，如果你不会 Matrix-Tree 定理，也可以直接枚举边集检查是否为最小生成树。

模数不为质数，需要使用辗转相除式的行列式求值。

时间复杂度 $\mathcal{O}(n^3)$ 。代码。

*CF888G Xor-MST

根据异或不难想到对 a_i 建出 01 Trie。

对于 01 Trie 上某个状态 p 的子树包含的所有结点 V_p ，它们之间已经连通： V_p 跨过该状态向其它状态 q 所表示的结点 V_q 连边的代价大于在 V_p 内部连边的代价。因为跨过连边时，代价在 p 对应位或其高位有值，但内部连边时 p 对应位及其高位没有值。整个过程类似 Kruskal 算法。

考虑计算答案。若 p 的左右儿子内均有结点，那么需要在 V_{ls_p} 和 V_{rs_p} 之间连一条边使得 V_p 连通。枚举 V_{ls_p} 中所有权值 v 并求出 V_{rs_p} 的子树中与 v 异或和最小的权值 u ，那么 $u \oplus v$ 的最小值即为连边代价。

因为一个结点最多被枚举 $\log V$ 次，所以即使不启发式合并，时间复杂度也可以接受，为 $\mathcal{O}(n \log^2 V)$ 。若启发式合并，则复杂度为 $\mathcal{O}(n \log n \log V)$ 。代码。

*CF1305G Kuroni and Antihype

这题就很厉害了。

先假设存在一个邀请方案，分析它的一般性质。

将邀请的操作视为有根树森林，每个点由它的父亲邀请而来，则总贡献为每个点的权值乘以它的儿子数量。对于除了根以外的节点，它的儿子个数为它的度数 -1 。那么我们有没有什么办法让根节点也满足这样的性质呢？很简单，用一个权值为 0 的虚点连向所有根，这样总贡献严格符合每个点的权值乘以度数 -1 ，且森林变成了一棵树。

将总贡献加上每个点的权值，这是一个定值，则我们希望最大化每个点的权值乘以度数。度数启发我们将每个点的贡献摊到与它相邻的边上，每条边的贡献即它的两端的权值之和。对原图求最大生成树（注意包含点 0， $a_0 = 0$ ），其权值减去每个点的权值之和即为答案。

考虑 Kruskal，从小到大枚举边权 i 及其子集 j ，将所有权值为 j 的点和权值为 $i - j$ 的点连成一个连通块。时间复杂度 $\mathcal{O}(3^k)$ ，其中 $k = \lceil \log_2 n \rceil$ 。代码。

考虑 Boruvka，我们需要找到一个连通块向外的最小边权。考虑高维前缀和，对每个 mask 维护权值是它的子集的最大点权以及对应点编号，但这样在查询一个点权值补集的信息时，点权最大的点可能和该点在同一连通块。因此，对每个 mask，我们还要维护权值是它子集且不与权值最大点在同一连通块的次大点。写起来细节较多，但时间复杂度更优，为 $\mathcal{O}(2^k \log^2 n)$ 。代码。

3. 无向图连通性：双连通分量

无向图连通性中的双连通分量和有向图可达性中的强连通分量是图论的重要部分，分两章节介绍。

本章研究双连通分量相关知识点，研究对象为无向连通图。默认图为 **无向连通图**。

3.1 相关定义

无向图连通性，主要研究割点和割边。

- 割点：在无向图中，删去后使得连通分量数增加的点称为 **割点**。
- 割边：在无向图中，删去后使得连通分量数增加的边称为 **割边**，也称 **桥**。

孤立点和孤立边的两个端点都不是割点，但孤立边是割边。非连通图的割边为其每个连通分量的割边的并。

为什么割点和割边这么重要？对于无向连通图上的非割点，删去它，图仍然连通，但删去割点后图就不连通了。因此割点相较于非割点对连通性有更大的影响。割边同理。

- 点双连通图：不存在割点的无向连通图称为 **点双连通图**。根据割点的定义，孤立点和孤立边均为点双连通图。
- 边双连通图：不存在割边的无向连通图称为 **边双连通图**。根据割边的定义，孤立点是边双连通图，但孤立边不是。
- 点双连通分量：一张图的极大点双连通子图称为 **点双连通分量 (V-BCC)**，简称 **点双**。
- 边双连通分量：一张图的极大边双连通子图称为 **边双连通分量 (E-BCC)**，简称 **边双**。

将某种类型的连通分量根据等价性或独立性缩成一个点的操作称为 **缩点**，原来连接两个不同连通分量的边在缩点后的图上连接对应连通分量缩点后形成的两个点。根据连通分量的类型不同，缩点可分为无向图上的点双连通分量缩点（圆方树，见“图论 II”），边双连通分量缩点（本章），以及有向图上的强连通分量缩点（下一章）。

边双和点双缩点后均得到一棵树，而强连通分量缩点后得到一张有向无环图。

- 点双连通：若 u, v 处于同一个点双连通分量，则称 u, v **点双连通**。一个点和它自身点双连通。由一条边直接相连的两点也是点双连通的。
- 边双连通：若 u, v 处于同一个边双连通分量，则称 u, v **边双连通**。一个点和它自身边双连通，但由一条边直接相连的两点不一定边双连通。

点双连通和边双连通是无向图连通性相关最基本的两条性质。

注：点双连通和边双连通有若干等价定义，本文选取的定义并非最常见的定义。其它定义将会作为连通性性质在下文介绍。

3.2 双连通的基本性质

研究双连通的性质时，最重要的是把定义中的基本元素——割点和割边的性质理清楚。然后从整体入手，考察对应分量在原图上的分布形态，再深入单个分量，考察分量内两点之间的性质。以求对该连通性以及连通分量有直观的印象，思考时有清晰的图像作为辅助，做题思路更流畅。

边双连通比点双连通简单，所以先介绍边双连通。

3.2.1 边双连通

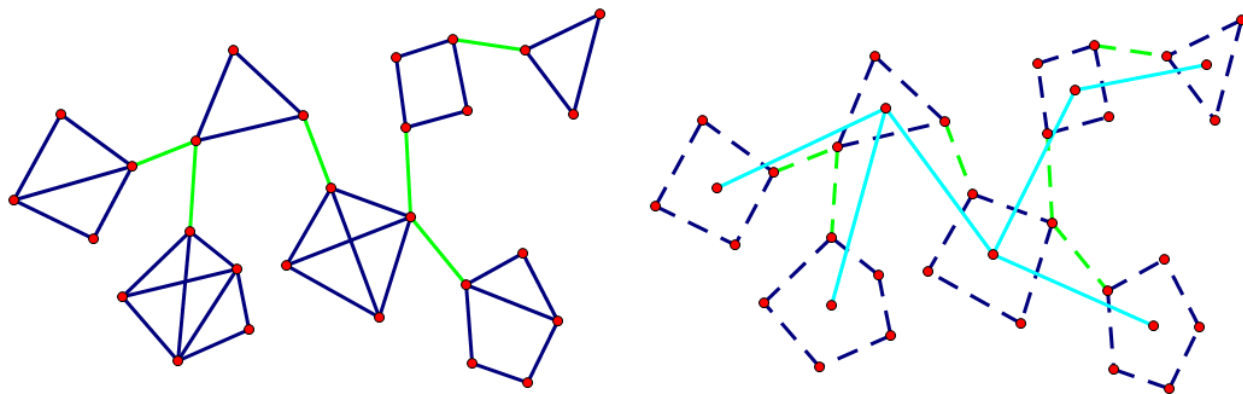
考虑割边两侧的两个点 u, v （不是割边的两端）。因为删去割边后 u, v 不连通，所以考虑任何一条 u 到 v 的迹，这条割边一定在迹上——否则删去后 u, v 仍连通。将这样的边称为必经边：从 u 到 v 必须要经过的边。两点之间的所有必经边，就是连接它们所有迹的边集的交。在研究必经边时，重复经过一条边是不优的，所以只需考虑两点之间的所有迹。

结论

两点之间任意一条迹上的所有割边，就是两点之间的所有必经边。

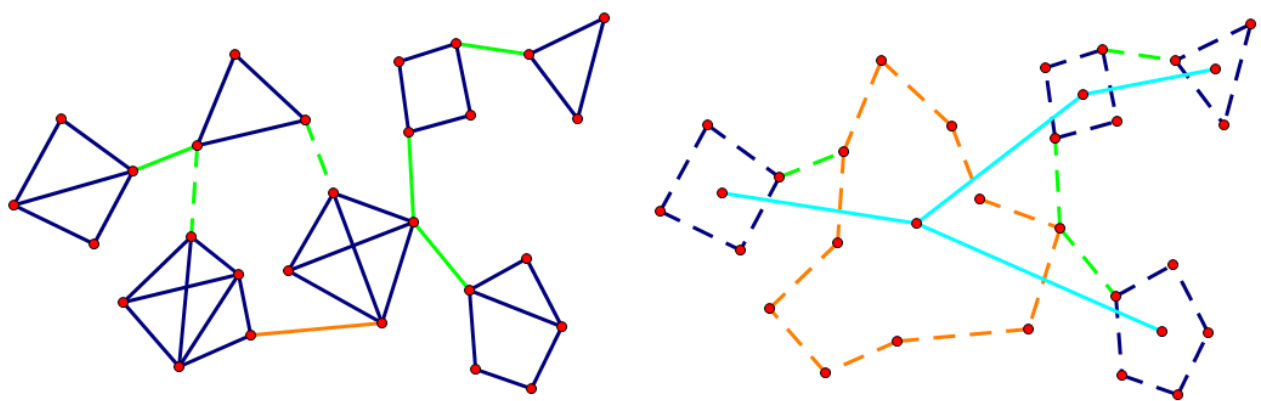
可以看出割边就是必经边的代名词。必经边需要两个点才有定义，而割边直接由原图定义，且边双缩点后可根据树边（割边）直接求出两点之间的必经边。

断开一条割边，整张图会裂成两个连通块。断开所有割边，整张图会裂成割边条数 $+1$ 个连通块。每个连通块内部不含割边且不能再扩大（再扩大就包含割边了），是原图的边双连通分量。这说明边双连通分量由割边连接，且形成一棵树的形态。将边双缩成一个点，得到边双缩点树，每条边对应原图的一条割边，每个点对应原图的一个边双连通分量，如下图。



洛谷@Alex_Wei

添加任意一条边 u, v 。考虑边双形成的树， u, v 所在边双的树上简单路径上的所有边双会被缩成一个大边双：由于添加了 u, v ，连接这些边双的割边变成了非割边。如下图，加入橙色边后，原来的三个边双形成了橙色轮廓的大边双。



洛谷@Alex_Wei

不同边双没有公共点，即每个点恰属于一个边双，所以：

边双连通的传递性

若 a 和 b 边双连通， b 和 c 边双连通，则 a 和 c 边双连通。

边双内部不含割边，所以：

结论

u, v 边双连通当且仅当 u, v 之间没有必经边。

考虑边双中的一条边 (u, v) 。将其删去后，其两端仍连通，将连接它们的路径和 u, v 拼起来，得到不经过重复边的回路。

结论

对于边双内任意一条边 (u, v) ，存在经过 (u, v) 的回路。

结论

对于边双内任意一点 u ，存在经过 u 的回路。

这个结论可以继续加强，见 3.2.3 小节 Menger 定理。

3.2.2 点双连通

删去割点后不连通的两个点之间任意一条路径必然经过该割点，称这样的点为必经点：从 u 到 v 必须要经过的点。在研究必经点时，重复经过一个点是不优的，所以只需考虑两点之间的所有路径。

错误结论

两点之间任意一条路径上的所有割点，就是两点之间的所有必经点。

错误原因是对于割边，经过割边一定会从删去割边的一个连通块走到另一个连通块，但经过割点不一定从删去割点的一个连通块走到另一个连通块。如图 $G = \{(1, 2), (1, 3), (2, 3), (3, 4)\}$ ，3 是割点，也出现在 $1 \rightsquigarrow 2$ 的路径上，但不是 $1 \rightsquigarrow 2$ 的必经点。

关于两点之间割点的刻画，见“图论 II”广义圆方树。

与边双不同的是，两个点双之间可能有交：考虑“8”型结构，中间的交点在两个点双都出现了。因此 **点双连通不具有传递性**。但若两个点双有交，交点一定唯一，否则两点双可以合并为更大的点双。证明点双相关性质时，点双的极大性通常是导出矛盾的关键。

进一步地，若两点双有交，交点一定阻碍了它们继续扩大：如果删去该点之后两点双仍连通，同样地，两点双可以合并为更大的点双。

结论：若两点双有交，那么交点一定是割点。

区分：称两点属于同一个边双，即两点边双连通，就是判断它们所在边双是否相同。称两点属于同一个点双，即两点点双连通，是检查是否存在点双同时包含这两个点。因为两点双至多有一个交点，所以若两点点双连通，那么包含这两点的点双唯一。

说明：点双包含原图割点，但在只关心该点双时，原图割点变成了非割点。删去割点后，原图分裂为若干连通分量，但该点双仍连通。

现在我们知道点双交点是割点，那么割点一定是点双交点吗？删去一个割点，将整张图分成若干连通块。存在割点的两个邻居 u, v 不连通。 u, v 不会处于同一个点双。假设存在这样的点双，则删去割点后 u, v 不连通，矛盾。又因为“一边连两点”是点双连通图，所以割点和 u, v 点双连通。这说明割点属于超过一个点双。

结论

一个点是割点当且仅当它属于超过一个点双。

结论

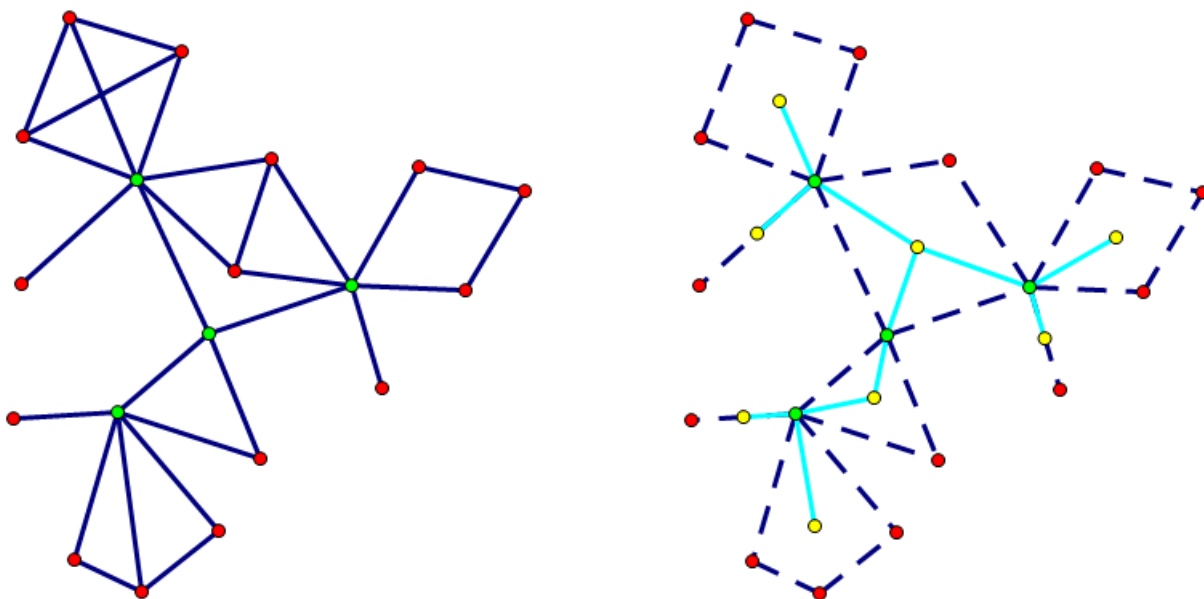
由一条边直接相连的两点点双连通。

结合上述性质，得：

推论

一条边恰属于一个点双。

可知割点是连接点双的桥梁，正如割边是连接边双的桥梁。用一个点代表一个点双，并将点双代表点向它包含的割点连边，得到 **块割树** (Block-Cut Tree, “点双连通块 - 割点” 树，和圆方树有细微差别)。



洛谷@Alex_Wei

考虑点双中的一个点 x 。将其删去后，剩下所有点连通。因此，若 $d(x) > 1$ ，考虑其任意两个邻居 $u \neq v$ ，将新图 u, v 之间的路径和 x 连起来，得到经过 x 的简单环。若 $d(x) = 1$ ，则整个点双为“一边连两点”的平凡形态。

结论

对于 $n \geq 3$ 的点双内任意一点 u ，存在经过 u 的简单环。

关于更多点双连通的性质和点双缩点，见 3.2.3 小节 Menger 定理和“图论 II”广义圆方树。

3.2.3 Menger 定理及其推论

前置知识：最大流最小割定理。

Menger 定理是研究图连通性时相当重要的定理。从 Menger 定理出发，能得到大量双连通问题的关键结论。

让我们先回顾一下最大流最小割定理：在一张网络上， $s \rightarrow t$ 的最大流等于 s, t 之间的最小割。现在将该定理应用在一般无向图上：

- $s \rightarrow t$ 的最大流等于 f 表示存在 f 条 $s \rightsquigarrow t$ 的边不相交的迹，且 f 不能更大。
- s, t 之间的最小割等于 c 表示存在 c 条边使得断开这些边后 s, t 不连通，且 c 不能更小。

聪明的读者已经想到，如果将定理应用在边双连通图上，对于任意 $u \neq v$ ，都需要断开至少两条边才能使它们不连通，所以它们之间至少存在两条边不相交的迹（可以在结点处相交），继而得到边双最重要的性质之一。

结论

对于边双内任意两点 u, v ，存在经过 u, v 的回路。

对于点双，目标是割掉（删去）若干个点使得 s, t 不连通。应用网络流点边转化的技巧，建新图 G' ，将除了 s, t 的每个点 u 拆成两个点 u_{in} 和 u_{out} ， u 的所有邻居 $v \in N(u)$ 向 u_{in} 连边容量为 $+\infty$ ， u_{out} 向所有 $v \in N(u)$ 连边容量为 $+\infty$ ，再从 u_{in} 向 u_{out} 连边容量 1。连容量为 $+\infty$ 的边是为了防止割掉这些边，因为它们对应原图的边，而希望割掉的是点。

- G' 上 $s \rightarrow t$ 的最大流等于 f 表示存在 f 条 $s \rightsquigarrow t$ 的点不相交（不在除了 s, t 以外的点相交）的路径，且 f 不能更大。
- G' 上 s, t 之间的最小割等于 c 表示存在 c 个点使得删去这些点后 s, t 不连通，且 c 不能更小。

特别地，当 s, t 相邻时， $f = +\infty$ ，表示无论删去多少个点都无法使得 s, t 不连通。

结论

对于点双内任意两点 u, v ，存在经过 u, v 的简单环。

结论

对于 $n \geq 3$ 的点双内任意两点，存在经过 u, v 的长度不小于 3 的简单环。

证明

若 u, v 不直接相邻，使用上述结论。若 u, v 直接相邻，因为删去 (u, v) 后整张图仍连通（否则 u, v 至少有一个是割点），所以将 (u, v) 接在图上 u, v 之间的路径即可。□

通过上述铺垫，可以很自然地推出 Menger 定理：

边形式

对于无向图 G 上任意不同的两点 u, v ，使得 u, v 不连通所需删去的的边的数量的最小值，等于 u, v 之间边不相交的迹的数量的最大值。

点形式

对于无向图 G 上任意不同且不相邻的两点 u, v ，使得 u, v 不连通所需删去的的点的数量的最小值，等于 u, v 之间点不相交（不在除了 u, v 以外的点相交）的路径数量的最大值。

根据 Menger 定理, 对于无向图 G 上任意两点 $u \neq v$, 可以定量描述它们关于边的连通性和关于点的连通性。

- 局部边连通度: 使得 u, v 不连通所需删去的边的数量的最小值为 u, v 的 **局部边连通度**, 记作 $\lambda(u, v)$ 。它等于 u, v 之间边不相交的迹数量的最大值。
- 局部点连通度: 对于 $(u, v) \notin E$, 使得 u, v 不连通所需删去的点的数量的最小值为 u, v 的 **局部点连通度**, 记作 $\kappa(u, v)$ 。它等于 u, v 之间点不相交的路径数量的最大值。
- k -边连通: 若 $\lambda(u, v) \geq k$, 则称 u, v 之间是 **k -边连通** 的。
- k -点连通: 若 $\kappa(u, v) \geq k$, 则称 u, v 之间是 **k -点连通** 的。

在此基础上, 可以定量描述整张图的连通性。

- 全局边连通度: 使得存在两个点不连通所需删去的边的数量的最小值为 G 的 **全局边连通度**, 记作 $\lambda(G)$ 。它等于任意两点局部边连通度的最小值, 即 $\lambda(G) = \min_{u \neq v} \lambda(u, v)$ 。特殊定义 $\lambda(K_1) = +\infty$ 。
- 全局点连通度: 使得存在两个点不连通所需删去的点的数量的最小值为 G 的 **全局点连通度**, 记作 $\kappa(G)$ 。它等于任意不相邻两点局部点连通度的最小值, 即 $\kappa(G) = \min_{u \neq v \wedge (u, v) \notin E} \kappa(u, v)$ 。因 K_n 不存在不相邻的两点, 特殊定义 $\kappa(K_n) = n$ 。
- k -边连通图: 对于 $k \in \mathbb{N}$, 若 $\lambda(G) \geq k$, 则称 G 是 **k -边连通图**, 其性质称为 **k -边连通性**。它可以等价表述为: 删去任意不超过 $k - 1$ 条边, 整张图仍然连通。
- k -点连通图: 对于 $k \in \mathbb{N}$, 若 $\kappa(G) \geq k$, 则称 G 是 **k -点连通图**, 也称 **k -连通图**, 其性质称为 **k -点连通性**。它可以等价表述为: $|G| \geq k$ 且删去任意不超过 $k - 1$ 个点, 整张图仍然连通。

将 Menger 定理推广至任意点对, 得到以下结论:

边形式推广

一张图是 k -边连通图当且仅当每对点之间有 k 条边不相交的迹。

点形式推广

一张图是 k -连通图当且仅当 $|G| \geq k$ 且每对点之间有 k 条点不相交的路径。

3.2.4 双连通总结

现在我们知道为什么不存在割边(点)的图叫作边(点)双连通图了: 不存在割边说明 $\lambda(G) \geq 2$, 即不存在割边的图为 2-边连通图。不存在割点说明 $\kappa(G) \geq 2$, 即不存在割点的图为 2-点连通图。

根据 Menger 定理, 一张图不存在割边等价于对任意两点 u, v , 存在经过 u, v 的回路。从一条性质出发可以推出另一条性质, 称前者为边双连通图的定义, 后者为边双连通图的基本性质。根据边双连通的传递性, 每条边属于至少一个回路的图也是边双连通图。类似地, 一张图不存在割点等价于对任意两点 u, v , 存在经过 u, v 的环。称前者为点双连通图的定义, 后者为点双连通图的基本性质。

目前为止我们都是单独研究边双和点双，接下来探究它们的联系。下文讨论的点双忽略了“一边连两点”等平凡情况，默认 $n \geq 3$ 。

如果一张点数大于 2 的图没有割点，那么它一定没有割边。假设存在割边，那么删去割边后大小大于 1 的连通块对应的割边端点一定是割点。因此，任何边双满足的性质，点数大于 2 的点双一定满足（这里的逻辑有点绕，可以这么想： p 能推出 q ，那么 q 推出的性质也能由 p 推出）。也就是说，**点双连通的性质比边双连通更强**。这一点在它们的基本性质中也有体现：不经过重复点一定不经过重复边，两条无重复点的路径一定是两条无重复边的迹，环一定是回路，所以 **点双连通推出边双连通**。

- 根据基本性质，一般“不经过重复边”的问题借助边双解决，而“不经过重复点”的问题借助点双解决。
- 每个点恰属于一个边双，每条边可能恰属于一个边双（非割边），也可能不属于任何边双（割边）；每条边恰属于一个点双，每个点可能属于一个点双（非割点），也可能属于多个点双（割点）。

3.2.5 点双连通的更多性质

以下假设图无自环。

在点双基本性质的基础上，可以推出一些点双的常用性质。

基本性质：对于 $n \geq 3$ 的点双中任意两点 $x \neq y$ ，存在经过 x, y 的简单环。

性质 1

对于 $n \geq 3$ 的点双中任意一点 x 与一边 e ，存在经过 x, e 的简单环。

证明

将 $e = (u, v)$ 拆成 (u, w) 和 (w, v) 不影响点双连通性。根据基本性质，存在经过 x, w 的简单环。因 w 仅与 u, v 相连，故 $(u, w), (w, v)$ 在环上，将这两条边替换为 (u, v) 得经过 x, e 的简单环。

不影响点双连通性的证明：

- 删去 u 或 v ，由 w 与 v 或 u 连通且删去 u 或 v 后原图连通可知 u, v 不是割点。这一步要求 $u \neq v$ ，用到了无自环的条件。
- 删去 u, v, w 以外的点，将 (u, w) 和 (w, v) 视为 (u, v) ，原图连通。
- 删去 w ，相当于删去 (u, v) ，若原图不连通则 (u, v) 为割边，当 $n \geq 3$ 时 u 或 v 为割点，矛盾。□

实际上，钦定经过一条边和一个点是几乎等价的（除非钦定经过 x 或 y ），它们可以相互归约：若证明了可以钦定经过一条边，则钦定经过某点时，钦定经过该点任意邻边即可。若证明了可以钦定经过一个点，则钦定经过某条边时，将该边像上述证明一样拆成两条边，并钦定经过中间点即可。

性质 2

对于 $n \geq 3$ 的点双中任意不同两点 $x \neq y$ 与一边 e , 存在 $x \rightsquigarrow e \rightsquigarrow y$ 的简单路径。

证明

由性质 1, 存在经过 x, e 的简单环 C , 若 $y \in C$ 则结论成立。否则令 P 为任意 $y \rightsquigarrow x$ 的有向路径, 考虑 P 上第一个属于 C 的交点 z , 存在使得 $z \neq x$ 的 P , 否则 x 为割点: 删去 x 后 y 无法到达 C 的剩余结点。令路径 Q 为 P 上 $y \rightsquigarrow z$ 的部分, 接上 z 通过 C 上有 e 的一侧到 x 的路径, 则 Q 即为所求。□

性质 3

对于 $n \geq 3$ 的点双中任意不同三点 x, y, z , 存在 $x \rightsquigarrow z \rightsquigarrow y$ 的简单路径。

证明

考虑某条以 z 为端点的边 e 。由性质 2, 存在 $x \rightarrow e \rightarrow y$, 因此存在 $x \rightarrow z \rightarrow y$ 。□

3.3 Tarjan 求割点

前置知识: DFS 树, DFS 序。

注意区分: DFS 序表示对一张图 DFS 得到的结点序列, 而时间戳 dfn 表示每个结点在 DFS 序中的位置。

记 x 的子树为 x 在 DFS 树上的子树, 包含 x 本身, 记作 $T(x)$ 。记 $T'(x) = V \setminus T(x)$, 即整张图除了 $T(x)$ 以外的部分。

不妨认为 G 是无向连通图。对于非连通图, 对每个连通分量分别求割点。

笔者希望提出一种新的理解 Tarjan 算法的方式。网上大部分博客讲解 Tarjan 算法时 low 数组凭空出现, 抽象的定义让很多初学者摸不着头脑, 从提出问题到解决问题的逻辑链的不完整性让我们无法感受到究竟是怎样的灵感启发了这一算法的诞生。

3.3.1 非根结点的割点判定

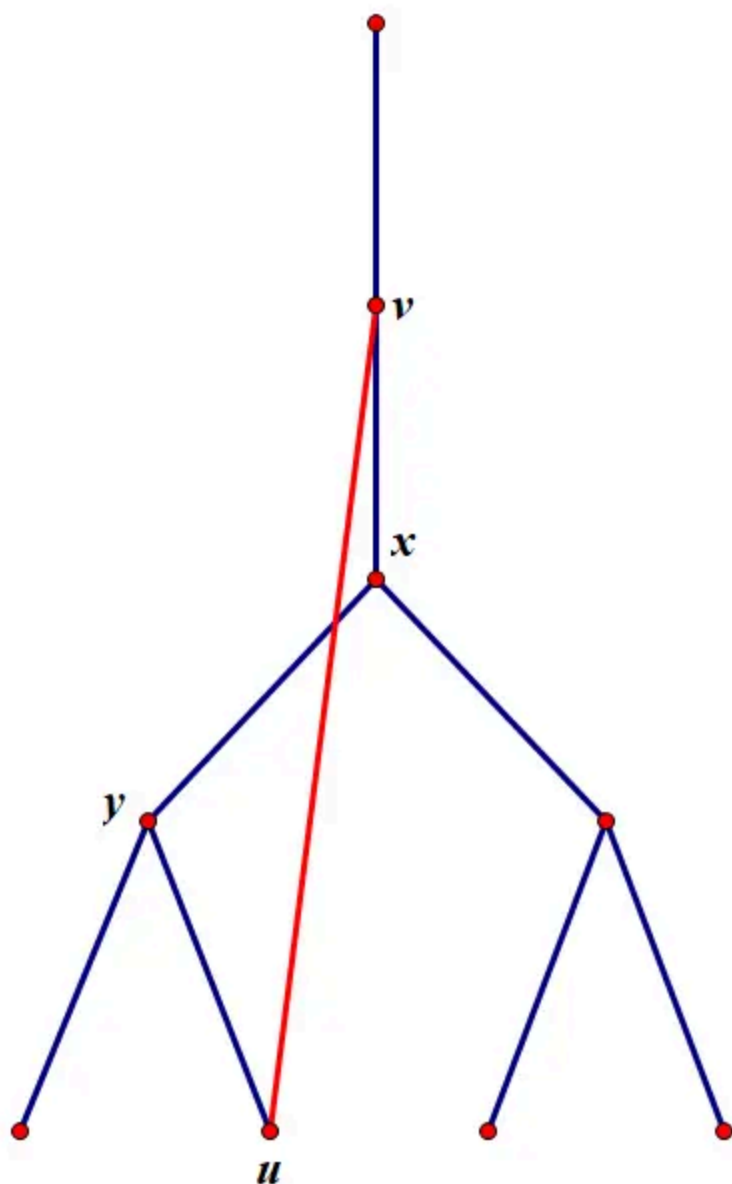
设 x 不为 DFS 树的根, 则 $T'(x)$ 非空。

若 x 是割点, 则删去 x 之后, 对于 $z \in T'(x)$, 存在 y 和它不连通。而删去 x 之后 $T'(x)$ 通过树边仍然连通, 所以 $y \in T(x)$ 。而如果 y 和 z 不连通, 又因为 $T'(x)$ 连通, 那么 y 和所有 $T'(x)$ 的点均不连通。

反之，若删去 x 之后存在 $y \in T(x)$ 和 $T'(x)$ 的点均不连通，那么 x 显然是割点。这说明 x 是割点当且仅当存在 $y \in T(x)$ 不经过 x 能到达的所有点均属于 $T(x)$ 。

现在要刻画“不经过 x 能到达的所有点均属于 $T(x)$ ”。

注意到，如果 $y \in T(x)$ 不经过 x 就和 $T'(x)$ 连通，那么存在 y 到 $v \in T'(x)$ 的路径，满足 v 是路径上第一个属于 $T'(x)$ 的结点。设路径上倒数第二个点为 u ，则 $u \in T(x)$ 。如果 (u, v) 是树边，那么 $u = x$ ，矛盾。因此 (u, v) 是非树边，那么 v 是 u 的祖先（祖先后代性）。又因为 x 是 u 的祖先且 v 在 x 的子树外，所以 v 是 x 的祖先。如下图。



洛谷@Alex_Wei

进一步地，因为 x 的不同儿子子树之间没有非树边（子树独立性），设 x 的儿子 y' 的子树包含 y ，那么 $u \in T(y')$ 。

因此，如果 y 不经过 x 和 $T'(x)$ 连通，即 x 不是割点，那么存在 $u \in T(y')$ 使得 u 可以通过一条非树边到达 x 的祖先。设 f_x 表示与 x 通过 **非树边** 相连的所有点的时间戳的最小值，则条件可写为 $f_u < d_x$ 。

- 对于 $T(y')$ ，如果存在 $u \in T(y')$ 满足 $f_u < d_x$ ，那么删去 x 后 $T(y')$ 的每个点和 $T'(x)$ 均连通： $T(y')$ 内所有点通过树边连通，且 u 和 $T'(x)$ 某点直接相连。
- 反之，如果 $T(y')$ 内所有点的 f 值均不小于 d_x ，那么删去 x 后 $T(y')$ 的每个点和 $T'(x)$ 均不连通。因为如果连通，那么总得有一个点能一步连通。

这样，我们得到了非根结点的割点判定法则：

x 是割点当且仅当存在树边 $x \rightarrow y'$ ，使得 y' 子树 **不存在** 点 u 使得 $f_u < d_x$ 。

这等价于存在 x 的儿子 y' ，满足 $\min_{u \in T(y')} f_u \geq d_x$ 。

设 g_x 表示 x 的子树内所有点 $u \in T(x)$ 的 f_u 的最小值（low 的真正含义），根据树形 DP，有

$$g_x = \min \left(\min_{y' \in \text{son}(x)} g_{y'}, \min_{(x,y) \in E \wedge (x,y) \notin T} d_y \right)$$

对于后半部分，忽略 (x, y) 必须是非树边的条件不会导致错误：如果用儿子更新，显然没有问题。如果用父亲更新，即用 d_x 更新 g_y ，也不会导致错误，因为判定是 $g_y \geq d_x$ ，有等号。但注意求解割边时不能忽略，因为判定是 $g_y > d_x$ 。

特别地，若使用树边更新，则 g_y 会被 d_x 更新到，使得 $g_y \leq d_x$ 。此时判定条件等价于 $g_y = d_x$ 。

说明：将 g_x 初始化为 d_x 显然不会导致错误。

应用：研究删去 x 后整张图的形态。删去 x 后，每个判定 x 为割点的 y' 的 $T(y')$ 单独形成一个连通块，剩余部分（其它所有 $T(y')$ 和 $T'(x)$ ）形成一个连通块。因为判定割点的准则就是删去 x 后 y' 是否与 $T'(x)$ 连通。

- 一个小结论：由 Tarjan 的过程可知只保留最浅的返祖边不改变图的点双连通性。在双极定向时有用（见图论 II）。

3.3.2 根的割点判定、代码

设 x 为 DFS 树的根。

若 x 在 DFS 树上有大于一个儿子，根据子树独立性，删去 x 后各儿子子树不连通，所以 x 是割点。反之删去 x 后剩余部分通过树边连通， x 不是割点。

综上，使用 Tarjan 算法求无向图 G 的所有割点的时间复杂度为 $\mathcal{O}(n + m)$ 。

再次强调，以下代码仅在求割点时正确。求割边需要额外的特判。

模板题 代码。

```
#include <bits/stdc++.h>
using namespace std;
constexpr int N = 1e5 + 5;
int n, m, R;
int dn, dfn[N], low[N], cnt, buc[N]; // dfn 是时间戳 d, low 是 g
vector<int> e[N];
void dfs(int id) {
    dfn[id] = low[id] = ++dn; // 将 low[id] 初始化为 dn 不会导致错误，一般都这么写
    int son = 0;
    for(int it : e[id]) {
        if(!dfn[it]) {
            son++, dfs(it), low[id] = min(low[id], low[it]);
            if(low[it] >= dfn[id] && id != R) cnt += !buc[id], buc[id] = 1; // 写 low[it] == dfn[id] 会 t
        }
        else low[id] = min(low[id], dfn[it]);
    }
    if(son >= 2 && id == R) cnt += !buc[id], buc[id] = 1;
}
int main() {
    cin >> n >> m;
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v), e[v].push_back(u);
    }
    for(int i = 1; i <= n; i++) if(!dfn[i]) R = i, dfs(i);
    cout << cnt << endl;
    for(int i = 1; i <= n; i++) if(buc[i]) cout << i << " ";
    return 0;
}
```

例题：P3469。

3.4 割边的求法

3.4.1 Tarjan

Tarjan 求割边的思路和求割点的思路类似。

探究一条边是割边的充要条件。

首先，树边使得整张图连通，所以割掉非树边不影响连通性。因此 $e = (u, v)$ 是割边的 **必要条件** 是 e 为树边。

不妨设 v 是 u 的儿子。若割掉 e 后图不连通，那么因为 $T(v)$ 和 $T'(v)$ 内部通过树边连通，所以图只能分裂为 $T(v)$ 和 $T'(v)$ 两部分。这说明 $T(v)$ 的所有结点必须通过 e 才能到达 $T'(v)$ ，即 e 是连通 $T(v)$ 内外的“桥”。

考虑如何判定这种情况。根据求解割点的经验，不难得出 e 为割边当且仅当 $g_v > d_u$ 。对比割点判定法则，没有等号的原因为：删去的是边而非结点，所以只要子树内结点能绕过 e 到达 $T'(v)$ ，包括 u 本身，那么 e 就不是割边。

Tarjan 求割边有个细节，就是判断非树边。对于当前边 $u \rightarrow v$ ，若 v 是 u 的父亲，那么跳过这条边。这样做在有重边时会出现错误，因为会将树边的重边也判为树边。

解决方法是记录边的编号。对于 vector，在 push_back 时将当前边的编号一并压入。对于链式前向星，使用成对变换技巧：初始化 $\text{cnt} = 1$ ，每条边及其反边在链式前向星中存储的编号分别为 $2k$ 和 $2k + 1$ ，将当前边编号异或 1 即得反边编号。

算法的时间复杂度为 $\mathcal{O}(n + m)$ 。

3.4.2 树上差分法

求出 G 的任意一棵 DFS 树 T ，割边只能是树边。容易发现，一条树边是割边当且仅当不存在非树边 (u, v) 使得该树边属于 u, v 在 T 上的简单路径。

- 如果存在，那么删去树边后 T 分裂出的两部分通过 (u, v) 连接，树边不是割边。
- 否则，删去后 T 分裂出的两部分不连通，树边是割边。

对于非树边 (u, v) ，将 u, v 之间所有边标记为非割边。根据非树边的祖先后代性，树上差分即可。

算法的时间复杂度为 $\mathcal{O}(n + m)$ 。

3.5 边双连通分量缩点

3.2.1 小节有结论：将 G 的割边删去，剩余每个连通分量是原图的边双连通分量。得到原图边双连通分量之后，记录每个点所在边双编号，容易对 G 边双缩点。

接下来介绍 Tarjan 算法实现的边双连通分量缩点，类似方法可应用在点双缩点上。关于点双缩点，见“图论 II”广义圆方树。

可以先 Tarjan 找出所有割边，再 DFS 找出所有边双连通分量，但太麻烦。我们希望一遍 Tarjan 就可以求出所有割边和边双连通分量。为此，需要对算法进行一些改进。

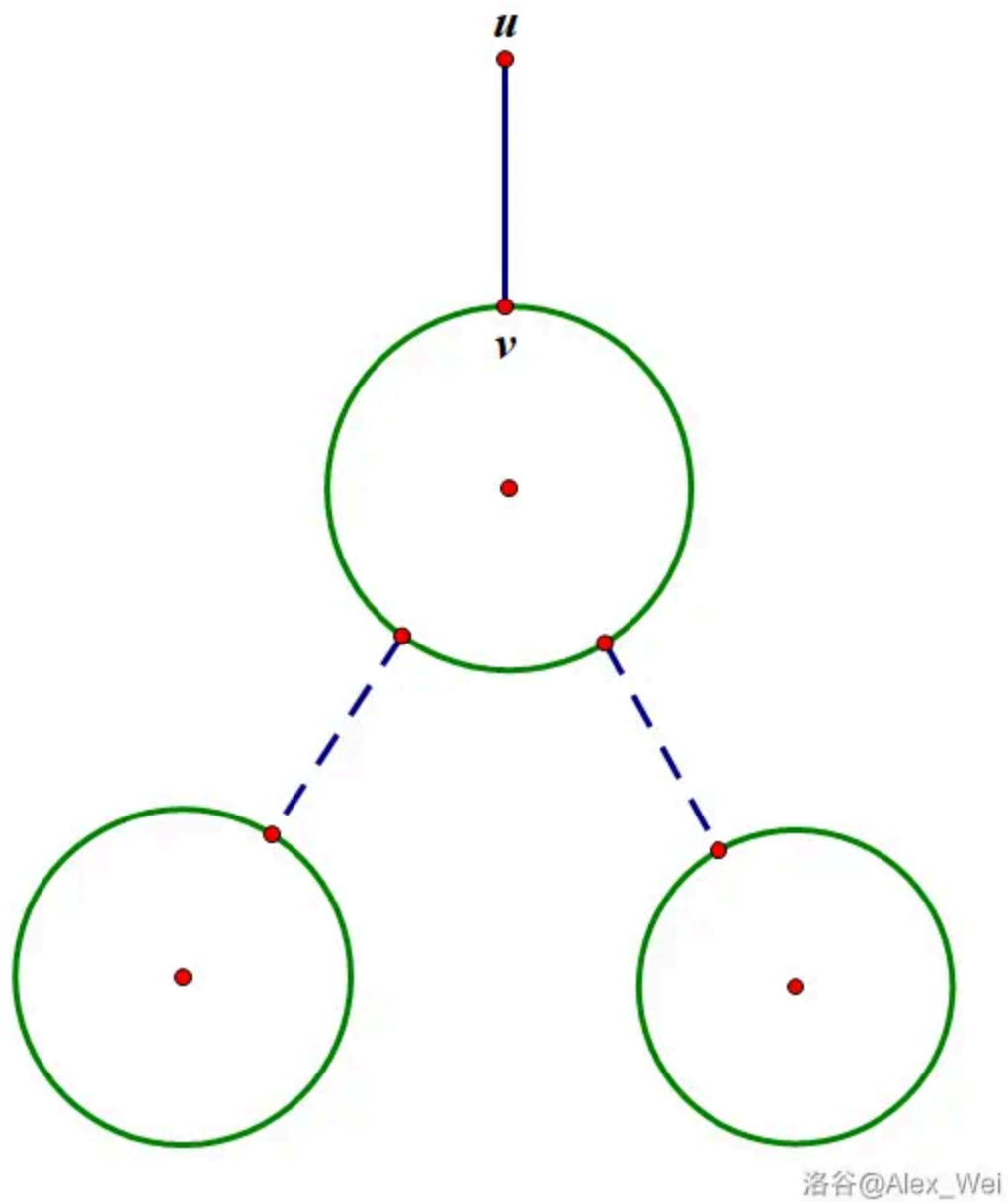
考虑 G 的 DFS 树 T ，找到其中任意一条割边，满足子树内没有割边。这样，该割边的子树内所有结点形成边双连通分量。将这些结点从图中删去，包括与它们相邻的所有边。重复该过程直到整张图不含割

边，则剩下的图也是原图的边双连通分量。整个过程可以视为在边双缩点树上 **剥叶子**，不断地断开一条与叶子相连的割边，剥下一个边双连通分量。如果有 c 条割边，那么有 $c + 1$ 个边双连通分量。

考虑将上述操作与 Tarjan 算法相结合：回溯时，若判定 $u \rightarrow v$ 为割边，则 v 的子树内还没有被删去的点形成一个边双，将它们全部删去。具体地，维护栈 S ，表示已经访问过且还未被删去的所有结点。每次 DFS 进入一个结点时，将其压入栈，那么从栈底到栈顶时间戳依次递增。遇到割边 $u \rightarrow v$ 时，从栈顶到 v 的所有结点构成一个边双，将它们全部弹出。

最后栈内剩下一些点，它们单独形成一个边双，且包含根结点。不要忘记将它们弹出。

正确性说明（直观理解，非严格证明）：设原图割掉 (u, v) 后形成的连通块分别为 $u \in U$ 和 $v \in V$ 。由于判定 $u \rightarrow v$ 为割边是在回溯时进行的，所以 V 中所有结点全部被访问过，且全部在 v 之后访问，即栈中从 v 往上一直到栈顶均属于 V ，这些结点就是我们要删除的。对于 V 内部的所有割边（下图的虚边），我们在判定它们为割边时就已经处理掉了它们对应的边双，所以弹出结点内部也不会有割边。



上述思想在点双缩点和强连通分量缩点时也会用到。

算法的时间复杂度 $\mathcal{O}(n + m)$ 。

[模板题](#) 代码。

```

#include <bits/stdc++.h>
using namespace std;
constexpr int N = 5e5 + 5;
int n, m;
vector<pair<int, int>> e[N];
vector<vector<int>> ans;
int dn, dfn[N], low[N], stc[N], top;
void form(int id) {
    vector<int> S;
    for(int x = 0; x != id; ) S.push_back(x = stc[top--]);
    ans.push_back(S);
}
void tarjan(int id, int eid) {
    dfn[id] = low[id] = ++dn;
    stc[++top] = id;
    for(auto _ : e[id]) {
        if(_.second == eid) continue;
        int it = _.first;
        if(!dfn[it]) {
            tarjan(it, _.second);
            low[id] = min(low[id], low[it]);
            if(low[it] > dfn[id]) form(it);
        }
        else low[id] = min(low[id], dfn[it]);
    }
}
int main() {
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> m;
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        e[u].push_back({v, i});
        e[v].push_back({u, i});
    }
    for(int i = 1; i <= n; i++) {
        if(!dfn[i]) tarjan(i, 0), form(i);
    }
    cout << ans.size() << "\n";
    for(auto S : ans) {
        cout << S.size() << " ";
        for(int it : S) cout << it << " ";
        cout << "\n";
    }
    return 0;
}

```

例题：P2860，CF51F。

3.6 例题

P3469 [POI2008] BLO-Blockade

一道 Tarjan 求割点的练手题。

设删去与结点 u 相连的所有边之后形成的连通块大小分别为 $s_1 \sim s_k$ ，则答案为 $\sum_{i=1}^k s_i(n - s_i)$ 。注意，不要忘记 u 没有被删去，它本身是一个大小为 1 的连通块。

因为 v 判定 u 为割点当且仅当封锁 u 之后 v 及其子树与整张图剩余部分不连通，所以考虑所有判定 x 为割点的 y_i ，它们的子树分别单独形成连通块。除去这些结点后，还有一个大小为 $n - 1 - \sum size(y_i)$ 的连通块（可能为空，但不影响答案）。

时间复杂度 $\mathcal{O}(n)$ 。[代码](#)。

双倍经验：[SP15577 Blockade](#)。

P2860 [USACO06JAN] Redundant Paths G

题目相当于添加最少的边使得整张图变成一个边双。

考虑边双缩点得到缩点树 T ，其上所有边都是原图割边。显然，如果在 u, v 之间加一条边，设 U 为 u 所在边双对应的缩点树上的结点， V 同理，则相当于将 U, V 简单路径上的所有边变成非割边。

我们希望用最少的路径覆盖 T 的每一条边。对此有经典结论：最小路径数为 T 的叶子个数除以 2 上取整。

证明这是答案下界非常容易，因为每条链至多覆盖两个叶子到它唯一相邻的点的边。当只有两个点的时候特殊讨论。接下来给出一个达到该下界的构造方法。

称两个叶子匹配表示在最终方案中，存在一条连接它们的链。

首先，当叶子个数为奇数时，考虑任意一个叶子 u ，从它出发找到第一个度数 ≥ 3 的结点 v ，令 u, v 匹配，转化为叶子个数为偶数的情况。如果不存在度数 ≥ 3 的结点，则 T 为一条链，与叶子个数为奇数矛盾。

先将所有叶子任意两两匹配，再调整。设在当前方案中，存在一条边 (u, v) 没有被覆盖，设断开 (u, v) 后 u, v 分别属于连通块 U, V 。

U 或 V 不可能只有一个原树的叶子，否则该叶子和另外某个叶子匹配时必然经过 (u, v) ，与 (u, v) 没有被覆盖矛盾。同理可证 U 或 V 不可能有奇数个叶子。

因此，当前方案必然是 U 的所有偶数个叶子两两匹配， V 的所有偶数个叶子两两匹配。设 U 的某两个匹配的叶子为 u_1, u_2 ，它们在以 u 为根时的 LCA 为 u_d 。对于 V ，类似定义 v_1, v_2 和 v_d 。

当前方案覆盖了 $u_1 \rightsquigarrow u_d \rightsquigarrow u_2$ 和 $v_1 \rightsquigarrow v_d \rightsquigarrow v_2$ 的所有边，但令 u_1 和 v_1 ， u_2 和 v_2 匹配， $u_i \rightsquigarrow u_d \rightsquigarrow u \rightarrow v \rightsquigarrow v_d \rightsquigarrow v_i$ ($i = 1, 2$) 的所有边被覆盖。原来被覆盖的边仍被覆盖，同时 (u, v) 也被覆盖了。

因此，对于当前方案，若某条边没有覆盖，通过上述调整一定在不改变链的条数使得原来被覆盖的边仍被覆盖，且该边也被覆盖。答案上界得证。

时间复杂度 $\mathcal{O}(n + m)$ 。代码。

CF51F Caterpillar

题目要求不能存在环，所以首先将所有边双缩成一个点。缩点后整张图会变成一棵森林，先处理每一棵树，再用连通块数量 -1 次操作将它们连起来。

不妨认为原图连通。

考虑确定主链后如何用最少的操作使得一棵树变成毛毛虫。

对于除了主链以外的结点，考虑它是否作为最终挂在主链旁边的叶子。将主链看成根，具有祖先后代关系的点对不能同时被选中作为叶子，因为此时后代和主链之间隔了一个祖先，说明它到主链的距离 ≥ 2 ，与限制不符。

问题转化为选择最多的点使得它们之间没有祖先后代关系。我们的决策是选择所有叶子，因为若一个非叶子被选中，我们一定可以取消选择它，并选它的所有儿子。

因此，最终保留的结点为主链上的结点和我们选中作为叶子的结点。这说明对于路径 P ，它作为主链时对应的最小操作次数为 $n - |P| - L$ ，其中 n 是树的大小， L 是主链外叶子的数量。

当 P 的两端不是叶子时，总可以调整使得 P 的两端是叶子，且 $n - |P| - L$ 不变或变小：向任意方向延伸，每次 $|P|$ 增加 1，且 L 要么不变（当延伸的点不是叶子），要么减少 1（当延伸的点是叶子）。

设缩点树的叶子数量为 K ，则操作次数为 $n - K + 2 - |P|$ 。 $n - K + 2$ 是定值，我们希望 $|P|$ 最大，自然是选取树的直径，同时恰好满足了 P 的两端是叶子的要求。

时间复杂度 $\mathcal{O}(n + m)$ 。代码。

4. 有向图可达性：强连通分量

研究有向图可达性时，强连通分量是最基本的结构。

本章研究强连通分量。默认图为 **有向弱连通图**。

4.1 相关定义

- 强连通：对于有向图的两点 u, v ，若它们相互可达，则称 u, v **强连通**，这种性质称为 **强连通性**。

显然，强连通是等价关系，**强连通性具有传递性**。

- 强连通图：满足任意两点强连通的有向图称为 **强连通图**。它等价于图上任意点可达其它所有点。
- 强连通分量：有向图的极大强连通子图称为 **强连通分量**（Strongly Connected Component，SCC）。

强连通分量在求解与有向图可达性相关的题目时很有用，因为在只关心可达性时，同一强连通分量内的所有点等价。

有了使用 Tarjan 算法求割点，割边和边双缩点的经验，我们可以自然地将这些方法进行调整后用于强连通分量缩点。不过在此之前，我们需要研究有向图 DFS 树的性质，毕竟整个过程离不开 DFS。

4.2 有向图 DFS 树

不同于无向图，对于弱连通图 G ，从某一点出发 DFS，不一定能访问到图上的所有结点。一般而言，有向图的“DFS 树”是一个森林：按任意顺序遍历每个结点，若当前点没有 DFS 过，则从该点开始 DFS，并按构建无向图 DFS 树的方式构建以该点为根的有向图 DFS 树。这会形成若干棵 DFS 树，在研究强连通性时，它们之间相互独立。对于任意强连通的两点 u, v ，第一次访问它们所在的强连通分量的 DFS 树一定同时包含 u, v 。因此不同 DFS 树之间的任意两点不强连通，只需考虑一棵 DFS 树。

考察有向图 DFS 树 T 的形态，从而得到判定 SCC 的准则。

除了有向树边以外， T 还有一些边：

- 从祖先指向后代的非树边，称为 **前向边**。
- 从后代指向祖先的非树边，称为 **返祖边**。
- 两端无祖先后代关系的非树边，称为 **横叉边**。

前向边和返祖边由无向图 DFS 树的非树边的两种定向得到，而横叉边则是无向图 DFS 树所不具有的，因为无向图 DFS 树的非树边具有祖先后代性。

为什么有向图 DFS 树会出现横叉边？因为一个后访问到的结点可以向之前访问的结点连边。例如 $G = (\{1, 2, 3\}, \{1 \rightarrow 2, 1 \rightarrow 3, 3 \rightarrow 2\})$ ，如果我们在 DFS 时先访问结点 2，那么在访问结点 3 时就会找到横叉边 $3 \rightarrow 2$ 。

接下来讨论有向边 $u \rightarrow v$ ，设 $d = lca(u, v)$ ， $d \rightarrow u$ 的第一个结点为 u' （若存在）， v' 同理。设 $fa(i)$ 表示 i 在 T 上的父亲。设 $T(i)$ 表示 i 的子树。设 $dfn(i)$ 表示点 i 的时间戳。

尽管子树的独立性变弱了，但我们依然可以发现一些性质：

- 对于前向边 $u \rightarrow v$, u 本身就能通过树边到达 v , 删去后没有任何影响。它甚至不影响可达性。
- 对于返祖边 $u \rightarrow v$, 它使 T 上从 v 到 u 的路径上的所有点之间强连通。多个返祖边结合可以形成更大更复杂的强连通结构。
- 对于横叉边 $u \rightarrow v$, $dfn(v) < dfn(u)$ 。否则 u 先被访问, 那么无论是 u 的儿子访问到了 v , 还是 u 本身直接访问 v , 离开 u 之前 v 一定被访问过了, 即 $v \in T(u)$ 。那么 $u \rightarrow v$ 为前向边, 矛盾。

综合上述三条性质, 可知返祖边和横叉边均减小时间戳, 只有树边增大时间戳。前向边不影响强连通性, 所以忽略掉所有前向边。

考虑横叉边 $u \rightarrow v$, 则 $dfn(v) < dfn(u)$ 。为了探究横叉边对强连通性的影响, 对于 $dfn(v) < dfn(u)$ 的点对 (u, v) , 我们希望知道 v 可达 u 的充要条件。

若 v 可达 u , 那么 $v \rightsquigarrow u$ 的路径上存在一条边 $x \rightarrow y$ 使得 $dfn(x) < dfn(u) \leq dfn(y)$ 。因为 $dfn(x) < dfn(y)$, 所以 $x \rightarrow y$ 是树边。当 $y = u$ 时, 显然 $u \in T(x)$; 否则 $y \neq u$, 则 u 在 x 之后, y 之前被访问, 则 $u \in T(x)$ 。因此, 若 $dfn(v) < dfn(u)$ 且 v 可达 u , 则 v 可达 $fa(u)$ 。对 u, u' 的树上路径上的所有点依次使用该结论, 推出 v 可达 d 。

反之, 若 v 可达 d , 显然 v 可达 u 。

结论

若 $dfn(v) < dfn(u)$, 则 v 可达 u 当且仅当 v 可达 d 。

- 一个更深刻的结论和理解见图论 II 的 4.3 小节引理 2 (一般图支配树)。

因此, 对于横叉边 $u \rightarrow v$, 它对强连通性产生影响当且仅当 v 可达 d 。此时 u, v 强连通。称这些边为有用的横叉边。

该结论可以进一步推出:

结论

若 u, v 强连通, 则 u, v 在树上路径上的所有点强连通。

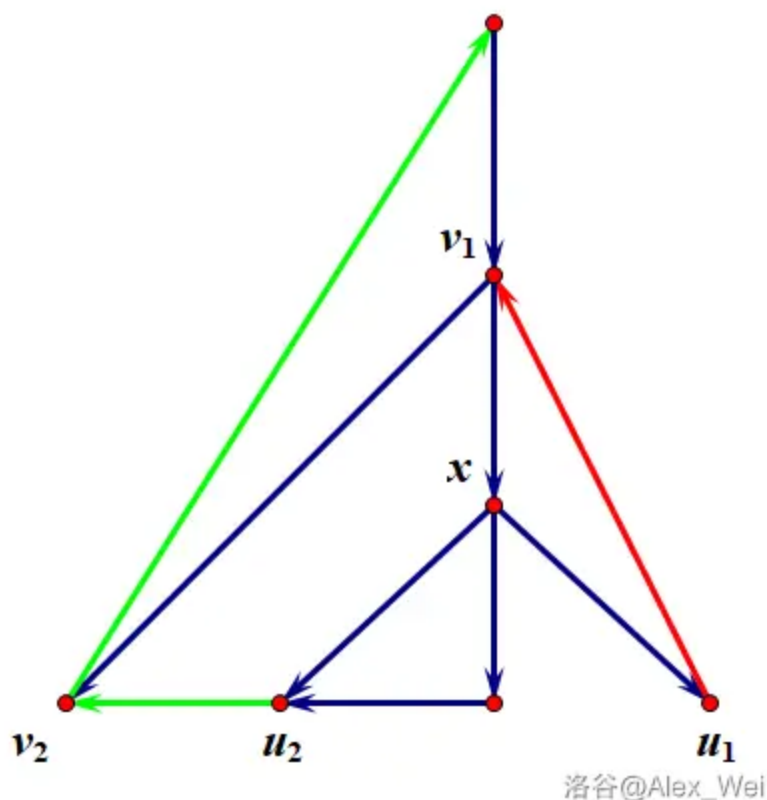
结论

强连通分量在有向图 DFS 树上弱连通。

4.3 Tarjan 求 SCC

因为每个 SCC 在 T 上的最浅结点唯一, 故考虑在最浅的结点处求出包含它的 SCC。称一个点是关键点, 当且仅当它是某个 SCC 的最浅结点。

考虑判定关键点。如果 x 不是关键点，那么因为 SCC 在 DFS 树上弱连通，所以 x 的父亲 fa 和 x 强连通。考虑 x 到 fa 的路径，存在一条边跳出 $T(x)$ ，可知 x 的子树内存在点 u ，使得从 u 出发存在返祖边 $u \rightarrow v$ 满足 v 是 x 的祖先（下图的红色部分），或存在横叉边 $u \rightarrow v$ 满足 v 可达 x 的祖先且 $v \notin T(x)$ （下图的绿色部分）。无论哪种情况均有 $dfn(v) < dfn(x)$ 。



因此，设 g_x 表示 $T(x)$ 内所有结点 u 的「返祖边 $u \rightarrow v$ 」和「 v 可达 d 的横叉边 $u \rightarrow v$ 」的所有 v 的最小时间戳。 x 是关键点当且仅当 $g_x \geq dfn(x)$ 。

证明

考虑证明 x 不是关键点当且仅当 $g_x < dfn(x)$ 。

充分性：若 $g_x < dfn(x)$ ，那么存在 $u \in T(x)$ ，使得从 u 出发存在返祖边或有用的横叉边 $u \rightarrow v$ 满足 $dfn(v) < dfn(x)$ 。如果 $u \rightarrow v$ 是返祖边，那么 v 显然是 x 的祖先；如果 $u \rightarrow v$ 是横叉边，那么 v 可达 d ，又因为 $dfn(d) < dfn(v) < dfn(x)$ 且 d, x 均为 u 的祖先，所以 d 是 x 的祖先。无论哪种情况，都说明包含 x 的 SCC 包含 x 的祖先， x 不是关键点。

必要性：若 x 不是关键点，那么 x 可达 $fa(x)$ 。考虑任意一条 $x \rightsquigarrow fa(x)$ 的路径，找到第一次离开 $T(x)$ 的边 $u \rightarrow v$ 。若 $u \rightarrow v$ 是返祖边，那么 v 是 x 的祖先；若 $u \rightarrow v$ 是横叉边，那么 $dfn(v) < dfn(x)$ 且 v 可达 $fa(x)$ ，则 v 可达 u ，则 v 可达 d 。无论哪种情况，都说明 $g_x < dfn(x)$ 。□

说明：一般将 g_x 初始化为 $dfn(x)$ ，判定条件为 $g_x = dfn(x)$ 。

类似边双缩点，这次我们依然采用“剥叶子”的方式：维护栈 S ，表示已经访问过但还没有被删去（未形成确定 SCC）的结点。在回溯时判定关键点。若 u 为关键点，则将栈顶一直到 u 的所有结点弹出，表示它们形成一个 SCC。

最后考虑如何求 g 。

- 对于树边 $u \rightarrow v$ ，用 g_v 更新 g_u 。
- 对于前向边 $u \rightarrow v$ ，用 $dfn(v)$ 更新 g_u 没有任何影响，因为 $dfn(v) \geq g_u$ 。
- 对于返祖边 $u \rightarrow v$ ，用 $dfn(v)$ 更新 g_u 。
- 对于横叉边 $u \rightarrow v$ ，若 v 不可达 d ，那么 v 和 d 不强连通，所以从 v 不断回溯至 d 的过程中一定有关键点弹出 v 。否则 v 可达 d ，那么 v 和 d 强连通，因为 d 没有被弹出，所以 v 也没有被弹出。但是直接这样说明会涉及 g 的正确性和弹出 SCC 的正确性之间的循环论证。注意到横叉边更新 g 的正确性只依赖于已经弹出的每个点集是 SCC，所以归纳假设已经弹出的每个点集为 SCC，这样当前弹出点集的每个点的 g 是正确的，从而保证了当前弹出点集也是 SCC。

对于返祖边 $u \rightarrow v$ ， v 显然没有被弹出。

综上，得到转移：对于树边 $u \rightarrow v$ ，用 g_v 更新 g_u 。对于非树边 $u \rightarrow v$ ，若 v 在栈内，则用 $dfn(v)$ 更新 g_u 。

SCC 缩点后，得到的新图不含环，否则环上所有结点对应的 SCC 可以形成一个更大的 SCC。这说明 SCC 缩点图是一张 DAG。

重要结论：对于两个 SCC S_1, S_2 ，若 S_1 可达 S_2 ，则 S_1 比 S_2 后弹出栈。按弹出顺序写下所有 SCC，得到缩点 DAG 的反拓扑序。因此，按编号从大到小遍历 SCC，就是按拓扑序遍历缩点 DAG。

以下是 [模板题](#) 题解。

因为可以多次经过同一个点，所以一旦进入某个 SCC，就一定可以走遍其中所有点，获得所有点权值之和的贡献。但是一旦离开 SCC，就没法再回来了，否则离开后遍历的点和当前 SCC 强连通。

因此，将有向图 SCC 缩点后得到 DAG，每个 SCC 缩点后的权值等于其中所有点的权值之和。问题转化为 DAG 最长带权路径，拓扑排序 DP 即可。时间复杂度 $\mathcal{O}(n + m)$ 。

```

#include <bits/stdc++.h>
using namespace std;
constexpr int N = 1e4 + 5;
int n, m, cn, col[N];
int a[N], val[N], f[N];
int top, stc[N], vis[N], dn, dfn[N], low[N];
vector<int> e[N], g[N];
void tarjan(int id) {
    vis[id] = 1, dfn[id] = low[id] = ++dn, stc[++top] = id;
    for(int it : e[id]) {
        if(!dfn[it]) tarjan(it), low[id] = min(low[id], low[it]); // 树边
        else if(vis[it]) low[id] = min(low[id], dfn[it]); // 在栈内则更新
    }
    if(dfn[id] == low[id]) {
        col[id] = ++cn;
        while(stc[top] != id) col[stc[top]] = cn, vis[stc[top--]] = 0; // 弹出栈内结点到 id 形成强连通分量
        vis[id] = 0, top--;
    }
}
int main() {
    cin >> n >> m;
    for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
    for(int i = 1; i <= m; i++) {
        int u, v;
        scanf("%d%d", &u, &v);
        e[u].push_back(v);
    }
    for(int i = 1; i <= n; i++) if(!dfn[i]) tarjan(i);
    for(int i = 1; i <= n; i++) {
        val[col[i]] += a[i];
        for(int it : e[i]) {
            if(col[i] != col[it]) g[col[i]].push_back(col[it]);
        }
    }
    int ans = 0;
    for(int i = cn; i; i--) { // 按编号顺序从大到小遍历就是按拓扑序遍历
        f[i] += val[i];
        ans = max(ans, f[i]);
        for(int it : g[i]) f[it] = max(f[it], f[i]);
    }
    cout << ans << endl;
    return 0;
}

```

4.4 Kosaraju

Kosaraju 的思想：利用 SCC 内任意两点相互可达的性质，先一次 DFS 求出缩点图的拓扑序，再一次 DFS 求出所有 SCC。

考虑能否通过时间戳求出拓扑序。设 SCC 的编号为它包含的所有点的最大时间戳，通过编号不能推导出关于拓扑序的信息：对于两个 SCC S_1, S_2 ， S_1 与 S_2 的拓扑序大小和编号大小之间没有联系。无论 S_1 的编号小于还是大于 S_2 ， S_1 均可能可达 S_2 。

时间戳不管用的本质原因在于，我们每访问到一个点就将其加入 DFS 序，这要求 DFS 的顺序就是合法拓扑序，但显然做不到：我们甚至不知道从哪里入手开始 DFS。

拓扑序是不现实的，只能寄希望于反拓扑序。如果我们将顺序改为离开每个点的次序，整个问题就豁然开朗了：离开 u 的时候，所有 u 可达的结点一定被遍历过了（这里可以看出为什么 SCC 需要被编号为最大离开时间，而不是最小或任意），所以若 S_1 可达 S_2 ，那么 S_1 的编号大于 S_2 。即若 S_1 的编号小于 S_2 ，那么 S_1 不可达 S_2 。

接下来是第二次 DFS。

从离开时间最大的结点开始 DFS，会访问到它所在的 SCC。为保证其它 SCC 不被访问到，将所有边的方向反过来。这不改变任意两点之间的强连通性，所以反图的 SCC 划分方案和原图相同，但过程中不会访问其它 SCC，因为它们的编号小于当前 SCC，在原图上不可达当前 SCC，因而在反图上不可由该 SCC 到达。

将遍历到的结点打包成 SCC 删去（标记为已访问），不断找到最大的未被访问的结点重复上述过程，即可得到原图的 SCC 划分。时间复杂度 $\mathcal{O}(n + m)$ 。

一般 SCC 缩点优先考虑 Tarjan，它只需要一次 DFS，而 Kosaraju 需要两次。Kosaraju 相比 Tarjan 的优势在于：整个过程只有 DFS 和标记。当图是稠密图时，可以 bitset 加速做到除预处理外 $\mathcal{O}(\frac{n^2}{w})$ 。而 Tarjan 是严格 $\mathcal{O}(n + m)$ 的，更新 g 必须考虑到每条边。例如：多次询问保留编号在一段区间的边的 SCC 数量，在特定的数据范围下，莫队 + bitset 优化 Kosaraju 比 Tarjan 优秀。

4.5 例题

P3436 [POI2006] PRO-Professor Szu

对于一个 SCC，如果它内部有边且可达 $n + 1$ ，则方案数为 $+\infty$ 。

SCC 缩点后，统计出所有这样的 SCC。设 f_i 表示编号为 i 的 SCC 到 $n + 1$ 所在 SCC 的方案数，建反图拓扑排序，若当前 SCC 内部有边且可达 $n + 1$ ，即 $f_i > 0$ ，那么令 $f_i \leftarrow 36501$ 。对于每条反边 $u \rightarrow v$ ，将 f_v 加上 f_u 后对 36501 取 min。

时间复杂度 $\mathcal{O}(n + m)$ 。[代码](#)。

P7737 [NOI2021] 庆典

首先 SCC 缩点。

缩点后删去重边，利用题目性质保证每个点的入度不超过 1：如果 $x \rightarrow z$ 且 $y \rightarrow z$ ，设 $x \rightarrow y$ ，那么删去 $x \rightarrow z$ ，可达性不变。这说明图的可达性可以用一棵叶向树刻画。

称给定的 $2(k+1)$ 个点为关键点。称关键点 i 是好点当且仅当 $s \rightsquigarrow i$ 且 $i \rightsquigarrow t$ 。一个点可以经过当且仅当其祖先和后代均存在好点。

求好点的方法：设新图 G ，初始为空。对于关键点 i, j ，若 i 是 j 的祖先或新加边 $i \rightarrow j$ ，则在 G 上连边 $i \rightarrow j$ 。 s 在 G 上可达的所有点就是 s 可达的关键点。对于 t 同理。复杂度 $\mathcal{O}(k^2)$ 。

法一：对于好点 i, j 且 i 是 j 的祖先， $i \rightsquigarrow j$ 上所有城市均可以经过，树剖求链并。时间复杂度 $\mathcal{O}(nk^2 \log n \log \log n)$ 。

法二：用类似虚树的方法统计答案，保证栈内是一条好点构成的从上到下的链。注意，使用 d 替换栈顶时，若 d 不等于栈顶且栈的大小为 1，说明 d 不是好点，直接清空栈。时间复杂度 $\mathcal{O}(nk^2)$ 。[代码](#)。

直接建虚树，加边 DFS，然后在虚树上统计答案，时间复杂度 $\mathcal{O}(nk \log k)$ ，但常数较大。

*ARC092D Two Faced Edges

对于连接 SCC S_1, S_2 之间的边，反向后使得 SCC 数量减少当且仅当 S_1 在 DAG 上能够不经过这条边到达 S_2 。 $\mathcal{O}(nm)$ 求出任意两点之间的路径数对 2 取 min 判断。

对于 SCC 内部的边，若反向 $u \rightarrow v$ 后 u 不可达 v ，那么 u, v 不强连通，SCC 数量改变。否则 u 可达 v ，可达性不弱于原图可达性，不影响点对之间的可达性。

因此，反向 $u \rightarrow v$ 后强连通性不变的充要条件为：反向 $u \rightarrow v$ 后 u 可达 v 。因为反向边 $v \rightarrow u$ 显然无用，故等价于删去 $u \rightarrow v$ 后 u 可达 v 。

问题转化为对每个点的每条出边 $u \rightarrow v_i$ 求出在 u 在不经过 $u \rightarrow v_i$ 的情况下可达哪些点。考虑到 u 保留的出边是一段前缀 $v_{1 \sim i-1}$ 和后缀 $v_{i+1 \sim k}$ 的形式，且路径不会经过同一个点两次，所以 $u \rightarrow v_i$ 合法当且仅当保留 $u \rightarrow v_{1 \sim i-1}$ 或 $u \rightarrow v_{i+1 \sim k}$ 时， u 可达 v 。

正反扫一遍出边，每次从 v_i 开始 DFS 并记录每个点的可达性，并在扫当前出边 $u \rightarrow v_i$ 之前，若 u 可达 v_i 则 $u \rightarrow v_i$ 合法。总时间复杂度为 $\mathcal{O}(nm)$ 。

注意：我们考虑的是保留 u 的一段前缀或后缀的出边时每个点的可达情况，因此当 DFS 到 u 时需及时返回。

进一步地，发现对于连接 SCC 之间的边 $u \rightarrow v$ ，我们也希望判断去掉该边后 u 能否到达 v 。这和 SCC 内部的边的形式一致。只不过对于不同 SCC 之间的边，若 $u \not\rightsquigarrow v$ 则反转 $u \rightarrow v$ 不会改变 SCC 个数，反之则改变，这与 SCC 内部的边相反。这样就不需要显式建出 DAG 了。

对整张图进行 $O(nm)$ 的深搜会 TLE，我们需要更加高效的算法。Bitset 优化图的遍历是经典套路：每次找到 u 的所有出边中第一个没有被访问的点。具体地，设 vis 表示每个结点是否访问过， $e[u]$ 表示 u 的所有出边，则 $(\sim vis \ \& \ e[u])._Find_first()$ 即为所求。时间复杂度 $O(\frac{n^3}{w})$ 。代码。

启示：序列去掉一个位置的信息可由前缀和后缀合并得到。

*CF1361E James and the Chase

很好的强连通题，需要对有向图 DFS 树有一定理解。

因为题目保证图强连通，所以从任意一个点出发 DFS 可达图上所有点。

判定一个点是否合法：以该点为根的任意 DFS 树只有树边和返祖边：横叉边和前向边将导致大于一条简单路径。

首先找到任意合法点：随机 100 个点判定是否合法。如果合法点的数量不少于 $0.2n$ ，那么单组测试点正确率不低于 $(1 - 0.8^{100})^{1000} \approx 1 - 2 \times 10^{-7}$ 。然后根据没有横插边和前向边的性质，求出其它每个点是否合法。

性质：点 x 到它子树内的点有且仅有一条简单路径，因为一个点向下走的唯一途径是树边，向上走的唯一途径是返祖边。除了一定存在的一直向下走的简单路径，如果还有别的简单路径，则这条路径一定走了一条返祖边。这条返祖边一定指向 x 的祖先，因为如果指向 x 子树内的结点，那么由于走返祖边之前必须经过该结点，所以路径经过了重复点，不合法。而从 x 的祖先回到目标点必须再次经过 x ，也不合法，所以这样的简单路径不存在。这说明，如果 x 到 y 有大于一条简单路径，那么 y 不在 x 的子树内，且这样的简单路径一定有且仅有一步从 x 的子树内跳了出去。

称一条返祖边 $u \rightarrow v$ 覆盖 u, v 的树上简单路径的除了 v 以外的所有点，一个点可以通过覆盖它的返祖边离开它的子树。

如果点 x 被大于一条边覆盖，那么它就有两种方法离开子树，然后向下走到它的父亲，即 x 到 x 的父亲的简单路径数量大于 1， x 不合法。

如果 x 只有一种方法离开子树，设对应返祖边为 $u \rightarrow v$ 。因为不存在同时覆盖 x 和 v 的返祖边，也就是覆盖 v 的返祖边的较深一端一定为 x 的祖先，所以从 x 经过 u 走到 v 之后，如果 v 到其它点有大于一条简单路径，因为这样的路径一定会跳出 v 的子树，所以一定不会经过 x 以及它的子树，将 x 到 v 的路径接上这样的路径不会使得路径经过重复点。相反，如果 v 到其它所有点有且仅有一条简单路径，那么容易通过反证法证明 x 同样如此。

又因为一个非根节点必须能够离开子树（否则不满足原图强连通的性质），所以每个点被至少一条返祖边覆盖。因此，点 x 合法的充要条件是： x 为根，或者有且仅有一条返祖边 $u \rightarrow v$ 覆盖 x 且 v 合法。

用树上差分维护每个点被多少条边覆盖，再用树形 DP 求出每个点最浅能够跳到哪个祖先，最后从上往下 DP 求出每个点是否合法，时间复杂度 $\mathcal{O}(n + m)$ 。

总时间复杂度 $\mathcal{O}(c \cdot (n + m))$ ，其中 c 是随机次数。[代码](#)。

5. 欧拉回路

小学奥数之一笔画问题。

5.1 相关定义

- 欧拉路径：经过连通图中所有边恰好一次的迹称为 **欧拉路径**。
- 欧拉回路：经过连通图中所有边恰好一次的回路称为 **欧拉回路**。
- 欧拉图：有欧拉回路的图称为 **欧拉图**。
- 半欧拉图：有欧拉路径但没有欧拉回路的图称为 **半欧拉图**。

欧拉图能够以任意一点作为起点一笔画整张图然后回到该点，而半欧拉图只能从 S 开始， $T \neq S$ 结束一笔画。

5.2 欧拉图的判定

5.2.1 有向图

考察有向欧拉图 G 的性质。

首先 G 显然弱连通。此外，回路上从每个点出发的边的数量和到达该点的边的数量相等。因此 G 的每个点的入度等于出度。

对于满足上述条件的图 G ，从任意一点 u 出发不断走当前没走过的边，最后回到 u ，得到任意回路 C 。我们不会在其它点 $v \neq u$ 停下来，因为此时 v 的已经走过的入边数量大于已经走过的出边数量，所以总存在没走过的出边。

将 C 的边从 G 中删去，并删去所有度数为 0 的点，得到若干弱连通子图 G_i 。 G_i 仍然满足每个点的入度等于出度，且和 C 有交点（根据 G 的弱连通性易证）。假设 G_i 存在欧拉回路，那么将每个 G_i 的欧拉回路分别插入 C ，即得 G 的欧拉回路。

综上，可归纳证明有向欧拉图的判定准则：

有向图 G 是欧拉图，当且仅当 G 弱连通且 G 的每个点的入度等于出度。

读者可类似推导有向半欧拉图的判定准则，这里不加证明地给出结论：

有向图 G 是半欧拉图，当且仅当 G 弱连通，存在两个点的入度分别等于出度减 1 和出度加 1，且其余每个点的入度等于出度。

入度等于出度减 1 的点为欧拉路径的起点，入度等于出度加 1 的点为欧拉路径的终点。

5.2.2 无向图

考察无向欧拉图 G 的性质。

首先 G 显然连通。此外，对于非起始点，欧拉回路每次进入该点后总会离开；对于起始点，欧拉回路每次离开该点后总会回来。每次进入和离开一个点均对其度数产生 1 的贡献，所以每个点的度数均为偶数。

对于满足条件的图 G ，从任意一点 u 出发不断走当前没走过的边，最后回到 u ，得到任意回路 C 。我们不会在其它点 $v \neq u$ 停下来，因为此时 v 的已经走过的邻边关于 v 的总度数为奇数，所以总存在没走过的邻边。如果将自环算成两条邻边，也可以理解为 v 的已经走过的邻边数量为奇数。

将 C 的边从 G 中删去，并删去所有度数为 0 的点，得到若干连通子图 G_i 。 G_i 仍满足每个点的度数为偶数，且和 C 有交点。假设 G_i 存在欧拉回路，那么将每个 G_i 的欧拉回路分别插入 C ，即得 G 的欧拉回路。

综上，可以归纳证明无向欧拉图的判定准则：

无向图 G 是欧拉图，当且仅当 G 连通且 G 的每个点的度数为偶数。

读者可类似推导无向半欧拉图的判定准则，这里不加证明地给出结论：

无向图 G 是半欧拉图，当且仅当 G 连通且恰存在两个点的度数为奇数。

两个奇度点分别为欧拉路径的起点和终点。

5.2.3 混合图

给定图 G ， G 有若干无向边和有向边，称为混合图。

忽略掉将所有有向边改为无向边后图不连通的情况，此时 G 显然不存在欧拉回路。

考虑钦定无向边的方向，将 G 转化为有向图。有向图存在欧拉回路的充要条件是弱连通且每个点的入度等于出度，第一个条件已经满足，要求满足第二个条件。

注意到每个点的入度加出度固定，就是它在 G 上的度数。所以，若原图存在奇度数点，无解。否则限制转化为要求每个点的出度为 $\frac{d(i)}{2}$ 。若只考虑有向边时出度或入度已经大于 $\frac{d(i)}{2}$ ，无解。否则要求有 $\frac{d(i)}{2} - d^+(i)$ 条将无向边定向后从 i 出发的有向边。

每条无向边要么给 $d^+(u)$ 加 1，要么给 $d^+(v)$ 加 1。经典网络流模型：将所有无向边 $e_i = (u_i, v_i)$ 抽象为左部点，所有点作为右部点。源点 S 向左部点连容量 1 的边。左部点 e_i 向右部点 u_i 和 v_i 分别连容量 1 的边。右部点 i 向汇点连容量 $\frac{d(i)}{2} - d^+(i)$ 的边。若最大流为无向边总数，则有解，且当前流给出了一种定向方案。否则无解。

对于半欧拉图判定， G 必须存在两个奇度点。枚举哪个奇度点作为起点，然后类似跑网络流即可。

5.3 Hierholzer

在研究欧拉图的判定条件时，我们通过构造一条合法的欧拉回路证明了判定条件的充要性。而构造的方法就可以用来求欧拉回路。

先考虑有向图，因为无向图需要处理重边的问题。

Hierholzer 的核心是不断往当前回路的某个点中插入环。具体地，从任意一点出发 DFS，找到任意一个回路 C 。删去 C 上的所有边，然后依次加入 C 的每个结点 $p_1, p_2, \dots, p_{|C|}, p_1$ 。加入 p_i 之前，我们先递归找到 p_i 所在连通子图的欧拉回路，将其插入 C 。这是一个递归形式的算法。

使用链式前向星维护每个点的剩余出边（类似 Dinic 的当前弧优化），每条边只会被遍历一次。用双向链表维护回路的合并，时间复杂度 $\mathcal{O}(n + m)$ 。

做法的时间复杂度优秀，但实现起来有些复杂。我们可以先理解复杂的方法做了些什么，再思考有哪些地方可以简化。

从整体上考察，我们实现了这样的步骤：从一个起点开始找到一个回路，然后以环上的每个点为起点找到一个回路，不断递归下去。为了依次从回路上的每个点开始找回路，需要显式地求出这个回路，再依次处理上面的所有结点。所以我们需要一个 DFS 函数找回路，另一个递归式函数求欧拉回路。

但显式地求出回路是不必要的：我们可以按任意顺序从当前回路上的点出发找回路。考虑在 DFS 找回路的回溯过程中直接对当前回路上的每个点找回路。换言之，将原本找回路的顺序倒过来，这样就没有必要显式地求出当前回路，而是在回溯的过程中，一边对当前点找回路，一边往回路中插入当前回路。

综上，我们得到求欧拉回路的最常用算法——Hierholzer 的具体步骤：遍历当前点 u 的所有出边 $u \rightarrow v$ 。若该边未走过，则向点 v DFS。遍历完所有出边后，将 u 加入回路。最终得到一条反着的欧拉回路（图有向，且倒序加入环），将其翻转即可。

若要求字典序最小，只需在一开始将每个点的所有出边从小到大排序，并从编号最小的结点开始 DFS。这样，在欧拉回路上从左往右看，每个点都取到了理论最小的编号。

对于无向图欧拉回路，和有向图一样做。注意判重边，使用求割边时的成对变换技巧，用 $2k$ 和 $2k + 1$ 作编号存储一条边的两个方向。注意不能只判父亲，因为可能有重边。

对于有向图和无向图的欧拉路径，从出度等于入度加 1 的点或奇度数点开始 DFS，其它部分和欧拉回路没有区别。

在具体实现中，需要先判定再求解，否则求出的欧拉路径有误：存在路径上相邻的两点在图上不相邻。

模板题 代码。

```
#include <bits/stdc++.h>
using namespace std;
constexpr int N = 2e5 + 5;
int n, m, lar;
int top, stc[N], in[N], hd[N]; // hd 用于当前弧优化
vector<int> e[N];
void dfs(int id) {
    for(int &i = hd[id]; i < e[id].size(); ) dfs(e[id][i++]);
    stc[++top] = id;
}
int main() {
    cin >> n >> m;
    for(int i = 1; i <= m; i++) {
        int u, v;
        scanf("%d%d", &u, &v);
        e[u].push_back(v), in[v]++;
    }
    for(int i = 1; i <= n; i++) {
        sort(e[i].begin(), e[i].end());
        if(abs(int(e[i].size()) - in[i]) > 1) puts("No"), exit(0); // 注意 e[i].size() 要强制转成 int
        if(e[i].size() > in[i]) {
            if(lar) puts("No"), exit(0);
            else lar = i;
        }
    }
    dfs(lar ? lar : 1);
    if(top != m + 1) puts("No"); // 图不连通
    else {
        reverse(stc + 1, stc + top + 1);
        for(int i = 1; i <= top; i++) cout << stc[i] << " ";
    }
    return 0;
}
```

5.4 例题

P2731 [USACO3.3] 骑马修栅栏 Riding the Fences

无向图最小字典序欧拉路径板子题。

代码。

P1127 词链

从每个字符串的第一个字符向最后一个字符连边，跑有向图欧拉回路。

注意，对邻接链表排序要按照每条边对应字符串的字典序排序，而非指向结点的编号大小。

时间复杂度 $\mathcal{O}(S \log S)$ ，其中 $S = \sum |s_i|$ 。代码。

P3520 [POI2011] SMI-Garbage

设 $f(i)$ 表示以 i 为一端的需要经过的边的数量。对于一条回路，所有点度数均为偶数，因此一次操作后 $f(i)$ 的奇偶性不变。若存在 $f(i)$ 为奇数，则无解。否则，注意到有解是需要经过的边形成的图的每个连通分量均存在欧拉回路的充要条件，对这张图跑一遍欧拉回路。因为要求不能经过相同的点，而路径上相同的点之间也一定是回路，所以借助栈把相同的点之间的回路弹出。

时间复杂度 $\mathcal{O}(n + m)$ 。代码。

*P3443 [POI2006] LIS-The Postman

题目保证无重边，所以每条路径片段的限制形如片段中相邻的两条边必须先后走。

所有限制关系形成若干条边先后走的链。将每条链缩成起点到终点的一条边，然后跑欧拉回路，最后输出这条边时需要重新展开成链。

若限制关系出现环或分叉则无解，并查集 + 链表维护。

时间复杂度是离散化的 $\mathcal{O}(m \log m)$ 。代码。

P3511 [POI2010] MOS-Bridges

若存在 $d(i)$ 为偶数则无解。

二分答案转混合图欧拉回路判定，构建二分图后网络流即可。

时间复杂度 $\mathcal{O}(m\sqrt{m} \log V)$ 。代码。

CF1361C Johnny and Megan's Necklace

枚举权值 k 。若答案不小于 k ，那么存在一组合法方案使得新添加的线连接的相邻两个珠子模 2^k 的余数相同。将每个同余类抽象成一个点，原有的线相当于一条连接 $a \bmod 2^k$ 和 $b_i \bmod 2^k$ 的边。在这张图上跑无向图欧拉回路即可。

时间复杂度 $\mathcal{O}(n \log V)$ 。

*CF1458D Flip and Reverse

很棒的题目。

将 01 写成折线图，即 $0 \rightarrow -1$, $1 \rightarrow 1$ 做前缀和得到 v_i 。

考虑一次操作的本质：它相当于选取折线图上等高的两点，水平翻转。这不改变折线上每个线段连接的点的高度。

又因为如果 $v_i \rightarrow v_{i+1}$ 向上走，那么一定有向下回到高度 v_i 的边，可以感知到翻转的自由度很大。

因此，在 v_i 和 v_{i+1} 之间连无向边，求出 $v_0 \rightarrow v_n$ 的字典序最小的欧拉路径就是答案。

证明也很容易，根据上一段提到的性质，分成“右侧有相同高度”和“右侧没有相同高度”两种情况讨论即可。

时间复杂度 $\mathcal{O}(n)$ 。

CHANGE LOG

- 2021.12.5：修改例题代码与部分表述，增加基础定义。
- 2022.4.22：重构文章。
- 2022.5.21：进行一些增补，添加 Floyd 算法和 SCC 缩点。
- 2022.5.25：添加 Hierholzer 算法。
- 2022.8.15：修正强连通分量部分的事实性错误。
- 2022.11.1：重构文章。
- 2023.4.10：重写最短路和差分约束，添加例题代码。
- 2023.4.29：重写无向图连通性部分，添加说明。
- 2023.6.21：重写有向图可达性部分，添加 Kosaraju 算法。
- 2023.7.3：添加用拟阵理解生成树的部分。
- 2023.7.4：添加生成树相关的扩展问题。
- 2023.7.6：添加同余最短路。
- 2023.7.31：添加边双连通分量缩点的代码，添加点双性质进阶，修改小错误。
- 2023.8.5：补充说明。
- 2023.8.25：修改 typo，添加例题 ABC232G。
- 2023.8.28：添加例题 CF1361E。
- 2023.9.11：添加基图定义。
- 2024.8.9：修订文章，添加一些图片。
- 2024.8.13：修正事实性错误（3.2.5 小节添加无自环的要求）。

参考资料

定义

- [图论相关概念](#) —— OI Wiki。
- [图论概念梳理](#) —— OI 中转站。

第一章

- [Dijkstra 算法描述及正确性证明](#) —— ciwei。
- [Floyd 算法正确性](#) —— dypdypdyp123。
- [【图论】传递闭包的概念](#) —— zkq_1986。
- [删边最短路问题](#) —— dengyaotriangle。
- [堆的可持久化](#) —— 俞鼎力。
- [可持久化线段树求 k 短路](#) —— ix35。

第二章

- [最小生成树](#) —— OI Wiki。
- [怎么理解拟阵 \(matroid\)](#) —— SleepyBag。
- [拟阵及应用](#) —— 好地方 bug。
- 2018 集训队论文《浅谈拟阵的一些拓展及其应用》—— 杨乾澜。

第三章

- [Menger's theorem](#) —— Wikipedia。
- [k-vertex-connected graph](#) —— Wikipedia。
- [k-edge-connected graph](#) —— Wikipedia。
- [双连通 / 圆方树 胡扯笔记](#) —— ycx060617。

第四章

- [Tarjan 算法求 SCC 学习笔记](#) —— ycx060617。

第五章

- [欧拉回路与欧拉通路存在性的充要条件及其证明](#)。
- [欧拉图](#) —— OI Wiki。